Transformer_Captioning

November 14, 2026

Generative AI Use: For the purposes of the assignments, the use of generative AI is subject to the same policies regarding collaboration. Just as with other collaborators, each student must write down the solutions independently of the output of the interaction and the submission should include a note denoting the nature of the collaboration. The use of generative AI tools to substantially complete sections of the assignments is not in line with the spirit of the assignments, and would be a violation of the Honor Code.

Student Declaration (must be completed)

SUNet id:
4NI23CI053

Did you use generative AI tools (e.g., ChatGPT, Copilot, etc.)?
Yes

If yes, describe how you used generative AI and for what parts of the assignment :
I used ChatGPT & Claude to understand the assignment concepts, debug errors in my code, and clarify implementation details. I also used it to understand model architectures, interpret training and validation results, and draft answers to the inline questions. I reviewed and tested the code myself while completing the assignment.

Confirm that all submitted work reflects your own understanding and that you did not rely on AI to complete substantial portions of the assignment:
- [ ] I confirm

[1]:
```
# This mounts your Google Drive to the Colab VM.
from google.colab import drive
drive.mount('/content/drive')

# TODO: Enter the foldername in your Drive where you have saved the unzipped
# assignment folder, e.g. 'cs231n/assignments/assignment3/'
FOLDERNAME = 'cs231n/assignments/assignment3/'
assert FOLDERNAME is not None, "[!] Enter the foldername."

# Now that we've mounted your Drive, this ensures that
# the Python interpreter of the Colab VM can load
# python files from within it.
import sys
sys.path.append('/content/drive/My Drive/{}'.format(FOLDERNAME))
```

```
# This downloads the COCO dataset to your Drive
# if it doesn't already exist.
%cd /content/drive/My\ Drive/$FOLDERNAME/cs231n/datasets/
# !bash get_datasets.sh
!bash get_coco_dataset.sh
%cd /content/drive/My\ Drive/$FOLDERNAME
```

```
Mounted at /content/drive
/content/drive/My Drive/cs231n/assignments/assignment3/cs231n/datasets
--2026-11-14 11:29:37--  http://cs231n.stanford.edu/coco_captioning.zip
Resolving cs231n.stanford.edu (cs231n.stanford.edu)… 171.64.64.21
Connecting to cs231n.stanford.edu (cs231n.stanford.edu)|171.64.64.21|:80…
connected.
HTTP request sent, awaiting response… 301 Moved Permanently
Location: https://cs231n.stanford.edu/coco_captioning.zip [following]
--2026-11-14 11:29:37--  https://cs231n.stanford.edu/coco_captioning.zip
Connecting to cs231n.stanford.edu (cs231n.stanford.edu)|171.64.64.21|:443…
connected.
HTTP request sent, awaiting response… 200 OK
Length: 1035210391 (987M) [application/zip]
Saving to: ‘coco_captioning.zip’

coco_captioning.zip 100%[===================>] 987.25M  32.0MB/s    in 22s

2026-11-14 11:29:59 (44.2 MB/s) - ‘coco_captioning.zip’ saved
[1035210391/1035210391]

Archive:  coco_captioning.zip
   creating: coco_captioning/
  inflating: coco_captioning/coco2014_captions.h5
  inflating: coco_captioning/coco2014_vocab.json
  inflating: coco_captioning/train2014_images.txt
  inflating: coco_captioning/train2014_urls.txt
  inflating: coco_captioning/train2014_vgg16_fc7.h5
  inflating: coco_captioning/train2014_vgg16_fc7_pca.h5
  inflating: coco_captioning/val2014_images.txt
  inflating: coco_captioning/val2014_urls.txt
  inflating: coco_captioning/val2014_vgg16_fc7.h5
  inflating: coco_captioning/val2014_vgg16_fc7_pca.h5
/content/drive/My Drive/cs231n/assignments/assignment3
```

1 Image Captioning with Transformers

You have now implemented a vanilla RNN and for the task of image captioning. In this notebook you will implement key pieces of a transformer decoder to accomplish the same task.

NOTE: This notebook will be primarily written in PyTorch rather than NumPy, unlike the RNN notebook.

[2]:
```
# Setup cell.
import time, os, json
import numpy as np
import matplotlib.pyplot as plt
import sys
import types
import importlib

from cs231n.gradient_check import eval_numerical_gradient,␣
 ↪eval_numerical_gradient_array
from cs231n.transformer_layers import *
from cs231n.captioning_solver_transformer import CaptioningSolverTransformer
from cs231n.classifiers.transformer import CaptioningTransformer
from cs231n.coco_utils import load_coco_data, sample_coco_minibatch,␣
 ↪decode_captions
from cs231n.image_utils import image_from_url

%matplotlib inline
plt.rcParams['figure.figsize'] = (10.0, 8.0) # Set default size of plots.
plt.rcParams['image.interpolation'] = 'nearest'
plt.rcParams['image.cmap'] = 'gray'

if "imp" not in sys.modules:
    imp = types.ModuleType("imp")
    imp.reload = importlib.reload
    sys.modules["imp"] = imp

%load_ext autoreload
%autoreload 2

def rel_error(x, y):
    """ returns relative error """
    return np.max(np.abs(x - y) / (np.maximum(1e-8, np.abs(x) + np.abs(y))))
```

2 COCO Dataset

As in the previous notebooks, we will use the COCO dataset for captioning.

[3]:
```
# Load COCO data from disk into a dictionary.
data = load_coco_data(pca_features=True)

# Print out all the keys and values from the data dictionary.
for k, v in data.items():
    if type(v) == np.ndarray:
        print(k, type(v), v.shape, v.dtype)
    else:
        print(k, type(v), len(v))
```

```
base dir  /content/drive/MyDrive/cs231n/assignments/assignment3/cs231n/datasets/
coco_captioning
train_captions <class 'numpy.ndarray'> (400135, 17) int32
train_image_idxs <class 'numpy.ndarray'> (400135,) int32
val_captions <class 'numpy.ndarray'> (195954, 17) int32
val_image_idxs <class 'numpy.ndarray'> (195954,) int32
train_features <class 'numpy.ndarray'> (82783, 512) float32
val_features <class 'numpy.ndarray'> (40504, 512) float32
idx_to_word <class 'list'> 1004
word_to_idx <class 'dict'> 1004
train_urls <class 'numpy.ndarray'> (82783,) <U63
val_urls <class 'numpy.ndarray'> (40504,) <U63
```

3 Transformer

As you have seen, RNNs are incredibly powerful but often slow to train. Further, RNNs struggle to encode long-range dependencies (though LSTMs are one way of mitigating the issue). In 2017, Vaswani et al introduced the Transformer in their paper "Attention Is All You Need" to a) introduce parallelism and b) allow models to learn long-range dependencies. The paper not only led to famous models like BERT and GPT in the natural language processing community, but also an explosion of interest across fields, including vision. While here we introduce the model in the context of image captioning, the idea of attention itself is much more general.

4 Transformer: Multi-Headed Attention

4.0.1 Dot-Product Attention

Recall that attention can be viewed as an operation on a query $q \in \mathbb{R}^d$, a set of value vectors $\{v_1, \ldots, v_n\}, v_i \in \mathbb{R}^d$, and a set of key vectors $\{k_1, \ldots, k_n\}, k_i \in \mathbb{R}^d$, specified as

$$c = \sum_{i=1}^{n} v_i \alpha_i \tag{1}$$

$$\alpha_i = \frac{\exp(k_i^\top q)}{\sum_{j=1}^{n} \exp(k_j^\top q)} \tag{2}$$

$$\tag{3}$$

where α_i are frequently called the "attention weights", and the output $c \in \mathbb{R}^d$ is a correspondingly weighted average over the value vectors.

4.0.2 Self-Attention

In Transformers, we perform self-attention, which means that the values, keys and query are derived from the input $X \in \mathbb{R}^{\ell \times d}$, where ℓ is our sequence length. Specifically, we learn parameter matrices $V, K, Q \in \mathbb{R}^{d \times d}$ to map our input X as follows:

$$v_i = Vx_i \quad i \in \{1, \ldots, \ell\} \tag{4}$$
$$k_i = Kx_i \quad i \in \{1, \ldots, \ell\} \tag{5}$$
$$q_i = Qx_i \quad i \in \{1, \ldots, \ell\} \tag{6}$$

4.0.3 Multi-Headed Scaled Dot-Product Attention

In the case of multi-headed attention, we learn a parameter matrix for each head, which gives the model more expressivity to attend to different parts of the input. Let h be number of heads, and Y_i be the attention output of head i. Thus we learn individual matrices Q_i, K_i and V_i. To keep our overall computation the same as the single-headed case, we choose $Q_i \in \mathbb{R}^{d \times d/h}$, $K_i \in \mathbb{R}^{d \times d/h}$ and $V_i \in \mathbb{R}^{d \times d/h}$. Adding in a scaling term $\frac{1}{\sqrt{d/h}}$ to our simple dot-product attention above, we have

$$Y_i = \text{softmax}\left(\frac{(XQ_i)(XK_i)^\top}{\sqrt{d/h}}\right)(XV_i) \tag{7}$$

where $Y_i \in \mathbb{R}^{\ell \times d/h}$, where ℓ is our sequence length.

In our implementation, we apply dropout to the attention weights (though in practice it could be used at any step):

$$Y_i = \text{dropout}\left(\text{softmax}\left(\frac{(XQ_i)(XK_i)^\top}{\sqrt{d/h}}\right)\right)(XV_i) \tag{8}$$

Finally, then the output of the self-attention is a linear transformation of the concatenation of the heads:

$$Y = [Y_1; \ldots; Y_h]A \tag{9}$$

were $A \in \mathbb{R}^{d \times d}$ and $[Y_1; \ldots; Y_h] \in \mathbb{R}^{\ell \times d}$.

Implement multi-headed scaled dot-product attention in the `MultiHeadAttention` class in the file `cs231n/transformer_layers.py`. The code below will check your implementation. The relative error should be less than `7e-3`.

[4]:
```
torch.manual_seed(231)

# Choose dimensions such that they are all unique for easier debugging:
# Specifically, the following values correspond to N=1, H=2, T=3, E//H=4, and␣
 ↪E=8.
batch_size = 1
sequence_length = 3
embed_dim = 8
attn = MultiHeadAttention(embed_dim, num_heads=2)
attn.eval()

# Self-attention.
```

```
data = torch.randn(batch_size, sequence_length, embed_dim)
self_attn_output = attn(query=data, key=data, value=data)

# Masked self-attention.
mask = torch.randn(sequence_length, sequence_length) < 0.5
masked_self_attn_output = attn(query=data, key=data, value=data, attn_mask=mask)

# Attention using two inputs.
other_data = torch.randn(batch_size, sequence_length, embed_dim)
attn_output = attn(query=data, key=other_data, value=other_data)

expected_self_attn_output = np.asarray([[
    [-0.1030,  0.2090,  0.7481, -0.4116, -0.2772,  0.2357, -0.2740, -0.2798],
    [-0.1794,  0.1637,  0.6930, -0.3435, -0.2300,  0.2457, -0.2504, -0.2323],
    [-0.1861,  0.1567,  0.6677, -0.3078, -0.2731,  0.2302, -0.2714, -0.2082]]])

expected_masked_self_attn_output = np.asarray([[
    [-0.1209,  0.1806,  0.8056, -0.4654, -0.2262,  0.2552, -0.2544, -0.2812],
    [-0.2374,  0.1183,  0.7423, -0.3777, -0.1460,  0.2716, -0.2213, -0.2077],
    [-0.0851,  0.2423,  0.7941, -0.3778, -0.4924,  0.1512, -0.4241, -0.1740]]])

expected_attn_output = np.asarray([[
    [-0.1965,  0.0702,  0.5227, -0.1388, -0.3530,  0.2163, -0.2643, -0.1902],
    [-0.3592, -0.0899,  0.5836,  0.0029, -0.3643,  0.2667, -0.1973, -0.1698],
    [-0.3795,  0.0433,  0.5634, -0.0749, -0.4379,  0.2714, -0.2207, -0.1980]]])

print('self_attn_output error: ', rel_error(expected_self_attn_output,␣
 ↪self_attn_output.detach().numpy()))
print('masked_self_attn_output error: ',␣
 ↪rel_error(expected_masked_self_attn_output, masked_self_attn_output.detach().
 ↪numpy()))
print('attn_output error: ', rel_error(expected_attn_output, attn_output.
 ↪detach().numpy()))
```

```
self_attn_output error:  1.0
masked_self_attn_output error:  1.0
attn_output error:  1.0
```

5 Positional Encoding

While transformers are able to easily attend to any part of their input, the attention mechanism has no concept of token order. However, for many tasks (especially natural language processing), relative token order is very important. To recover this, the authors add a positional encoding to the embeddings of individual word tokens.

Let us define a matrix $P \in \mathbb{R}^{l \times d}$, where $P_{ij} = $

$$\begin{cases} \sin\left(i \cdot 10000^{-\frac{j}{d}}\right) & \text{if j is even} \\ \cos\left(i \cdot 10000^{-\frac{(j-1)}{d}}\right) & \text{otherwise} \end{cases}$$

Rather than directly passing an input $X \in \mathbb{R}^{l \times d}$ to our network, we instead pass $X + P$.

Implement this layer in `PositionalEncoding` in `cs231n/transformer_layers.py`. Once you are done, run the following to perform a simple test of your implementation. You should see errors on the order of `e-3` or less.

[6]:
```
torch.manual_seed(231)

batch_size = 1
sequence_length = 2
embed_dim = 6
data = torch.randn(batch_size, sequence_length, embed_dim)

pos_encoder = PositionalEncoding(embed_dim)
pos_encoder.eval()
output = pos_encoder(data)

expected_pe_output = np.asarray([[[-1.1106,  1.0014,  1.5280, -0.0778, -0.6964,␣
 ↪ 1.1455],
                                  [ 0.8125, -0.4303,  0.4981,  0.7320,  1.1380,␣
 ↪ 1.5331]]])
print('pe_output error: ', rel_error(expected_pe_output, output.detach().
 ↪numpy()))
```

```
pe_output error:  0.00021699471301977143
```

6 Inline Question 1

Several key design decisions were made in designing the scaled dot product attention we introduced above. Explain why the following choices were beneficial: 1. Using multiple attention heads as opposed to one. 2. Dividing by $\sqrt{d/h}$ before applying the softmax function. Recall that d is the feature dimension and h is the number of heads. 3. Adding a linear transformation to the output of the attention operation.

Only one or two sentences per choice is necessary, but be sure to be specific in addressing what would have happened without each given implementation detail, why such a situation would be suboptimal, and how the proposed implementation improves the situation.

Your Answer:

1. Multiple attention heads: They allow the model to focus on different parts of the input at the same time and capture different relationships. With only one head, the model may miss some important patterns.

2. Dividing by $\sqrt{d/h}$: It prevents dot products from becoming too large, which would make the softmax produce very sharp probabilities and slow down learning. Scaling keeps the values more

stable.

3. Linear transformation: It combines information from all attention heads into a useful representation. Without it, the model would have less flexibility in processing and combining the attention outputs.

7 Transformer Decoder Block

Transformer decoder layer consists of three modules: (1) self attention to process input sequence of vectors, (2) cross attention to process based on available context (i.e. image features in our case), (3) feedforward module to process each vector of the sequence independently. Complete the implementation of `TransformerDecoderLayer` in `cs231n/transformer_layers.py` and test it below. The relative error should be less than 1e-6.

The Transformer decoder layer has three main components: (1) a self-attention module that processes the input sequence of vectors, (2) a cross-attention module that incorporates additional context (e.g., image features in our case), and (3) a feedforward module that independently processes each vector in the sequence. Complete the implementation of `TransformerDecoderLayer` in `cs231n/transformer_layers.py` and test it below. The relative error should be less than 1e-6.

[7]:
```
torch.manual_seed(231)
np.random.seed(231)

N, T, TM, D = 1, 4, 5, 12

decoder_layer = TransformerDecoderLayer(D, 2, 4*D)
decoder_layer.eval()
tgt = torch.randn(N, T, D)
memory = torch.randn(N, TM, D)
tgt_mask = torch.randn(T, T) < 0.5

output = decoder_layer(tgt, memory, tgt_mask)

expected_output = np.asarray([
    [[ 0.27005595, -0.21056601,  0.63376445, -0.35391954,  0.60071295,
      -1.7566615,  -0.1314159,  -1.7659538,  -0.5575517,   0.680143,
       1.8622686,   0.7291234],
     [-0.6080135,   0.4421802,  -0.07590749, -0.12093157,  0.94722354,
      -1.0223433,   0.47332686, -1.3958666,   2.1890075,  -1.0380646,
       0.9437816,  -0.7343922],
     [-0.89690155,  0.60447216,  0.10688882,  0.03492759,  1.8388084,
      -0.9772293,   0.7621334,  -0.668234,    1.3260399,  -1.8417019,
       0.21718435, -0.506388],
     [-0.70961505, -0.02918372, -0.39894593, -0.8069395,   0.04967685,
       0.29345992,  0.7871747,  -1.3172938,   1.4744806,   1.2633642,
       1.1611108,  -1.767289]]
])
print('error: ', rel_error(expected_output, output.detach().numpy()))
```

```
error:  8.386729410631422e-08
```

8 Transformer for Image Captioning

Now that you have implemented the previous layers, you can combine them to build a Transformer-based image captioning model. Open the file `cs231n/classifiers/transformer.py` and look at the `CaptioningTransformer` class.

Implement the `forward` function of the class. After doing so, run the following to check your forward pass using a small test case; you should see error on the order of `e-5` or less.

[8]:
```
torch.manual_seed(231)
np.random.seed(231)

N, D, W = 4, 20, 30
word_to_idx = {'<NULL>': 0, 'cat': 2, 'dog': 3}
V = len(word_to_idx)
T = 3

transformer = CaptioningTransformer(
    word_to_idx,
    input_dim=D,
    wordvec_dim=W,
    num_heads=2,
    num_layers=2,
    max_length=30
)
transformer.eval()

features = torch.randn(N, D)
captions = torch.randint(0, V, (N, T))

scores = transformer(features, captions)

expected_scores = np.asarray([
    [[ 0.62165695, -0.6203744,  -1.116114  ],
     [ 0.6641097,  -0.60309553, -1.3479083 ],
     [ 0.62117404, -0.79205745, -1.1095243 ]],
    [[ 0.62160987, -0.6214724,  -1.1171186 ],
     [ 0.6093992,  -0.61166143, -1.3365762 ],
     [ 0.62108713, -0.79322916, -1.1104958 ]],
    [[ 0.6406587,  -0.57419956, -1.1300566 ],
     [ 0.6075228,  -0.60872054, -1.3372457 ],
     [ 0.6471472,  -0.8270494,  -1.1064028 ]],
    [[ 0.64029807, -0.5755552,  -1.1284486 ],
     [ 0.60708034, -0.6099963,  -1.3356377 ],
     [ 0.59367496, -0.8346836,  -1.0914496 ]]
])
```

```
print('scores error: ', rel_error(expected_scores, scores.detach().numpy()))
```

```
scores error:  1.4597957790730836e-07
```

9 Overfit Transformer Captioning Model on Small Data

Run the following to overfit the Transformer-based captioning model on the same small dataset as we used for the RNN previously.

[9]:
```
torch.manual_seed(231)
np.random.seed(231)

data = load_coco_data(max_train=50)

transformer = CaptioningTransformer(
          word_to_idx=data['word_to_idx'],
          input_dim=data['train_features'].shape[1],
          wordvec_dim=256,
          num_heads=2,
          num_layers=2,
          max_length=30
        )

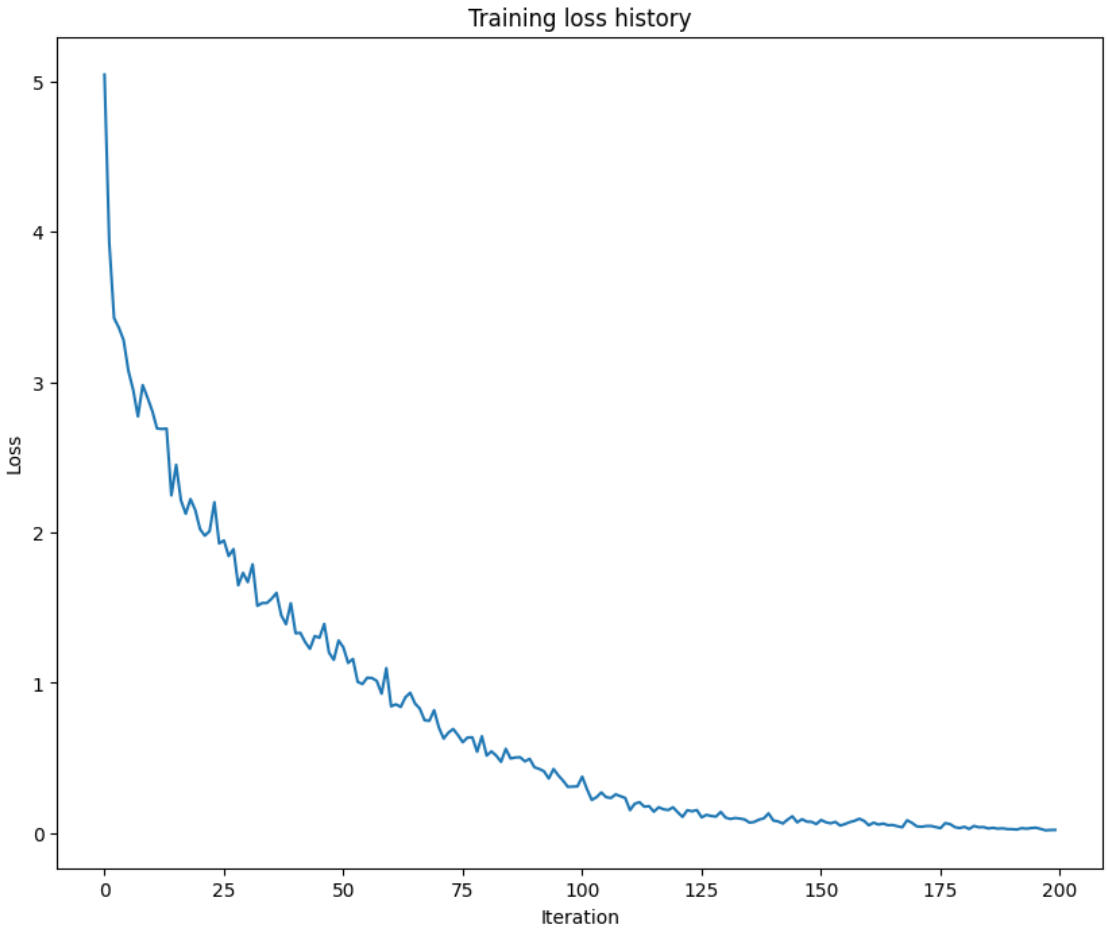

transformer_solver = CaptioningSolverTransformer(transformer, data,␣
 ↪idx_to_word=data['idx_to_word'],
           num_epochs=100,
           batch_size=25,
           learning_rate=0.001,
           verbose=True, print_every=10,
         )

transformer_solver.train()

# Plot the training losses.
plt.plot(transformer_solver.loss_history)
plt.xlabel('Iteration')
plt.ylabel('Loss')
plt.title('Training loss history')
plt.show()
```

```
base dir  /content/drive/MyDrive/cs231n/assignments/assignment3/cs231n/datasets/
coco_captioning
(Iteration 1 / 200) loss: 5.045327
(Iteration 11 / 200) loss: 2.808727
(Iteration 21 / 200) loss: 2.022354
(Iteration 31 / 200) loss: 1.670440
```

```
(Iteration 41 / 200) loss: 1.330231
(Iteration 51 / 200) loss: 1.236568
(Iteration 61 / 200) loss: 0.843831
(Iteration 71 / 200) loss: 0.702294
(Iteration 81 / 200) loss: 0.516776
(Iteration 91 / 200) loss: 0.438581
(Iteration 101 / 200) loss: 0.376332
(Iteration 111 / 200) loss: 0.153824
(Iteration 121 / 200) loss: 0.139354
(Iteration 131 / 200) loss: 0.104465
(Iteration 141 / 200) loss: 0.084865
(Iteration 151 / 200) loss: 0.087971
(Iteration 161 / 200) loss: 0.053218
(Iteration 171 / 200) loss: 0.046330
(Iteration 181 / 200) loss: 0.043136
(Iteration 191 / 200) loss: 0.027111
```



Print final training loss. You should see a final loss of less than 0.05 .

[10]:
```
print('Final loss: ', transformer_solver.loss_history[-1])
```

```
Final loss:  0.022365998
```

10 Transformer Sampling at Test Time

The sampling code has been written for you. You can simply run the following to compare with the previous results with the RNN. As before the training results should be much better than the validation set results, given how little data we trained on.

[11]:
```
# If you get an error, the URL just no longer exists, so don't worry!
# You can re-sample as many times as you want.
for split in ['train', 'val']:
    minibatch = sample_coco_minibatch(data, split=split, batch_size=2)
    gt_captions, features, urls = minibatch
    gt_captions = decode_captions(gt_captions, data['idx_to_word'])

    sample_captions = transformer.sample(features, max_length=30)
    sample_captions = decode_captions(sample_captions, data['idx_to_word'])

    for gt_caption, sample_caption, url in zip(gt_captions, sample_captions,␣
 ↪urls):
        img = image_from_url(url)
        # Skip missing URLs.
        if img is None: continue
        plt.imshow(img)
        plt.title('%s\n%s\nGT:%s' % (split, sample_caption, gt_caption))
        plt.axis('off')
        plt.show()
```

```
URL Error:  Gone http://farm1.staticflickr.com/202/487987371_489a65d670_z.jpg
```

train
a <UNK> decorated living room with a big tv in it <END>
GT:<START> a <UNK> decorated living room with a big tv in it <END>



URL Error: Not Found http://farm1.staticflickr.com/25/44101107_9491d72776_z.jpg

val
a man <UNK> with a bite on the busy while having his picture taken <END>
GT:<START> a group of people <UNK> outside by a wall <END>



11 Vision Transformer (ViT)

Dosovitskiy et. al. showed that applying a transformer model on a sequence of image patches (referred to as Vision Transformer) not only achieves impressive performance but also scales more effectively than convolutional neural networks when trained on large datasets. We will build a version of Vision Transformer using our existing implementation of transformer components and train it on the CIFAR-10 dataset.

Vision Transformer converts input image into a sequence of patches of fixed size and embed each patch into a latent vector. In `cs231n/transformer_layers.py`, complete the implementation of `PatchEmbedding` and test it below. You should see relative error less than 1e-4.

[12]:
```
from cs231n.transformer_layers import PatchEmbedding

torch.manual_seed(231)
np.random.seed(231)

N = 2
```

```
HW = 16
PS = 8
D = 8

patch_embedding = PatchEmbedding(
    img_size=HW,
    patch_size=PS,
    embed_dim=D
)

x = torch.randn(N, 3, HW, HW)
output = patch_embedding(x)

expected_output = np.asarray([
        [[-0.6312704 ,  0.02531429,  0.6112642 , -0.49089882,
          0.01412961, -0.6959372 , -0.32862484, -0.45402682],
        [ 0.18816411, -0.08142513, -0.9829535 , -0.23975623,
         -0.23109074,  0.97950286, -0.40997326,  0.7457837 ],
        [ 0.01810865,  0.15780598, -0.91804236,  0.36185235,
          0.8379501 ,  1.0191797 , -0.29667392,  0.20322265],
        [-0.18697818, -0.45137224, -0.40339014, -1.4381214 ,
         -0.43450755,  0.7651071 , -0.83683825, -0.16360264]],

       [[-0.39786366,  0.16201034, -0.19008337, -1.0602452 ,
         -0.28693503,  0.09791763,  0.26614824,  0.41781986],
        [ 0.35146567, -0.4469593 , -0.1841726 ,  0.45757473,
         -0.61304873, -0.29104248, -0.16124889, -0.14987172],
        [-0.2996967 ,  0.27353522, -0.09929767,  0.01973832,
         -1.2312065 , -0.6374332 , -0.22963578,  0.55696607],
        [-0.93818814,  0.02465284, -0.21117875,  1.1860403 ,
         -0.06137538, -0.21062079, -0.094347  ,  0.50032747]]])

print('error: ', rel_error(expected_output, output.detach().numpy()))
```

```
error:  9.182286955268188e-08
```

The sequence of patch vectors is processed by transformer encoder layers, each consisting of a self-attention and a feed-forward module. Since all vectors attend to one another, attention masking is not strictly necessary. However, we still implement it for the sake of consistency.

Implement `TransformerEncoderLayer` in `cs231n/transformer_layers.py` and test it below. You should see relative error less than 1e-6.

[13]:
```
torch.manual_seed(231)
np.random.seed(231)

from cs231n.transformer_layers import TransformerEncoderLayer
```

```
N, T, TM, D = 1, 4, 5, 12

encoder_layer = TransformerEncoderLayer(D, 2, 4*D)
encoder_layer.eval()
x = torch.randn(N, T, D)
x_mask = torch.randn(T, T) < 0.5

output = encoder_layer(x, x_mask)

expected_output = np.asarray([
    [[-0.48887438, -0.16470742,  0.7249629,  -1.0907345,   1.5764195,
       0.17484017,  0.8643941,  -0.5209561,  -1.6651545,  -1.1943513,
       1.4809005,   0.30326116],
     [-0.14663944,  1.052812,   -0.96227,    -0.20350897, -2.1478589,
       1.2637341,   0.04015125, -0.945255,    1.408903,   -0.43051198,
       0.37110543,  0.69933903],
     [ 0.08718029,  0.91050833, -1.208469,    1.1440394,  -1.9794976,
      -0.38342738, -0.47407156, -0.17345548,  0.11738186,  1.9534125,
       0.3525986,  -0.3461999],
     [-0.81389004,  0.76213264, -0.7094313,  -1.5332246,   1.3837544,
       0.4091982,   0.6569406,   0.7305365,  -1.197077,   -1.3698943,
       0.58292854,  1.0980265]]
])

print('error: ', rel_error(expected_output, output.detach().numpy()))
```

```
error:  4.162545799227941e-08
```

Take a look at the `VisionTransformer` implementation in `cs231n/classifiers/transformer.py`.

For classification, ViT divides the input image into patches and processes the sequence of patch vectors using a transformer. Finally, all the patch vectors are average-pooled and used to predict the image class. We will use the same 1D sinusoidal positional encoding to inject ordering information, though 2D sinusoidal and learned positional encodings are also valid choices.

Complete the ViT forward pass and test it below. You should see relative error less than 1e-6.

[14]:
```
torch.manual_seed(231)
np.random.seed(231)
from cs231n.classifiers.transformer import VisionTransformer

imgs = torch.randn(3, 3, 32, 32)
transformer = VisionTransformer()
transformer.eval()

scores = transformer(imgs)

expected_scores = np.asarray(
    [[-0.15802915,  0.15734261, -0.06002094, -0.1860142,  -0.09376919,
```

```
     -0.08837911,  0.16224845,  0.0168601,   0.20606893,  0.13079852],
    [-0.12838936,  0.20798579, -0.0620573,  -0.11705838, -0.14088857,
     -0.11387511,  0.18839046,  0.03078492,  0.17386372,  0.06414902],
    [-0.09435399,  0.20495278, -0.03362986, -0.12232997, -0.15535004,
     -0.10380758,  0.21114832,  0.03481449,  0.18798208,  0.12231653]])

print('scores error: ', rel_error(expected_scores, scores.detach().numpy()))
```

```
scores error:  2.3590082771647052e-07
```

We will first verify our implementation by overfitting it on one training batch. Tune learning rate and weight decay accordingly.

[15]:
```
from torchvision import transforms
from torchvision.datasets import CIFAR10
from tqdm.auto import tqdm
from torch.utils.data import DataLoader

train_data = CIFAR10(root='data', train=True, transform=transforms.ToTensor(),␣
 ↪download=True)
test_data = CIFAR10(root='data', train=False, transform=transforms.ToTensor(),␣
 ↪download=True)
```

```
100%|      | 170M/170M [00:38<00:00, 4.43MB/s]
```

[16]:
```
learning_rate = 1e-4  # Experiment with this
weight_decay = 1.e-4  # Experiment with this


batch = next(iter(DataLoader(train_data, batch_size=64, shuffle=False)))
model = VisionTransformer(dropout=0.0)
loss_criterion = torch.nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate,␣
 ↪weight_decay=weight_decay)
model.train()

epochs = 100
for epoch in range(epochs):
    imgs, target = batch
    out = model(imgs)
    loss = loss_criterion(out, target)

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    top1 = (out.argmax(-1) == target).float().mean().item()
    if epoch % 10 == 0:
```

```
    print(f"[{epoch}/{epochs}] Loss {loss.item():.6f}, Top-1 Accuracy: {top1:.
↪3f}")
```

```
[0/100] Loss 2.328464, Top-1 Accuracy: 0.094
[10/100] Loss 2.208234, Top-1 Accuracy: 0.188
[20/100] Loss 2.199911, Top-1 Accuracy: 0.188
[30/100] Loss 2.186795, Top-1 Accuracy: 0.188
[40/100] Loss 2.164992, Top-1 Accuracy: 0.188
[50/100] Loss 2.126720, Top-1 Accuracy: 0.203
[60/100] Loss 2.077387, Top-1 Accuracy: 0.250
[70/100] Loss 2.017539, Top-1 Accuracy: 0.266
[80/100] Loss 1.950788, Top-1 Accuracy: 0.266
[90/100] Loss 1.893538, Top-1 Accuracy: 0.250
```

[ ]:
```
# You should get perfect 1.00 accuracy
print(f"Overfitting ViT on one batch. Top-1 accuracy: {top1}")
```

Now we will train it on the entire dataset.

[17]:
```
from cs231n.classification_solver_vit import ClassificationSolverViT

################################################################################
# TODO: Train a Vision Transformer model that achieves over 0.45 test          #
# accuracy on CIFAR-10 after 2 epochs by adjusting the model architecture      #
# and/or training parameters as needed.                                        #
#                                                                              #
# Note: If you want to use a GPU runtime, go to `Runtime > Change runtime      #
# type` and set `Hardware accelerator` to `GPU`. This will reset Colab,        #
# so make sure to rerun the entire notebook from the beginning afterward.      #
################################################################################


learning_rate = 1e-4
weight_decay = 0.0
batch_size = 64
learning_rate = 5e-4
weight_decay = 1e-4
batch_size = 16
# Smaller patches give the model 8x8 = 64 tokens instead of 8x8 -> 4x4 = 16,␣
↪which
# helps a lot in the few updates that fit into 2 epochs; the rest of the␣
↪settings
# (depth 6, 4 heads, hidden 128, ff 256, no dropout) also widen the headroom.
model = VisionTransformer(
    patch_size=4,
    embed_dim=128,
    num_layers=6,
    num_heads=4,
```

```
    dim_feedforward=256,
    dropout=0.0,
) # You may want to change the default params.




################################################################################
#                                END OF YOUR CODE                              #
################################################################################

solver = ClassificationSolverViT(
    train_data=train_data,
    test_data=test_data,
    model=model,
    num_epochs = 2,  # Don't change this
    learning_rate = learning_rate,
    weight_decay = weight_decay,
    batch_size = batch_size,
)

solver.train('cuda' if torch.cuda.is_available() else 'cpu')
```

```
  0%|          | 0/3125 [00:00<?, ?it/s]
  0%|          | 0/625 [00:00<?, ?it/s]
  0%|          | 0/3125 [00:00<?, ?it/s]
  0%|          | 0/625 [00:00<?, ?it/s]
```

[18]:
```
print(f"Accuracy on test set: {solver.results['best_test_acc']}")
```

```
Accuracy on test set: 0.4941
```

12 Inline Question 2

Despite their recent success in large-scale image recognition tasks, ViTs often lag behind traditional CNNs when trained on smaller datasets. What underlying factor contribute to this performance gap? What techniques can be used to improve the performance of ViTs on small datasets?

Your Answer:

ViTs have weaker built-in inductive biases than CNNs, such as locality and translation invariance, so they require more data to learn these patterns. Their performance on small datasets can be improved using data augmentation, transfer learning with pretrained models, regularization, and hybrid CNN-ViT architectures.

13 Inline Question 3

How does the computational cost of the self-attention layers in a ViT change if we independently make the following changes? Please ignore the computation cost of QKV and output projection.

(i) Double the hidden dimension.
(ii) Double the height and width of the input image.
(iii) Double the patch size.
(iv) Double the number of layers.

Your Answer:

The computational cost of self-attention is $O(N^2D)$, where N is the number of patches and D is the hidden dimension.

1. **(i) Double hidden dimension:** Cost increases by **2×**.
2. **(ii) Double image height and width:** Number of patches becomes 4×, so cost increases by **16×**.
3. **(iii) Double patch size:** Number of patches becomes ¼, so cost decreases to **1/16×**.
4. **(iv) Double number of layers:** Cost increases by **2×**.

[ ]:

Self_Supervised_Learning

November 14, 2026

Generative AI Use: For the purposes of the assignments, the use of generative AI is subject to the same policies regarding collaboration. Just as with other collaborators, each student must write down the solutions independently of the output of the interaction and the submission should include a note denoting the nature of the collaboration. The use of generative AI tools to substantially complete sections of the assignments is not in line with the spirit of the assignments, and would be a violation of the Honor Code.

[1]:
```
# This mounts your Google Drive to the Colab VM.
from google.colab import drive
drive.mount('/content/drive')

# TODO: Enter the foldername in your Drive where you have saved the unzipped
# assignment folder, e.g. 'cs231n/assignments/assignment3/'
FOLDERNAME = "cs231n/assignments/assignment3/"
assert FOLDERNAME is not None, "[!] Enter the foldername."

# Now that we've mounted your Drive, this ensures that
# the Python interpreter of the Colab VM can load
# python files from within it.
import sys
sys.path.append('/content/drive/My Drive/{}'.format(FOLDERNAME))

# This downloads the COCO dataset to your Drive
# if it doesn't already exist.
%cd /content/drive/My\ Drive/$FOLDERNAME/cs231n/datasets/
!bash get_datasets.sh
%cd /content/drive/My\ Drive/$FOLDERNAME
```

```
Mounted at /content/drive
/content/drive/My Drive/cs231n/assignments/assignment3/cs231n/datasets
--2026-11-14 11:42:53--  http://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz
Resolving www.cs.toronto.edu (www.cs.toronto.edu)… 128.100.3.30
Connecting to www.cs.toronto.edu (www.cs.toronto.edu)|128.100.3.30|:80…
connected.
HTTP request sent, awaiting response… 301 Moved Permanently
Location: https://cave.cs.toronto.edu/kriz/cifar-10-python.tar.gz [following]
--2026-11-14 11:42:53--  https://cave.cs.toronto.edu/kriz/cifar-10-python.tar.gz
Resolving cave.cs.toronto.edu (cave.cs.toronto.edu)… 128.100.3.67
Connecting to cave.cs.toronto.edu (cave.cs.toronto.edu)|128.100.3.67|:443…
```

```
connected.
HTTP request sent, awaiting response… 200 OK
Length: 170498071 (163M) [application/x-gzip]
Saving to: ‘cifar-10-python.tar.gz’

cifar-10-python.tar 100%[===================>] 162.60M  19.8MB/s    in 6.2s

2026-11-14 11:43:00 (26.4 MB/s) - ‘cifar-10-python.tar.gz’ saved
[170498071/170498071]

cifar-10-batches-py/
cifar-10-batches-py/data_batch_4
cifar-10-batches-py/readme.html
cifar-10-batches-py/test_batch
cifar-10-batches-py/data_batch_3
cifar-10-batches-py/batches.meta
cifar-10-batches-py/data_batch_2
cifar-10-batches-py/data_batch_5
cifar-10-batches-py/data_batch_1
--2026-11-14 11:43:04--  http://cs231n.stanford.edu/imagenet_val_25.npz
Resolving cs231n.stanford.edu (cs231n.stanford.edu)… 171.64.64.21
Connecting to cs231n.stanford.edu (cs231n.stanford.edu)|171.64.64.21|:80…
connected.
HTTP request sent, awaiting response… 301 Moved Permanently
Location: https://cs231n.stanford.edu/imagenet_val_25.npz [following]
--2026-11-14 11:43:04--  https://cs231n.stanford.edu/imagenet_val_25.npz
Connecting to cs231n.stanford.edu (cs231n.stanford.edu)|171.64.64.21|:443…
connected.
HTTP request sent, awaiting response… 200 OK
Length: 3940548 (3.8M)
Saving to: ‘imagenet_val_25.npz’

imagenet_val_25.npz 100%[===================>]   3.76M  --.-KB/s    in 0.1s

2026-11-14 11:43:04 (27.2 MB/s) - ‘imagenet_val_25.npz’ saved [3940548/3940548]

/content/drive/My Drive/cs231n/assignments/assignment3
```

0.1 Using GPU

Go to `Runtime > Change runtime type` and set `Hardware accelerator` to `GPU`. This will reset Colab. **Rerun the top cell to mount your Drive again.**

1 Self-Supervised Learning

1.1 What is self-supervised learning?

Modern day machine learning requires lots of labeled data. But often times it's challenging and/or expensive to obtain large amounts of human-labeled data. Is there a way we could ask machines to automatically learn a model which can generate good visual representations without a labeled dataset? Yes, enter self-supervised learning!

Self-supervised learning (SSL) allows models to automatically learn a "good" representation space using the data in a given dataset without the need for their labels. Specifically, if our dataset were a bunch of images, then self-supervised learning allows a model to learn and generate a "good" representation vector for images.

The reason SSL methods have seen a surge in popularity is because the learnt model continues to perform well on other datasets as well i.e. new datasets on which the model was not trained on!

1.2 What makes a "good" representation?

A "good" representation vector needs to capture the important features of the image as it relates to the rest of the dataset. This means that images in the dataset representing semantically similar entities should have similar representation vectors, and different images in the dataset should have different representation vectors. For example, two images of an apple should have similar representation vectors, while an image of an apple and an image of a banana should have different representation vectors.

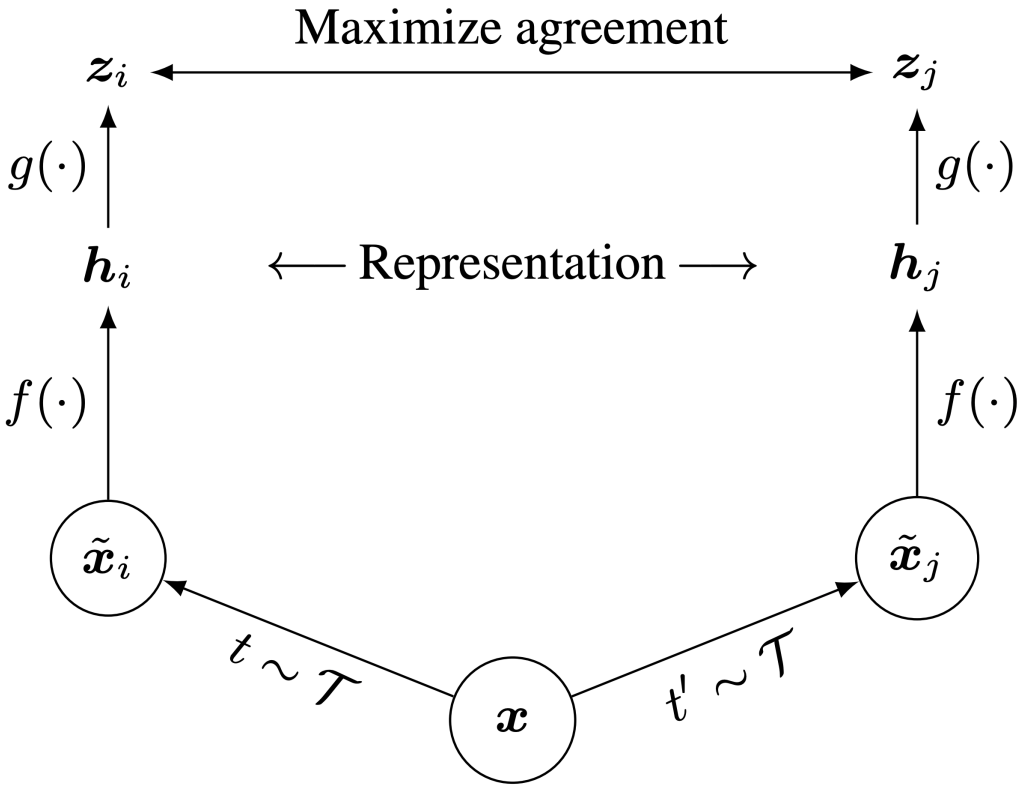
1.3 Contrastive Learning: SimCLR

Recently, SimCLR introduces a new architecture which uses **contrastive learning** to learn good visual representations. Contrastive learning aims to learn similar representations for similar images and different representations for different images. As we will see in this notebook, this simple idea allows us to train a surprisingly good model without using any labels.

Specifically, for each image in the dataset, SimCLR generates two differently augmented views of that image, called a **positive pair**. Then, the model is encouraged to generate similar representation vectors for this pair of images. See below for an illustration of the architecture (Figure 2 from the paper).

[2]:
```
# Run this cell to view the SimCLR architecture.
from IPython.display import Image
Image('images/simclr_fig2.png', width=500)
```

[2]:



Given an image **x**, SimCLR uses two different data augmentation schemes **t** and **t'** to generate the positive pair of images $\tilde{x}_i$ and $\tilde{x}_j$. f is a basic encoder net that extracts representation vectors from the augmented data samples, which yields h_i and h_j, respectively. Finally, a small neural network projection head g maps the representation vectors to the space where the contrastive loss is applied. The goal of the contrastive loss is to maximize agreement between the final vectors $z_i = g(h_i)$ and $z_j = g(h_j)$. We will discuss the contrastive loss in more detail later, and you will get to implement it.

After training is completed, we throw away the projection head g and only use f and the representation h to perform downstream tasks, such as classification. You will get a chance to finetune a layer on top of a trained SimCLR model for a classification task and compare its performance with a baseline model (without self-supervised learning).

1.4 Pretrained Weights

For your convenience, we have given you pretrained weights (trained for ~18 hours on CIFAR-10) for the SimCLR model. Run the following cell to download pretrained model weights to be used later. (This will take ~1 minute)

[3]:
```
%%bash
DIR=pretrained_model/
```

```
if [ ! -d "$DIR" ]; then
    mkdir "$DIR"
fi

URL=http://downloads.cs.stanford.edu/downloads/cs231n/pretrained_simclr_model.
 ↪pth
# Try this if above doesn't work.
# URL=http://cs231n.stanford.edu/2025/storage/a3/pretrained_simclr_model.pth
FILE=pretrained_model/pretrained_simclr_model.pth
if [ ! -f "$FILE" ]; then
    echo "Downloading weights..."
    wget "$URL" -O "$FILE"
fi
```

```
Downloading weights…

--2026-11-14 11:43:24--
http://downloads.cs.stanford.edu/downloads/cs231n/pretrained_simclr_model.pth
Resolving downloads.cs.stanford.edu (downloads.cs.stanford.edu)… 171.64.64.22
Connecting to downloads.cs.stanford.edu
(downloads.cs.stanford.edu)|171.64.64.22|:80… connected.
HTTP request sent, awaiting response… 200 OK
Length: 98808459 (94M) [application/octet-stream]
Saving to: ‘pretrained_model/pretrained_simclr_model.pth’

     0K … … … … …  0% 1.51M 62s
    50K … … … … …  0% 3.00M 47s
   100K … … … … …  0% 74.8M 32s
   150K … … … … …  0% 93.1T 24s
   200K … … … … …  0% 3.13M 25s
   250K … … … … …  0% 72.3M 21s
   300K … … … … …  0%  131M 18s
   350K … … … … …  0%  148M 16s
   400K … … … … …  0% 3.29M 17s
   450K … … … … …  0% 59.6M 16s
   500K … … … … …  0% 54.6M 14s
   550K … … … … …  0%  131M 13s
   600K … … … … …  0% 93.1T 12s
   650K … … … … …  0%  147M 11s
   700K … … … … …  0%  172M 11s
   750K … … … … …  0%  175M 10s
   800K … … … … …  0% 3.58M 11s
   850K … … … … …  0%  107M 10s
   900K … … … … …  0% 93.1T 10s
   950K … … … … …  1% 34.8M 9s
  1000K … … … … …  1% 25.3M 9s
  1050K … … … … …  1% 56.8M 9s
  1100K … … … … …  1% 93.1T 8s
```

```
1150K … … … … …  1%  158M 8s
1200K … … … … …  1%  184M 8s
1250K … … … … …  1%  159M 8s
1300K … … … … …  1%  156M 7s
1350K … … … … …  1% 4.86M 8s
1400K … … … … …  1% 94.7M 7s
1450K … … … … …  1% 98.7M 7s
1500K … … … … …  1%  172M 7s
1550K … … … … …  1%  167M 7s
1600K … … … … …  1% 93.1T 7s
1650K … … … … …  1% 57.8M 6s
1700K … … … … …  1% 68.9M 6s
1750K … … … … …  1%  140M 6s
1800K … … … … …  1%  124M 6s
1850K … … … … …  1% 93.1T 6s
1900K … … … … …  2%  135M 6s
1950K … … … … …  2%  117M 6s
2000K … … … … …  2%  119M 5s
2050K … … … … …  2% 93.1T 5s
2100K … … … … …  2%  114M 5s
2150K … … … … …  2% 26.9M 5s
2200K … … … … …  2%  127M 5s
2250K … … … … …  2%  120M 5s
2300K … … … … …  2% 93.1T 5s
2350K … … … … …  2%  130M 5s
2400K … … … … …  2%  111M 5s
2450K … … … … …  2%  135M 5s
2500K … … … … …  2% 93.1T 5s
2550K … … … … …  2%  127M 4s
2600K … … … … …  2%  122M 4s
2650K … … … … …  2% 54.7M 4s
2700K … … … … …  2%  129M 4s
2750K … … … … …  2% 9.63M 4s
2800K … … … … …  2%  122M 4s
2850K … … … … …  3% 93.1T 4s
2900K … … … … …  3%  145M 4s
2950K … … … … …  3%  136M 4s
3000K … … … … …  3%  145M 4s
3050K … … … … …  3%  144M 4s
3100K … … … … …  3% 93.1T 4s
3150K … … … … …  3% 53.7M 4s
3200K … … … … …  3% 95.8M 4s
3250K … … … … …  3% 86.1M 4s
3300K … … … … …  3% 93.1T 4s
3350K … … … … …  3%  110M 4s
3400K … … … … …  3%  122M 4s
3450K … … … … …  3%  161M 4s
3500K … … … … …  3%  163M 4s
```

```
3550K … … … … …  3% 93.1T 3s
3600K … … … … …  3%  172M 3s
3650K … … … … …  3% 70.1M 3s
3700K … … … … …  3%  153M 3s
3750K … … … … …  3% 93.1T 3s
3800K … … … … …  3%  173M 3s
3850K … … … … …  4%  168M 3s
3900K … … … … …  4%  148M 3s
3950K … … … … …  4%  151M 3s
4000K … … … … …  4% 93.1T 3s
4050K … … … … …  4%  167M 3s
4100K … … … … …  4%  130M 3s
4150K … … … … …  4% 63.4M 3s
4200K … … … … …  4% 93.1T 3s
4250K … … … … …  4%  155M 3s
4300K … … … … …  4%  168M 3s
4350K … … … … …  4%  167M 3s
4400K … … … … …  4%  154M 3s
4450K … … … … …  4% 93.1T 3s
4500K … … … … …  4%  172M 3s
4550K … … … … …  4%  184M 3s
4600K … … … … …  4% 9.25M 3s
4650K … … … … …  4% 84.6M 3s
4700K … … … … …  4% 93.1T 3s
4750K … … … … …  4% 50.2M 3s
4800K … … … … …  5%  118M 3s
4850K … … … … …  5%  104M 3s
4900K … … … … …  5%  123M 3s
4950K … … … … …  5% 93.1T 3s
5000K … … … … …  5% 94.6M 3s
5050K … … … … …  5%  160M 3s
5100K … … … … …  5%  147M 3s
5150K … … … … …  5%  123M 3s
5200K … … … … …  5% 93.1T 3s
5250K … … … … …  5% 42.7M 3s
5300K … … … … …  5%  110M 3s
5350K … … … … …  5%  105M 3s
5400K … … … … …  5% 93.1T 3s
5450K … … … … …  5%  135M 3s
5500K … … … … …  5%  154M 3s
5550K … … … … …  5%  154M 2s
5600K … … … … …  5%  161M 2s
5650K … … … … …  5% 93.1T 2s
5700K … … … … …  5%  157M 2s
5750K … … … … …  6% 63.8M 2s
5800K … … … … …  6%  152M 2s
5850K … … … … …  6% 93.1T 2s
5900K … … … … …  6%  164M 2s
```

```
5950K … … … … …  6%  169M 2s
6000K … … … … …  6%  141M 2s
6050K … … … … …  6% 5.09M 2s
6100K … … … … …  6% 83.1M 2s
6150K … … … … …  6% 93.1T 2s
6200K … … … … …  6%  108M 2s
6250K … … … … …  6%  160M 2s
6300K … … … … …  6% 51.0M 2s
6350K … … … … …  6% 93.1T 2s
6400K … … … … …  6%  120M 2s
6450K … … … … …  6% 99.3M 2s
6500K … … … … …  6%  140M 2s
6550K … … … … …  6%  158M 2s
6600K … … … … …  6% 93.1T 2s
6650K … … … … …  6%  142M 2s
6700K … … … … …  6%  144M 2s
6750K … … … … …  7%  161M 2s
6800K … … … … …  7% 62.6M 2s
6850K … … … … …  7% 93.1T 2s
6900K … … … … …  7%  160M 2s
6950K … … … … …  7%  165M 2s
7000K … … … … …  7%  156M 2s
7050K … … … … …  7% 93.1T 2s
7100K … … … … …  7%  161M 2s
7150K … … … … …  7% 84.0M 2s
7200K … … … … …  7%  151M 2s
7250K … … … … …  7%  158M 2s
7300K … … … … …  7% 93.1T 2s
7350K … … … … …  7% 62.8M 2s
7400K … … … … …  7%  158M 2s
7450K … … … … …  7%  121M 2s
7500K … … … … …  7% 93.1T 2s
7550K … … … … …  7%  160M 2s
7600K … … … … …  7%  150M 2s
7650K … … … … …  7%  145M 2s
7700K … … … … …  8%  148M 2s
7750K … … … … …  8% 93.1T 2s
7800K … … … … …  8%  163M 2s
7850K … … … … …  8% 67.5M 2s
7900K … … … … …  8%  135M 2s
7950K … … … … …  8% 93.1T 2s
8000K … … … … …  8%  136M 2s
8050K … … … … …  8%  143M 2s
8100K … … … … …  8%  156M 2s
8150K … … … … …  8%  149M 2s
8200K … … … … …  8% 93.1T 2s
8250K … … … … …  8%  162M 2s
8300K … … … … …  8%  162M 2s
```

```
 8350K … … … … …  8% 65.9M 2s
 8400K … … … … …  8%  117M 2s
 8450K … … … … …  8% 93.1T 2s
 8500K … … … … …  8%  139M 2s
 8550K … … … … …  8%  156M 2s
 8600K … … … … …  8%  148M 2s
 8650K … … … … …  9% 93.1T 2s
 8700K … … … … …  9%  121M 2s
 8750K … … … … …  9%  136M 2s
 8800K … … … … …  9%  128M 2s
 8850K … … … … …  9% 60.7M 2s
 8900K … … … … …  9% 93.1T 2s
 8950K … … … … …  9%  103M 2s
 9000K … … … … …  9%  144M 2s
 9050K … … … … …  9%  108M 2s
 9100K … … … … …  9% 93.1T 2s
 9150K … … … … …  9%  120M 2s
 9200K … … … … …  9%  100M 2s
 9250K … … … … …  9%  121M 2s
 9300K … … … … …  9%  119M 2s
 9350K … … … … …  9% 93.1T 2s
 9400K … … … … …  9% 52.7M 2s
 9450K … … … … …  9%  107M 2s
 9500K … … … … …  9%  144M 2s
 9550K … … … … …  9% 93.1T 2s
 9600K … … … … … 10%  144M 2s
 9650K … … … … … 10%  133M 2s
 9700K … … … … … 10%  150M 2s
 9750K … … … … … 10%  133M 2s
 9800K … … … … … 10% 93.1T 2s
 9850K … … … … … 10%  148M 2s
 9900K … … … … … 10% 62.0M 2s
 9950K … … … … … 10%  152M 2s
10000K … … … … … 10%  162M 2s
10050K … … … … … 10% 93.1T 2s
10100K … … … … … 10%  158M 2s
10150K … … … … … 10%  165M 2s
10200K … … … … … 10%  170M 2s
10250K … … … … … 10% 93.1T 2s
10300K … … … … … 10%  160M 2s
10350K … … … … … 10%  167M 2s
10400K … … … … … 10% 69.9M 2s
10450K … … … … … 10%  130M 2s
10500K … … … … … 10% 93.1T 2s
10550K … … … … … 10%  151M 2s
10600K … … … … … 11%  149M 2s
10650K … … … … … 11%  147M 2s
10700K … … … … … 11% 93.1T 2s
```

```
10750K … … … … … 11%  159M 2s
10800K … … … … … 11%  132M 2s
10850K … … … … … 11%  131M 2s
10900K … … … … … 11% 61.4M 2s
10950K … … … … … 11% 93.1T 2s
11000K … … … … … 11%  157M 2s
11050K … … … … … 11%  162M 2s
11100K … … … … … 11%  156M 2s
11150K … … … … … 11% 93.1T 2s
11200K … … … … … 11%  159M 2s
11250K … … … … … 11%  161M 1s
11300K … … … … … 11%  161M 1s
11350K … … … … … 11%  178M 1s
11400K … … … … … 11% 93.1T 1s
11450K … … … … … 11% 73.9M 1s
11500K … … … … … 11%  148M 1s
11550K … … … … … 12%  121M 1s
11600K … … … … … 12%  134M 1s
11650K … … … … … 12% 93.1T 1s
11700K … … … … … 12%  113M 1s
11750K … … … … … 12%  121M 1s
11800K … … … … … 12%  121M 1s
11850K … … … … … 12% 93.1T 1s
11900K … … … … … 12% 19.3M 1s
11950K … … … … … 12% 93.1T 1s
12000K … … … … … 12% 51.9M 1s
12050K … … … … … 12%  149M 1s
12100K … … … … … 12%  159M 1s
12150K … … … … … 12% 93.1T 1s
12200K … … … … … 12%  168M 1s
12250K … … … … … 12%  158M 1s
12300K … … … … … 12%  160M 1s
12350K … … … … … 12%  141M 1s
12400K … … … … … 12% 93.1T 1s
12450K … … … … … 12%  128M 1s
12500K … … … … … 13% 65.5M 1s
12550K … … … … … 13%  133M 1s
12600K … … … … … 13% 5.49M 1s
12650K … … … … … 13% 93.1T 1s
12700K … … … … … 13% 72.8M 1s
12750K … … … … … 13%  112M 1s
12800K … … … … … 13%  109M 1s
12850K … … … … … 13% 92.1M 1s
12900K … … … … … 13% 93.1T 1s
12950K … … … … … 13%  126M 1s
13000K … … … … … 13%  119M 1s
13050K … … … … … 13% 51.5M 1s
13100K … … … … … 13% 93.1T 1s
```

```
13150K … … … … … 13%  155M 1s
13200K … … … … … 13%  166M 1s
13250K … … … … … 13%  173M 1s
13300K … … … … … 13%  168M 1s
13350K … … … … … 13% 93.1T 1s
13400K … … … … … 13%  170M 1s
13450K … … … … … 13%  176M 1s
13500K … … … … … 14%  180M 1s
13550K … … … … … 14% 93.1T 1s
13600K … … … … … 14% 69.7M 1s
13650K … … … … … 14%  171M 1s
13700K … … … … … 14%  166M 1s
13750K … … … … … 14%  171M 1s
13800K … … … … … 14% 93.1T 1s
13850K … … … … … 14%  167M 1s
13900K … … … … … 14%  168M 1s
13950K … … … … … 14%  174M 1s
14000K … … … … … 14%  145M 1s
14050K … … … … … 14% 93.1T 1s
14100K … … … … … 14% 66.2M 1s
14150K … … … … … 14%  103M 1s
14200K … … … … … 14%  170M 1s
14250K … … … … … 14% 93.1T 1s
14300K … … … … … 14%  172M 1s
14350K … … … … … 14%  172M 1s
14400K … … … … … 14%  166M 1s
14450K … … … … … 15%  139M 1s
14500K … … … … … 15% 93.1T 1s
14550K … … … … … 15%  142M 1s
14600K … … … … … 15% 58.2M 1s
14650K … … … … … 15%  147M 1s
14700K … … … … … 15% 93.1T 1s
14750K … … … … … 15%  139M 1s
14800K … … … … … 15%  153M 1s
14850K … … … … … 15%  144M 1s
14900K … … … … … 15%  160M 1s
14950K … … … … … 15% 93.1T 1s
15000K … … … … … 15%  175M 1s
15050K … … … … … 15%  175M 1s
15100K … … … … … 15% 66.6M 1s
15150K … … … … … 15% 93.1T 1s
15200K … … … … … 15%  174M 1s
15250K … … … … … 15% 49.5M 1s
15300K … … … … … 15%  121M 1s
15350K … … … … … 15%  103M 1s
15400K … … … … … 16% 93.1T 1s
15450K … … … … … 16% 91.6M 1s
15500K … … … … … 16% 85.2M 1s
```

```
15550K … … … … … 16%  123M 1s
15600K … … … … … 16% 50.4M 1s
15650K … … … … … 16% 93.1T 1s
15700K … … … … … 16% 99.1M 1s
15750K … … … … … 16%  118M 1s
15800K … … … … … 16%  119M 1s
15850K … … … … … 16% 93.1T 1s
15900K … … … … … 16%  125M 1s
15950K … … … … … 16%  154M 1s
16000K … … … … … 16%  168M 1s
16050K … … … … … 16%  163M 1s
16100K … … … … … 16% 93.1T 1s
16150K … … … … … 16% 63.1M 1s
16200K … … … … … 16%  172M 1s
16250K … … … … … 16%  169M 1s
16300K … … … … … 16% 93.1T 1s
16350K … … … … … 16%  172M 1s
16400K … … … … … 17%  177M 1s
16450K … … … … … 17%  151M 1s
16500K … … … … … 17%  167M 1s
16550K … … … … … 17% 93.1T 1s
16600K … … … … … 17%  165M 1s
16650K … … … … … 17% 61.0M 1s
16700K … … … … … 17%  177M 1s
16750K … … … … … 17% 93.1T 1s
16800K … … … … … 17%  174M 1s
16850K … … … … … 17%  142M 1s
16900K … … … … … 17%  146M 1s
16950K … … … … … 17%  124M 1s
17000K … … … … … 17% 93.1T 1s
17050K … … … … … 17%  143M 1s
17100K … … … … … 17%  149M 1s
17150K … … … … … 17% 63.8M 1s
17200K … … … … … 17%  177M 1s
17250K … … … … … 17% 93.1T 1s
17300K … … … … … 17%  164M 1s
17350K … … … … … 18%  174M 1s
17400K … … … … … 18%  177M 1s
17450K … … … … … 18% 93.1T 1s
17500K … … … … … 18%  168M 1s
17550K … … … … … 18%  168M 1s
17600K … … … … … 18%  176M 1s
17650K … … … … … 18% 63.2M 1s
17700K … … … … … 18% 93.1T 1s
17750K … … … … … 18% 95.2M 1s
17800K … … … … … 18%  122M 1s
17850K … … … … … 18%  115M 1s
17900K … … … … … 18% 93.1T 1s
```

```
17950K … … … … … 18%  122M 1s
18000K … … … … … 18%  112M 1s
18050K … … … … … 18%  112M 1s
18100K … … … … … 18%  117M 1s
18150K … … … … … 18% 93.1T 1s
18200K … … … … … 18% 50.2M 1s
18250K … … … … … 18%  108M 1s
18300K … … … … … 19%  119M 1s
18350K … … … … … 19% 93.1T 1s
18400K … … … … … 19%  104M 1s
18450K … … … … … 19% 92.3M 1s
18500K … … … … … 19%  103M 1s
18550K … … … … … 19%  121M 1s
18600K … … … … … 19% 93.1T 1s
18650K … … … … … 19%  158M 1s
18700K … … … … … 19% 59.5M 1s
18750K … … … … … 19%  137M 1s
18800K … … … … … 19%  143M 1s
18850K … … … … … 19% 93.1T 1s
18900K … … … … … 19%  152M 1s
18950K … … … … … 19%  165M 1s
19000K … … … … … 19%  165M 1s
19050K … … … … … 19% 93.1T 1s
19100K … … … … … 19%  143M 1s
19150K … … … … … 19%  152M 1s
19200K … … … … … 19% 71.3M 1s
19250K … … … … … 20%  171M 1s
19300K … … … … … 20% 93.1T 1s
19350K … … … … … 20%  176M 1s
19400K … … … … … 20%  176M 1s
19450K … … … … … 20%  174M 1s
19500K … … … … … 20% 93.1T 1s
19550K … … … … … 20%  169M 1s
19600K … … … … … 20%  179M 1s
19650K … … … … … 20%  136M 1s
19700K … … … … … 20% 66.9M 1s
19750K … … … … … 20% 93.1T 1s
19800K … … … … … 20%  157M 1s
19850K … … … … … 20%  176M 1s
19900K … … … … … 20%  176M 1s
19950K … … … … … 20% 93.1T 1s
20000K … … … … … 20%  132M 1s
20050K … … … … … 20%  163M 1s
20100K … … … … … 20%  165M 1s
20150K … … … … … 20%  158M 1s
20200K … … … … … 20% 93.1T 1s
20250K … … … … … 21% 58.2M 1s
20300K … … … … … 21%  111M 1s
```

```
20350K … … … … … 21%  168M 1s
20400K … … … … … 21%  178M 1s
20450K … … … … … 21% 93.1T 1s
20500K … … … … … 21%  128M 1s
20550K … … … … … 21%  134M 1s
20600K … … … … … 21%  135M 1s
20650K … … … … … 21% 93.1T 1s
20700K … … … … … 21%  102M 1s
20750K … … … … … 21% 55.1M 1s
20800K … … … … … 21%  139M 1s
20850K … … … … … 21% 91.6M 1s
20900K … … … … … 21% 93.1T 1s
20950K … … … … … 21% 94.8M 1s
21000K … … … … … 21%  129M 1s
21050K … … … … … 21%  117M 1s
21100K … … … … … 21% 93.1T 1s
21150K … … … … … 21%  107M 1s
21200K … … … … … 22%  103M 1s
21250K … … … … … 22% 62.2M 1s
21300K … … … … … 22%  174M 1s
21350K … … … … … 22% 93.1T 1s
21400K … … … … … 22%  134M 1s
21450K … … … … … 22%  121M 1s
21500K … … … … … 22%  151M 1s
21550K … … … … … 22% 93.1T 1s
21600K … … … … … 22%  141M 1s
21650K … … … … … 22%  153M 1s
21700K … … … … … 22%  163M 1s
21750K … … … … … 22% 72.2M 1s
21800K … … … … … 22% 93.1T 1s
21850K … … … … … 22%  172M 1s
21900K … … … … … 22%  179M 1s
21950K … … … … … 22%  157M 1s
22000K … … … … … 22%  150M 1s
22050K … … … … … 22% 93.1T 1s
22100K … … … … … 22%  178M 1s
22150K … … … … … 23%  169M 1s
22200K … … … … … 23%  166M 1s
22250K … … … … … 23% 93.1T 1s
22300K … … … … … 23% 71.3M 1s
22350K … … … … … 23%  164M 1s
22400K … … … … … 23%  175M 1s
22450K … … … … … 23%  180M 1s
22500K … … … … … 23% 93.1T 1s
22550K … … … … … 23%  167M 1s
22600K … … … … … 23%  175M 1s
22650K … … … … … 23%  180M 1s
22700K … … … … … 23% 93.1T 1s
```

```
22750K … … … … … 23%  173M 1s
22800K … … … … … 23% 71.7M 1s
22850K … … … … … 23% 99.5M 1s
22900K … … … … … 23%  105M 1s
22950K … … … … … 23% 93.1T 1s
23000K … … … … … 23%  122M 1s
23050K … … … … … 23%  145M 1s
23100K … … … … … 23%  134M 1s
23150K … … … … … 24% 93.1T 1s
23200K … … … … … 24%  124M 1s
23250K … … … … … 24%  152M 1s
23300K … … … … … 24% 62.0M 1s
23350K … … … … … 24%  146M 1s
23400K … … … … … 24% 93.1T 1s
23450K … … … … … 24%  139M 1s
23500K … … … … … 24%  162M 1s
23550K … … … … … 24%  159M 1s
23600K … … … … … 24%  139M 1s
23650K … … … … … 24% 93.1T 1s
23700K … … … … … 24%  120M 1s
23750K … … … … … 24%  126M 1s
23800K … … … … … 24% 55.4M 1s
23850K … … … … … 24% 93.1T 1s
23900K … … … … … 24%  129M 1s
23950K … … … … … 24%  113M 1s
24000K … … … … … 24%  148M 1s
24050K … … … … … 24%  156M 1s
24100K … … … … … 25% 93.1T 1s
24150K … … … … … 25%  160M 1s
24200K … … … … … 25%  147M 1s
24250K … … … … … 25%  159M 1s
24300K … … … … … 25% 93.1T 1s
24350K … … … … … 25% 69.4M 1s
24400K … … … … … 25%  171M 1s
24450K … … … … … 25%  141M 1s
24500K … … … … … 25%  174M 1s
24550K … … … … … 25% 93.1T 1s
24600K … … … … … 25%  176M 1s
24650K … … … … … 25%  164M 1s
24700K … … … … … 25%  174M 1s
24750K … … … … … 25% 93.1T 1s
24800K … … … … … 25%  177M 1s
24850K … … … … … 25% 72.5M 1s
24900K … … … … … 25%  172M 1s
24950K … … … … … 25%  171M 1s
25000K … … … … … 25% 93.1T 1s
25050K … … … … … 26%  166M 1s
25100K … … … … … 26%  180M 1s
```

```
25150K … … … … … 26%  176M 1s
25200K … … … … … 26%  142M 1s
25250K … … … … … 26% 93.1T 1s
25300K … … … … … 26%  144M 1s
25350K … … … … … 26% 67.7M 1s
25400K … … … … … 26%  160M 1s
25450K … … … … … 26% 93.1T 1s
25500K … … … … … 26%  166M 1s
25550K … … … … … 26%  178M 1s
25600K … … … … … 26%  110M 1s
25650K … … … … … 26%  153M 1s
25700K … … … … … 26% 93.1T 1s
25750K … … … … … 26%  150M 1s
25800K … … … … … 26%  171M 1s
25850K … … … … … 26% 68.5M 1s
25900K … … … … … 26% 93.1T 1s
25950K … … … … … 26%  140M 1s
26000K … … … … … 26%  178M 1s
26050K … … … … … 27%  175M 1s
26100K … … … … … 27%  155M 1s
26150K … … … … … 27% 93.1T 1s
26200K … … … … … 27%  166M 1s
26250K … … … … … 27%  167M 1s
26300K … … … … … 27%  166M 1s
26350K … … … … … 27% 93.1T 1s
26400K … … … … … 27% 70.3M 1s
26450K … … … … … 27%  147M 1s
26500K … … … … … 27%  176M 1s
26550K … … … … … 27%  178M 1s
26600K … … … … … 27% 93.1T 1s
26650K … … … … … 27%  166M 1s
26700K … … … … … 27%  170M 1s
26750K … … … … … 27%  182M 1s
26800K … … … … … 27%  176M 1s
26850K … … … … … 27% 93.1T 1s
26900K … … … … … 27%  209K 1s
26950K … … … … … 27% 56.9M 1s
27000K … … … … … 28%  116M 1s
27050K … … … … … 28% 93.1T 1s
27100K … … … … … 28%  142M 1s
27150K … … … … … 28%  153M 1s
27200K … … … … … 28%  159M 1s
27250K … … … … … 28% 93.1T 1s
27300K … … … … … 28%  151M 1s
27350K … … … … … 28%  174M 1s
27400K … … … … … 28% 57.6M 1s
27450K … … … … … 28%  155M 1s
27500K … … … … … 28% 93.1T 1s
```

```
27550K … … … … … 28%  144M 1s
27600K … … … … … 28% 18.0M 1s
27650K … … … … … 28%  117M 1s
27700K … … … … … 28% 93.1T 1s
27750K … … … … … 28%  111M 1s
27800K … … … … … 28%  112M 1s
27850K … … … … … 28% 58.5M 1s
27900K … … … … … 28% 93.1T 1s
27950K … … … … … 29%  138M 1s
28000K … … … … … 29%  148M 1s
28050K … … … … … 29%  165M 1s
28100K … … … … … 29%  161M 1s
28150K … … … … … 29% 93.1T 1s
28200K … … … … … 29%  159M 1s
28250K … … … … … 29%  169M 1s
28300K … … … … … 29%  162M 1s
28350K … … … … … 29% 69.1M 1s
28400K … … … … … 29% 10.6M 1s
28450K … … … … … 29% 93.1T 1s
28500K … … … … … 29%  126M 1s
28550K … … … … … 29%  128M 1s
28600K … … … … … 29%  168M 1s
28650K … … … … … 29% 93.1T 1s
28700K … … … … … 29%  149M 1s
28750K … … … … … 29%  151M 1s
28800K … … … … … 29%  149M 1s
28850K … … … … … 29%  170M 1s
28900K … … … … … 30% 93.1T 1s
28950K … … … … … 30%  129K 2s
29000K … … … … … 30% 70.5M 2s
29050K … … … … … 30% 3.14M 2s
29100K … … … … … 30% 65.4M 2s
29150K … … … … … 30% 98.9M 2s
29200K … … … … … 30%  113M 2s
29250K … … … … … 30% 93.1T 2s
29300K … … … … … 30% 97.8M 2s
29350K … … … … … 30%  118M 2s
29400K … … … … … 30%  128M 2s
29450K … … … … … 30% 93.1T 2s
29500K … … … … … 30% 58.0M 2s
29550K … … … … … 30%  140M 2s
29600K … … … … … 30%  164M 2s
29650K … … … … … 30%  168M 2s
29700K … … … … … 30% 93.1T 2s
29750K … … … … … 30%  172M 2s
29800K … … … … … 30%  172M 2s
29850K … … … … … 30%  169M 2s
29900K … … … … … 31% 93.1T 2s
```

```
29950K … … … … … 31%  169M 2s
30000K … … … … … 31% 66.7M 2s
30050K … … … … … 31%  163M 2s
30100K … … … … … 31%  177M 2s
30150K … … … … … 31% 93.1T 2s
30200K … … … … … 31%  162M 2s
30250K … … … … … 31%  172M 2s
30300K … … … … … 31%  171M 2s
30350K … … … … … 31%  181M 2s
30400K … … … … … 31% 93.1T 2s
30450K … … … … … 31%  156M 2s
30500K … … … … … 31% 72.0M 2s
30550K … … … … … 31%  136M 2s
30600K … … … … … 31% 93.1T 2s
30650K … … … … … 31%  163M 2s
30700K … … … … … 31%  169M 2s
30750K … … … … … 31%  166M 2s
30800K … … … … … 31%  178M 2s
30850K … … … … … 32% 93.1T 2s
30900K … … … … … 32%  173M 2s
30950K … … … … … 32% 8.94M 2s
31000K … … … … … 32%  135K 3s
31050K … … … … … 32% 93.1T 3s
31100K … … … … … 32% 26.7M 3s
31150K … … … … … 32%  137M 3s
31200K … … … … … 32%  147M 3s
31250K … … … … … 32% 93.1T 3s
31300K … … … … … 32%  150M 3s
31350K … … … … … 32%  156M 3s
31400K … … … … … 32%  162M 3s
31450K … … … … … 32%  161M 3s
31500K … … … … … 32% 93.1T 3s
31550K … … … … … 32% 68.7M 3s
31600K … … … … … 32%  161M 3s
31650K … … … … … 32%  175M 3s
31700K … … … … … 32%  173M 3s
31750K … … … … … 32% 93.1T 3s
31800K … … … … … 33%  151M 3s
31850K … … … … … 33%  165M 3s
31900K … … … … … 33%  171M 3s
31950K … … … … … 33% 93.1T 3s
32000K … … … … … 33%  154M 3s
32050K … … … … … 33% 63.9M 3s
32100K … … … … … 33%  143M 3s
32150K … … … … … 33%  166M 3s
32200K … … … … … 33% 93.1T 3s
32250K … … … … … 33%  173M 3s
32300K … … … … … 33%  171M 3s
```

```
32350K … … … … … 33%  165M 3s
32400K … … … … … 33% 5.71M 3s
32450K … … … … … 33% 74.2M 3s
32500K … … … … … 33% 93.1T 3s
32550K … … … … … 33%  116M 3s
32600K … … … … … 33% 61.4M 3s
32650K … … … … … 33%  164M 3s
32700K … … … … … 33%  165M 3s
32750K … … … … … 33% 93.1T 3s
32800K … … … … … 34%  174M 3s
32850K … … … … … 34%  181M 3s
32900K … … … … … 34%  164M 3s
32950K … … … … … 34% 3.76M 3s
33000K … … … … … 34% 97.4M 3s
33050K … … … … … 34%  137K 3s
33100K … … … … … 34% 93.1T 3s
33150K … … … … … 34% 43.5M 3s
33200K … … … … … 34% 92.6M 3s
33250K … … … … … 34%  122M 3s
33300K … … … … … 34% 93.1T 3s
33350K … … … … … 34%  154M 3s
33400K … … … … … 34%  134M 3s
33450K … … … … … 34%  161M 3s
33500K … … … … … 34%  130M 3s
33550K … … … … … 34% 93.1T 3s
33600K … … … … … 34% 90.4M 3s
33650K … … … … … 34% 57.8M 3s
33700K … … … … … 34%  142M 3s
33750K … … … … … 35% 93.1T 3s
33800K … … … … … 35%  146M 3s
33850K … … … … … 35%  166M 3s
33900K … … … … … 35%  120M 3s
33950K … … … … … 35%  156M 3s
34000K … … … … … 35% 93.1T 3s
34050K … … … … … 35%  168M 3s
34100K … … … … … 35%  179M 3s
34150K … … … … … 35% 59.4M 3s
34200K … … … … … 35% 93.1T 3s
34250K … … … … … 35%  157M 3s
34300K … … … … … 35%  166M 3s
34350K … … … … … 35%  173M 3s
34400K … … … … … 35%  175M 3s
34450K … … … … … 35% 93.1T 3s
34500K … … … … … 35% 7.08M 3s
34550K … … … … … 35% 93.1T 3s
34600K … … … … … 35%  115M 3s
34650K … … … … … 35%  164M 3s
34700K … … … … … 36% 62.3M 3s
```

```
34750K … … … … … 36%  112M 3s
34800K … … … … … 36% 93.1T 3s
34850K … … … … … 36%  152M 3s
34900K … … … … … 36%  161M 3s
34950K … … … … … 36%  177M 3s
35000K … … … … … 36% 93.1T 3s
35050K … … … … … 36%  177M 3s
35100K … … … … … 36%  131K 4s
35150K … … … … … 36% 73.0M 4s
35200K … … … … … 36% 79.7M 4s
35250K … … … … … 36% 27.0M 4s
35300K … … … … … 36%  128M 4s
35350K … … … … … 36% 93.1T 4s
35400K … … … … … 36%  149M 4s
35450K … … … … … 36%  176M 4s
35500K … … … … … 36%  166M 4s
35550K … … … … … 36% 93.1T 4s
35600K … … … … … 36%  159M 4s
35650K … … … … … 36%  182M 4s
35700K … … … … … 37%  176M 4s
35750K … … … … … 37% 62.1M 4s
35800K … … … … … 37% 93.1T 4s
35850K … … … … … 37%  186M 4s
35900K … … … … … 37%  175M 4s
35950K … … … … … 37%  173M 4s
36000K … … … … … 37% 93.1T 4s
36050K … … … … … 37%  188M 4s
36100K … … … … … 37%  181M 4s
36150K … … … … … 37%  177M 4s
36200K … … … … … 37%  179M 4s
36250K … … … … … 37% 93.1T 4s
36300K … … … … … 37% 63.1M 4s
36350K … … … … … 37%  182M 4s
36400K … … … … … 37%  207M 4s
36450K … … … … … 37%  181M 4s
36500K … … … … … 37% 93.1T 4s
36550K … … … … … 37% 7.05M 4s
36600K … … … … … 37%  131M 4s
36650K … … … … … 38%  158M 4s
36700K … … … … … 38%  134M 4s
36750K … … … … … 38% 93.1T 4s
36800K … … … … … 38%  111M 4s
36850K … … … … … 38% 54.4M 4s
36900K … … … … … 38%  110M 4s
36950K … … … … … 38%  108M 3s
37000K … … … … … 38% 93.1T 3s
37050K … … … … … 38%  122M 3s
37100K … … … … … 38%  143M 3s
```

```
37150K … … … … … 38%  132K 4s
37200K … … … … … 38% 93.1T 4s
37250K … … … … … 38%  101M 4s
37300K … … … … … 38%  108M 4s
37350K … … … … … 38% 52.0M 4s
37400K … … … … … 38% 93.1T 4s
37450K … … … … … 38%  122M 4s
37500K … … … … … 38%  109M 4s
37550K … … … … … 38%  114M 4s
37600K … … … … … 39%  114M 4s
37650K … … … … … 39% 93.1T 4s
37700K … … … … … 39%  153M 4s
37750K … … … … … 39%  159M 4s
37800K … … … … … 39%  160M 4s
37850K … … … … … 39% 58.8M 4s
37900K … … … … … 39% 93.1T 4s
37950K … … … … … 39%  114M 4s
38000K … … … … … 39%  112M 4s
38050K … … … … … 39%  163M 4s
38100K … … … … … 39% 93.1T 4s
38150K … … … … … 39%  172M 4s
38200K … … … … … 39%  172M 4s
38250K … … … … … 39%  170M 4s
38300K … … … … … 39%  176M 4s
38350K … … … … … 39% 93.1T 4s
38400K … … … … … 39% 66.8M 4s
38450K … … … … … 39%  163M 4s
38500K … … … … … 39%  180M 4s
38550K … … … … … 40% 93.1T 4s
38600K … … … … … 40% 6.91M 4s
38650K … … … … … 40% 93.1T 4s
38700K … … … … … 40%  103M 4s
38750K … … … … … 40%  153M 4s
38800K … … … … … 40%  173M 4s
38850K … … … … … 40%  127M 4s
38900K … … … … … 40% 93.1T 4s
38950K … … … … … 40% 57.4M 4s
39000K … … … … … 40%  125M 4s
39050K … … … … … 40%  125M 4s
39100K … … … … … 40%  149M 4s
39150K … … … … … 40% 93.1T 4s
39200K … … … … … 40%  131K 4s
39250K … … … … … 40% 73.1M 4s
39300K … … … … … 40%  113M 4s
39350K … … … … … 40%  123M 4s
39400K … … … … … 40% 93.1T 4s
39450K … … … … … 40%  103M 4s
39500K … … … … … 40% 55.5M 4s
```

```
39550K … … … … … 41%  145M 4s
39600K … … … … … 41%  159M 4s
39650K … … … … … 41% 93.1T 4s
39700K … … … … … 41%  160M 4s
39750K … … … … … 41%  155M 4s
39800K … … … … … 41%  163M 4s
39850K … … … … … 41% 93.1T 4s
39900K … … … … … 41%  127M 4s
39950K … … … … … 41%  142M 4s
40000K … … … … … 41% 58.5M 4s
40050K … … … … … 41%  130M 4s
40100K … … … … … 41% 93.1T 4s
40150K … … … … … 41%  163M 4s
40200K … … … … … 41%  144M 4s
40250K … … … … … 41%  175M 4s
40300K … … … … … 41%  179M 4s
40350K … … … … … 41% 93.1T 4s
40400K … … … … … 41%  170M 4s
40450K … … … … … 41%  163M 4s
40500K … … … … … 42% 71.4M 4s
40550K … … … … … 42% 5.02M 4s
40600K … … … … … 42% 93.1T 4s
40650K … … … … … 42%  142M 4s
40700K … … … … … 42%  156M 4s
40750K … … … … … 42%  157M 4s
40800K … … … … … 42% 93.1T 4s
40850K … … … … … 42%  172M 4s
40900K … … … … … 42%  165M 4s
40950K … … … … … 42%  175M 4s
41000K … … … … … 42%  175M 4s
41050K … … … … … 42% 93.1T 4s
41100K … … … … … 42% 64.2M 4s
41150K … … … … … 42%  164M 4s
41200K … … … … … 42%  151M 4s
41250K … … … … … 42%  132K 4s
41300K … … … … … 42% 73.8M 4s
41350K … … … … … 42% 95.2M 4s
41400K … … … … … 42% 93.1T 4s
41450K … … … … … 43%  138M 4s
41500K … … … … … 43%  147M 4s
41550K … … … … … 43% 49.7M 4s
41600K … … … … … 43% 93.1T 4s
41650K … … … … … 43%  126M 4s
41700K … … … … … 43%  161M 4s
41750K … … … … … 43%  151M 4s
41800K … … … … … 43%  167M 4s
41850K … … … … … 43% 93.1T 4s
41900K … … … … … 43%  165M 4s
```

```
41950K … … … … … 43%  173M 4s
42000K … … … … … 43%  175M 4s
42050K … … … … … 43% 66.2M 4s
42100K … … … … … 43% 3.88M 4s
42150K … … … … … 43% 93.1T 4s
42200K … … … … … 43%  152M 4s
42250K … … … … … 43% 3.38M 4s
42300K … … … … … 43% 93.1T 4s
42350K … … … … … 43% 75.0M 4s
42400K … … … … … 43%  140M 4s
42450K … … … … … 44%  129M 4s
42500K … … … … … 44%  133M 4s
42550K … … … … … 44% 93.1T 4s
42600K … … … … … 44%  143M 4s
42650K … … … … … 44% 49.5M 4s
42700K … … … … … 44%  127M 4s
42750K … … … … … 44% 93.1T 4s
42800K … … … … … 44%  134M 4s
42850K … … … … … 44%  159M 4s
42900K … … … … … 44%  134M 4s
42950K … … … … … 44%  161M 4s
43000K … … … … … 44% 93.1T 4s
43050K … … … … … 44%  163M 4s
43100K … … … … … 44%  120M 4s
43150K … … … … … 44% 63.8M 4s
43200K … … … … … 44%  129M 4s
43250K … … … … … 44% 93.1T 4s
43300K … … … … … 44%  136K 5s
43350K … … … … … 44% 81.0M 5s
43400K … … … … … 45%  137M 5s
43450K … … … … … 45% 93.1T 5s
43500K … … … … … 45%  122M 5s
43550K … … … … … 45%  159M 5s
43600K … … … … … 45%  163M 5s
43650K … … … … … 45% 53.0M 5s
43700K … … … … … 45% 93.1T 5s
43750K … … … … … 45%  155M 5s
43800K … … … … … 45%  154M 5s
43850K … … … … … 45%  166M 5s
43900K … … … … … 45% 93.1T 5s
43950K … … … … … 45%  163M 4s
44000K … … … … … 45% 4.05M 5s
44050K … … … … … 45% 93.1T 4s
44100K … … … … … 45% 90.0M 4s
44150K … … … … … 45% 44.2M 4s
44200K … … … … … 45%  142M 4s
44250K … … … … … 45%  164M 4s
44300K … … … … … 45% 93.1T 4s
```

```
44350K … … … … … 46%  159M 4s
44400K … … … … … 46%  167M 4s
44450K … … … … … 46%  166M 4s
44500K … … … … … 46% 93.1T 4s
44550K … … … … … 46%  176M 4s
44600K … … … … … 46%  178M 4s
44650K … … … … … 46% 70.5M 4s
44700K … … … … … 46%  163M 4s
44750K … … … … … 46% 4.19M 4s
44800K … … … … … 46%  125M 4s
44850K … … … … … 46%  117M 4s
44900K … … … … … 46% 93.1T 4s
44950K … … … … … 46%  144M 4s
45000K … … … … … 46%  174M 4s
45050K … … … … … 46% 93.8M 4s
45100K … … … … … 46% 93.1T 4s
45150K … … … … … 46%  130M 4s
45200K … … … … … 46% 61.2M 4s
45250K … … … … … 46%  138M 4s
45300K … … … … … 46%  154M 4s
45350K … … … … … 47%  138K 5s
45400K … … … … … 47% 63.4M 5s
45450K … … … … … 47%  106M 5s
45500K … … … … … 47% 93.1T 5s
45550K … … … … … 47%  131M 5s
45600K … … … … … 47%  150M 5s
45650K … … … … … 47%  140M 5s
45700K … … … … … 47% 58.7M 5s
45750K … … … … … 47% 93.1T 5s
45800K … … … … … 47%  139M 5s
45850K … … … … … 47%  162M 5s
45900K … … … … … 47%  175M 5s
45950K … … … … … 47%  182M 5s
46000K … … … … … 47% 93.1T 5s
46050K … … … … … 47% 4.11M 5s
46100K … … … … … 47% 96.0M 5s
46150K … … … … … 47% 93.1T 5s
46200K … … … … … 47%  122M 5s
46250K … … … … … 47% 49.6M 5s
46300K … … … … … 48%  141M 5s
46350K … … … … … 48% 93.1T 5s
46400K … … … … … 48%  155M 5s
46450K … … … … … 48%  164M 4s
46500K … … … … … 48%  159M 4s
46550K … … … … … 48%  154M 4s
46600K … … … … … 48% 93.1T 4s
46650K … … … … … 48%  141M 4s
46700K … … … … … 48%  153M 4s
```

```
46750K … … … … … 48% 60.9M 4s
46800K … … … … … 48% 3.59M 4s
46850K … … … … … 48% 93.1T 4s
46900K … … … … … 48% 97.3M 4s
46950K … … … … … 48%  115M 4s
47000K … … … … … 48%  157M 4s
47050K … … … … … 48%  172M 4s
47100K … … … … … 48% 93.1T 4s
47150K … … … … … 48%  172M 4s
47200K … … … … … 48%  169M 4s
47250K … … … … … 49%  121M 4s
47300K … … … … … 49% 93.1T 4s
47350K … … … … … 49%  216K 5s
47400K … … … … … 49%  377K 5s
47450K … … … … … 49% 96.2M 5s
47500K … … … … … 49% 93.1T 5s
47550K … … … … … 49%  153M 5s
47600K … … … … … 49%  171M 5s
47650K … … … … … 49%  158M 5s
47700K … … … … … 49% 93.1T 5s
47750K … … … … … 49%  166M 5s
47800K … … … … … 49% 61.2M 5s
47850K … … … … … 49%  157M 5s
47900K … … … … … 49%  170M 5s
47950K … … … … … 49% 93.1T 5s
48000K … … … … … 49%  164M 5s
48050K … … … … … 49%  172M 5s
48100K … … … … … 49% 3.98M 5s
48150K … … … … … 49% 93.1T 5s
48200K … … … … … 50%  107M 5s
48250K … … … … … 50%  134M 5s
48300K … … … … … 50% 54.1M 5s
48350K … … … … … 50%  158M 5s
48400K … … … … … 50% 93.1T 5s
48450K … … … … … 50%  170M 5s
48500K … … … … … 50%  165M 5s
48550K … … … … … 50%  177M 5s
48600K … … … … … 50% 93.1T 5s
48650K … … … … … 50%  172M 4s
48700K … … … … … 50%  169M 4s
48750K … … … … … 50%  172M 4s
48800K … … … … … 50% 71.4M 4s
48850K … … … … … 50% 4.08M 4s
48900K … … … … … 50% 95.9M 4s
48950K … … … … … 50% 93.1T 4s
49000K … … … … … 50%  111M 4s
49050K … … … … … 50%  121M 4s
49100K … … … … … 50%  128M 4s
```

```
49150K … … … … … 50%  128M 4s
49200K … … … … … 51% 93.1T 4s
49250K … … … … … 51%  169M 4s
49300K … … … … … 51%  174M 4s
49350K … … … … … 51% 63.8M 4s
49400K … … … … … 51%  138K 5s
49450K … … … … … 51% 93.1T 5s
49500K … … … … … 51% 69.1M 5s
49550K … … … … … 51%  103M 5s
49600K … … … … … 51%  162M 5s
49650K … … … … … 51% 93.1T 5s
49700K … … … … … 51%  139M 5s
49750K … … … … … 51%  182M 5s
49800K … … … … … 51%  164M 5s
49850K … … … … … 51%  164M 5s
49900K … … … … … 51% 93.1T 5s
49950K … … … … … 51% 19.0M 5s
50000K … … … … … 51%  175M 5s
50050K … … … … … 51% 21.3M 5s
50100K … … … … … 51% 93.1T 5s
50150K … … … … … 52% 5.59M 5s
50200K … … … … … 52% 93.1T 5s
50250K … … … … … 52% 52.4M 5s
50300K … … … … … 52%  119M 5s
50350K … … … … … 52%  124M 5s
50400K … … … … … 52% 56.2M 5s
50450K … … … … … 52% 93.1T 5s
50500K … … … … … 52%  133M 5s
50550K … … … … … 52%  100M 5s
50600K … … … … … 52%  161M 5s
50650K … … … … … 52% 93.1T 5s
50700K … … … … … 52%  180M 4s
50750K … … … … … 52%  153M 4s
50800K … … … … … 52%  168M 4s
50850K … … … … … 52%  158M 4s
50900K … … … … … 52% 4.29M 4s
50950K … … … … … 52% 20.1M 4s
51000K … … … … … 52%  155M 4s
51050K … … … … … 52% 93.1T 4s
51100K … … … … … 53%  144M 4s
51150K … … … … … 53%  170M 4s
51200K … … … … … 53%  131M 4s
51250K … … … … … 53% 93.1T 4s
51300K … … … … … 53%  177M 4s
51350K … … … … … 53%  140M 4s
51400K … … … … … 53%  156M 4s
51450K … … … … … 53%  138K 5s
51500K … … … … … 53%  154M 5s
```

```
51550K … … … … … 53%  149M 5s
51600K … … … … … 53%  126M 5s
51650K … … … … … 53% 93.1T 5s
51700K … … … … … 53%  152M 5s
51750K … … … … … 53%  135M 5s
51800K … … … … … 53%  168M 5s
51850K … … … … … 53%  195M 5s
51900K … … … … … 53% 93.1T 5s
51950K … … … … … 53%  155M 5s
52000K … … … … … 53% 57.1M 5s
52050K … … … … … 53%  174M 5s
52100K … … … … … 54% 93.1T 5s
52150K … … … … … 54%  140M 5s
52200K … … … … … 54% 3.92M 5s
52250K … … … … … 54%  138M 5s
52300K … … … … … 54% 93.1T 5s
52350K … … … … … 54%  142M 5s
52400K … … … … … 54%  137M 5s
52450K … … … … … 54%  130M 5s
52500K … … … … … 54% 93.1T 4s
52550K … … … … … 54% 58.6M 4s
52600K … … … … … 54%  135M 4s
52650K … … … … … 54%  142M 4s
52700K … … … … … 54%  159M 4s
52750K … … … … … 54% 93.1T 4s
52800K … … … … … 54%  150M 4s
52850K … … … … … 54%  144M 4s
52900K … … … … … 54%  163M 4s
52950K … … … … … 54% 4.02M 4s
53000K … … … … … 54%  117M 4s
53050K … … … … … 55% 47.5M 4s
53100K … … … … … 55% 93.1T 4s
53150K … … … … … 55%  162M 4s
53200K … … … … … 55%  149M 4s
53250K … … … … … 55%  168M 4s
53300K … … … … … 55% 93.1T 4s
53350K … … … … … 55%  151M 4s
53400K … … … … … 55%  188M 4s
53450K … … … … … 55%  142M 4s
53500K … … … … … 55%  137K 5s
53550K … … … … … 55% 23.9M 5s
53600K … … … … … 55% 27.3M 5s
53650K … … … … … 55% 93.1T 5s
53700K … … … … … 55%  135M 5s
53750K … … … … … 55% 66.8M 5s
53800K … … … … … 55%  167M 5s
53850K … … … … … 55%  184M 5s
53900K … … … … … 55% 93.1T 5s
```

```
53950K … … … … … 55%  163M 5s
54000K … … … … … 56%  177M 5s
54050K … … … … … 56%  184M 5s
54100K … … … … … 56% 93.1T 5s
54150K … … … … … 56% 27.3M 5s
54200K … … … … … 56%  189M 4s
54250K … … … … … 56% 6.12M 4s
54300K … … … … … 56%  104M 4s
54350K … … … … … 56%  156M 4s
54400K … … … … … 56% 93.1T 4s
54450K … … … … … 56%  192M 4s
54500K … … … … … 56%  181M 4s
54550K … … … … … 56%  176M 4s
54600K … … … … … 56% 93.1T 4s
54650K … … … … … 56% 12.4M 4s
54700K … … … … … 56% 45.8M 4s
54750K … … … … … 56% 93.1T 4s
54800K … … … … … 56%  147M 4s
54850K … … … … … 56%  170M 4s
54900K … … … … … 56%  186M 4s
54950K … … … … … 56% 93.1T 4s
55000K … … … … … 57% 5.27M 4s
55050K … … … … … 57%  173M 4s
55100K … … … … … 57%  162M 4s
55150K … … … … … 57%  174M 4s
55200K … … … … … 57% 93.1T 4s
55250K … … … … … 57% 56.6M 4s
55300K … … … … … 57%  127M 4s
55350K … … … … … 57%  138M 4s
55400K … … … … … 57% 17.0M 4s
55450K … … … … … 57% 93.1T 4s
55500K … … … … … 57%  175M 4s
55550K … … … … … 57%  138K 5s
55600K … … … … … 57% 93.1T 5s
55650K … … … … … 57% 59.8M 5s
55700K … … … … … 57% 53.5M 5s
55750K … … … … … 57%  226M 4s
55800K … … … … … 57%  257M 4s
55850K … … … … … 57% 93.1T 4s
55900K … … … … … 57%  252M 4s
55950K … … … … … 58%  233M 4s
56000K … … … … … 58%  247M 4s
56050K … … … … … 58% 93.1T 4s
56100K … … … … … 58%  248M 4s
56150K … … … … … 58%  239M 4s
56200K … … … … … 58% 68.3M 4s
56250K … … … … … 58% 70.6M 4s
56300K … … … … … 58% 4.12M 4s
```

```
56350K … … … … … 58% 93.1T 4s
56400K … … … … … 58%  149M 4s
56450K … … … … … 58%  149M 4s
56500K … … … … … 58% 68.7M 4s
56550K … … … … … 58%  148M 4s
56600K … … … … … 58% 93.1T 4s
56650K … … … … … 58%  111M 4s
56700K … … … … … 58%  150M 4s
56750K … … … … … 58% 58.8M 4s
56800K … … … … … 58% 93.1T 4s
56850K … … … … … 58%  172M 4s
56900K … … … … … 59%  151M 4s
56950K … … … … … 59%  168M 4s
57000K … … … … … 59%  148M 4s
57050K … … … … … 59% 4.17M 4s
57100K … … … … … 59%  139M 4s
57150K … … … … … 59%  138M 4s
57200K … … … … … 59%  128M 4s
57250K … … … … … 59%  127M 4s
57300K … … … … … 59% 93.1T 4s
57350K … … … … … 59% 53.5M 4s
57400K … … … … … 59%  134M 4s
57450K … … … … … 59%  163M 4s
57500K … … … … … 59% 93.1T 4s
57550K … … … … … 59%  133M 4s
57600K … … … … … 59%  137K 4s
57650K … … … … … 59% 93.1T 4s
57700K … … … … … 59% 58.1M 4s
57750K … … … … … 59%  136M 4s
57800K … … … … … 59% 33.4M 4s
57850K … … … … … 60%  141M 4s
57900K … … … … … 60% 93.1T 4s
57950K … … … … … 60%  142M 4s
58000K … … … … … 60%  161M 4s
58050K … … … … … 60% 29.2M 4s
58100K … … … … … 60% 93.1T 4s
58150K … … … … … 60%  159M 4s
58200K … … … … … 60%  187M 4s
58250K … … … … … 60%  161M 4s
58300K … … … … … 60% 67.3M 4s
58350K … … … … … 60% 4.56M 4s
58400K … … … … … 60% 93.1T 4s
58450K … … … … … 60%  158M 4s
58500K … … … … … 60%  175M 4s
58550K … … … … … 60%  151M 4s
58600K … … … … … 60%  188M 4s
58650K … … … … … 60% 93.1T 4s
58700K … … … … … 60%  138M 4s
```

```
58750K … … … … … 60%  181M 4s
58800K … … … … … 60%  157M 4s
58850K … … … … … 61% 93.1T 4s
58900K … … … … … 61% 16.6M 4s
58950K … … … … … 61%  170M 4s
59000K … … … … … 61%  162M 4s
59050K … … … … … 61%  148M 4s
59100K … … … … … 61% 3.90M 4s
59150K … … … … … 61%  168M 4s
59200K … … … … … 61% 93.1T 4s
59250K … … … … … 61%  148M 4s
59300K … … … … … 61%  177M 4s
59350K … … … … … 61%  167M 4s
59400K … … … … … 61% 93.1T 4s
59450K … … … … … 61% 65.3M 4s
59500K … … … … … 61%  160M 4s
59550K … … … … … 61%  143M 4s
59600K … … … … … 61%  146M 4s
59650K … … … … … 61%  138K 4s
59700K … … … … … 61% 93.1T 4s
59750K … … … … … 61% 86.3M 4s
59800K … … … … … 62%  113M 4s
59850K … … … … … 62%  140M 4s
59900K … … … … … 62%  120M 4s
59950K … … … … … 62% 93.1T 4s
60000K … … … … … 62% 50.7M 4s
60050K … … … … … 62%  130M 4s
60100K … … … … … 62%  168M 4s
60150K … … … … … 62% 93.1T 4s
60200K … … … … … 62%  139M 4s
60250K … … … … … 62%  142M 4s
60300K … … … … … 62%  142M 4s
60350K … … … … … 62%  154M 4s
60400K … … … … … 62% 4.42M 4s
60450K … … … … … 62%  112M 4s
60500K … … … … … 62% 60.8M 4s
60550K … … … … … 62% 93.1T 4s
60600K … … … … … 62%  121M 4s
60650K … … … … … 62%  114M 4s
60700K … … … … … 62%  112M 4s
60750K … … … … … 63% 93.1T 4s
60800K … … … … … 63%  126M 4s
60850K … … … … … 63%  122M 4s
60900K … … … … … 63%  159M 4s
60950K … … … … … 63%  149M 4s
61000K … … … … … 63% 93.1T 4s
61050K … … … … … 63% 61.6M 4s
61100K … … … … … 63%  131M 4s
```

```
61150K … … … … … 63% 4.16M 4s
61200K … … … … … 63% 99.1M 4s
61250K … … … … … 63% 93.1T 4s
61300K … … … … … 63%  111M 4s
61350K … … … … … 63% 99.5M 4s
61400K … … … … … 63%  108M 4s
61450K … … … … … 63% 93.1T 4s
61500K … … … … … 63%  146M 4s
61550K … … … … … 63%  138M 4s
61600K … … … … … 63% 57.9M 4s
61650K … … … … … 63%  137M 4s
61700K … … … … … 63%  138K 4s
61750K … … … … … 64% 93.1T 4s
61800K … … … … … 64%  108M 4s
61850K … … … … … 64%  123M 4s
61900K … … … … … 64%  115M 4s
61950K … … … … … 64%  120M 4s
62000K … … … … … 64% 93.1T 4s
62050K … … … … … 64%  137M 4s
62100K … … … … … 64% 54.5M 4s
62150K … … … … … 64%  114M 4s
62200K … … … … … 64% 93.1T 4s
62250K … … … … … 64%  142M 4s
62300K … … … … … 64%  149M 4s
62350K … … … … … 64%  119M 4s
62400K … … … … … 64%  175M 4s
62450K … … … … … 64% 4.29M 4s
62500K … … … … … 64%  123M 4s
62550K … … … … … 64%  125M 4s
62600K … … … … … 64% 44.2M 4s
62650K … … … … … 64% 93.1T 4s
62700K … … … … … 65%  112M 4s
62750K … … … … … 65%  140M 4s
62800K … … … … … 65% 87.7M 4s
62850K … … … … … 65%  170M 4s
62900K … … … … … 65% 93.1T 4s
62950K … … … … … 65%  184M 4s
63000K … … … … … 65%  164M 4s
63050K … … … … … 65% 32.4M 4s
63100K … … … … … 65% 45.5M 4s
63150K … … … … … 65% 93.1T 4s
63200K … … … … … 65% 4.76M 4s
63250K … … … … … 65% 81.3M 4s
63300K … … … … … 65% 93.1T 4s
63350K … … … … … 65%  120M 4s
63400K … … … … … 65% 79.3M 4s
63450K … … … … … 65% 88.3M 4s
63500K … … … … … 65% 71.9M 4s
```

```
63550K … … … … … 65% 93.1T 4s
63600K … … … … … 65%  126M 4s
63650K … … … … … 66% 61.2M 4s
63700K … … … … … 66%  150M 4s
63750K … … … … … 66%  138K 4s
63800K … … … … … 66% 73.7M 4s
63850K … … … … … 66%  141M 4s
63900K … … … … … 66%  147M 4s
63950K … … … … … 66% 93.1T 4s
64000K … … … … … 66%  107M 4s
64050K … … … … … 66%  165M 4s
64100K … … … … … 66%  161M 4s
64150K … … … … … 66% 28.9M 4s
64200K … … … … … 66% 51.5M 4s
64250K … … … … … 66%  141M 4s
64300K … … … … … 66%  115M 4s
64350K … … … … … 66% 95.0M 4s
64400K … … … … … 66%  122M 4s
64450K … … … … … 66% 93.1T 4s
64500K … … … … … 66% 4.95M 4s
64550K … … … … … 66% 93.1T 4s
64600K … … … … … 66%  111M 4s
64650K … … … … … 67%  139M 4s
64700K … … … … … 67% 49.7M 4s
64750K … … … … … 67% 93.1T 4s
64800K … … … … … 67%  129M 4s
64850K … … … … … 67% 31.8M 4s
64900K … … … … … 67% 42.3M 4s
64950K … … … … … 67% 93.1T 4s
65000K … … … … … 67% 59.6M 4s
65050K … … … … … 67%  114M 4s
65100K … … … … … 67%  114M 4s
65150K … … … … … 67%  143M 4s
65200K … … … … … 67%  162M 4s
65250K … … … … … 67% 5.32M 4s
65300K … … … … … 67%  123M 4s
65350K … … … … … 67%  119M 4s
65400K … … … … … 67%  135M 4s
65450K … … … … … 67% 78.8M 4s
65500K … … … … … 67%  129M 4s
65550K … … … … … 67% 35.7M 4s
65600K … … … … … 68%  104M 4s
65650K … … … … … 68% 43.3M 4s
65700K … … … … … 68% 95.2M 4s
65750K … … … … … 68% 50.8M 4s
65800K … … … … … 68%  139K 4s
65850K … … … … … 68% 93.1T 4s
65900K … … … … … 68%  105M 4s
```

```
65950K … … … … … 68%  163M 4s
66000K … … … … … 68%  163M 4s
66050K … … … … … 68%  161M 4s
66100K … … … … … 68% 93.1T 4s
66150K … … … … … 68%  172M 4s
66200K … … … … … 68%  123M 4s
66250K … … … … … 68%  141M 4s
66300K … … … … … 68% 93.1T 4s
66350K … … … … … 68% 59.9M 4s
66400K … … … … … 68%  168M 4s
66450K … … … … … 68%  176M 4s
66500K … … … … … 68%  165M 4s
66550K … … … … … 69% 4.04M 4s
66600K … … … … … 69% 93.1T 4s
66650K … … … … … 69% 97.4M 4s
66700K … … … … … 69%  135M 4s
66750K … … … … … 69%  155M 4s
66800K … … … … … 69% 93.1T 4s
66850K … … … … … 69% 50.7M 4s
66900K … … … … … 69%  117M 4s
66950K … … … … … 69%  130M 4s
67000K … … … … … 69%  123M 4s
67050K … … … … … 69% 93.1T 4s
67100K … … … … … 69%  140M 4s
67150K … … … … … 69%  159M 4s
67200K … … … … … 69%  177M 4s
67250K … … … … … 69% 63.5M 4s
67300K … … … … … 69% 4.31M 4s
67350K … … … … … 69% 93.1T 4s
67400K … … … … … 69% 37.1M 4s
67450K … … … … … 69%  127M 4s
67500K … … … … … 70%  115M 4s
67550K … … … … … 70% 93.1T 4s
67600K … … … … … 70%  128M 3s
67650K … … … … … 70%  137M 3s
67700K … … … … … 70%  152M 3s
67750K … … … … … 70%  150M 3s
67800K … … … … … 70% 93.1T 3s
67850K … … … … … 70%  138K 4s
67900K … … … … … 70% 42.1M 4s
67950K … … … … … 70% 93.1T 4s
68000K … … … … … 70%  127M 4s
68050K … … … … … 70%  174M 4s
68100K … … … … … 70%  200M 4s
68150K … … … … … 70% 93.1T 4s
68200K … … … … … 70%  181M 4s
68250K … … … … … 70%  175M 4s
68300K … … … … … 70%  223M 4s
```

```
68350K … … … … … 70%  232M 4s
68400K … … … … … 70% 93.1T 4s
68450K … … … … … 70% 59.8M 4s
68500K … … … … … 71%  204M 3s
68550K … … … … … 71%  223M 3s
68600K … … … … … 71% 4.05M 3s
68650K … … … … … 71% 73.3M 3s
68700K … … … … … 71%  148M 3s
68750K … … … … … 71% 93.1T 3s
68800K … … … … … 71%  146M 3s
68850K … … … … … 71%  138M 3s
68900K … … … … … 71%  126M 3s
68950K … … … … … 71% 93.1T 3s
69000K … … … … … 71% 66.7M 3s
69050K … … … … … 71%  141M 3s
69100K … … … … … 71%  148M 3s
69150K … … … … … 71%  147M 3s
69200K … … … … … 71% 93.1T 3s
69250K … … … … … 71%  142M 3s
69300K … … … … … 71%  157M 3s
69350K … … … … … 71% 4.15M 3s
69400K … … … … … 71% 68.7M 3s
69450K … … … … … 72% 97.8M 3s
69500K … … … … … 72% 52.5M 3s
69550K … … … … … 72%  123M 3s
69600K … … … … … 72% 93.1T 3s
69650K … … … … … 72%  145M 3s
69700K … … … … … 72%  131M 3s
69750K … … … … … 72%  156M 3s
69800K … … … … … 72% 93.1T 3s
69850K … … … … … 72%  139M 3s
69900K … … … … … 72%  137K 3s
69950K … … … … … 72% 68.8M 3s
70000K … … … … … 72% 93.1T 3s
70050K … … … … … 72% 58.5M 3s
70100K … … … … … 72%  158M 3s
70150K … … … … … 72%  173M 3s
70200K … … … … … 72%  209M 3s
70250K … … … … … 72% 93.1T 3s
70300K … … … … … 72%  211M 3s
70350K … … … … … 72%  204M 3s
70400K … … … … … 73%  207M 3s
70450K … … … … … 73% 93.1T 3s
70500K … … … … … 73%  204M 3s
70550K … … … … … 73% 73.9M 3s
70600K … … … … … 73%  212M 3s
70650K … … … … … 73% 4.04M 3s
70700K … … … … … 73% 94.9M 3s
```

```
70750K … … … … … 73% 93.1T 3s
70800K … … … … … 73%  127M 3s
70850K … … … … … 73%  167M 3s
70900K … … … … … 73%  152M 3s
70950K … … … … … 73% 93.1T 3s
71000K … … … … … 73%  137M 3s
71050K … … … … … 73%  169M 3s
71100K … … … … … 73% 60.3M 3s
71150K … … … … … 73%  167M 3s
71200K … … … … … 73% 93.1T 3s
71250K … … … … … 73%  166M 3s
71300K … … … … … 73%  175M 3s
71350K … … … … … 73%  152M 3s
71400K … … … … … 74% 4.04M 3s
71450K … … … … … 74% 71.5M 3s
71500K … … … … … 74%  101M 3s
71550K … … … … … 74% 60.5M 3s
71600K … … … … … 74% 93.1T 3s
71650K … … … … … 74%  131M 3s
71700K … … … … … 74%  162M 3s
71750K … … … … … 74%  172M 3s
71800K … … … … … 74% 93.1T 3s
71850K … … … … … 74%  160M 3s
71900K … … … … … 74%  155M 3s
71950K … … … … … 74%  137K 3s
72000K … … … … … 74% 74.7M 3s
72050K … … … … … 74% 93.1T 3s
72100K … … … … … 74%  142M 3s
72150K … … … … … 74% 57.7M 3s
72200K … … … … … 74%  202M 3s
72250K … … … … … 74%  207M 3s
72300K … … … … … 74% 93.1T 3s
72350K … … … … … 75%  226M 3s
72400K … … … … … 75%  229M 3s
72450K … … … … … 75%  222M 3s
72500K … … … … … 75% 93.1T 3s
72550K … … … … … 75%  212M 3s
72600K … … … … … 75%  234M 3s
72650K … … … … … 75% 70.6M 3s
72700K … … … … … 75% 4.01M 3s
72750K … … … … … 75% 25.3M 3s
72800K … … … … … 75% 93.1T 3s
72850K … … … … … 75%  139M 3s
72900K … … … … … 75% 51.1M 3s
72950K … … … … … 75%  167M 3s
73000K … … … … … 75% 93.1T 3s
73050K … … … … … 75%  166M 3s
73100K … … … … … 75%  152M 3s
```

```
73150K … … … … … 75% 60.8M 3s
73200K … … … … … 75%  142M 3s
73250K … … … … … 75% 93.1T 3s
73300K … … … … … 76%  171M 3s
73350K … … … … … 76%  188M 3s
73400K … … … … … 76%  137M 3s
73450K … … … … … 76% 4.82M 3s
73500K … … … … … 76% 12.7M 3s
73550K … … … … … 76% 93.1T 3s
73600K … … … … … 76%  135M 3s
73650K … … … … … 76%  141M 3s
73700K … … … … … 76% 58.0M 3s
73750K … … … … … 76% 93.1T 3s
73800K … … … … … 76%  125M 3s
73850K … … … … … 76%  151M 3s
73900K … … … … … 76%  131M 3s
73950K … … … … … 76%  147M 3s
74000K … … … … … 76%  139K 3s
74050K … … … … … 76% 84.8M 3s
74100K … … … … … 76% 93.1T 3s
74150K … … … … … 76%  118M 3s
74200K … … … … … 76%  143M 3s
74250K … … … … … 77% 68.0M 3s
74300K … … … … … 77%  195M 3s
74350K … … … … … 77% 93.1T 3s
74400K … … … … … 77%  196M 3s
74450K … … … … … 77%  221M 3s
74500K … … … … … 77%  210M 3s
74550K … … … … … 77% 93.1T 3s
74600K … … … … … 77%  217M 3s
74650K … … … … … 77%  221M 3s
74700K … … … … … 77%  228M 3s
74750K … … … … … 77% 3.83M 3s
74800K … … … … … 77% 93.1T 3s
74850K … … … … … 77% 54.4M 3s
74900K … … … … … 77%  116M 3s
74950K … … … … … 77%  128M 3s
75000K … … … … … 77%  160M 3s
75050K … … … … … 77% 93.1T 3s
75100K … … … … … 77%  173M 3s
75150K … … … … … 77%  174M 3s
75200K … … … … … 77%  165M 3s
75250K … … … … … 78% 93.1T 3s
75300K … … … … … 78%  177M 3s
75350K … … … … … 78% 62.1M 3s
75400K … … … … … 78%  161M 3s
75450K … … … … … 78%  148M 3s
75500K … … … … … 78% 4.16M 3s
```

```
75550K … … … … … 78%  121M 3s
75600K … … … … … 78% 84.9M 3s
75650K … … … … … 78% 93.1T 3s
75700K … … … … … 78%  133M 3s
75750K … … … … … 78%  116M 3s
75800K … … … … … 78% 67.2M 3s
75850K … … … … … 78% 93.1T 3s
75900K … … … … … 78%  163M 3s
75950K … … … … … 78%  165M 3s
76000K … … … … … 78%  170M 3s
76050K … … … … … 78%  137K 3s
76100K … … … … … 78% 93.1T 3s
76150K … … … … … 78% 95.3M 3s
76200K … … … … … 79%  128M 3s
76250K … … … … … 79%  167M 3s
76300K … … … … … 79%  159M 3s
76350K … … … … … 79% 93.1T 3s
76400K … … … … … 79% 61.1M 3s
76450K … … … … … 79%  134M 3s
76500K … … … … … 79%  154M 3s
76550K … … … … … 79% 93.1T 3s
76600K … … … … … 79%  145M 3s
76650K … … … … … 79%  148M 3s
76700K … … … … … 79%  169M 3s
76750K … … … … … 79%  171M 3s
76800K … … … … … 79% 4.24M 3s
76850K … … … … … 79% 73.4M 3s
76900K … … … … … 79% 47.8M 3s
76950K … … … … … 79%  139M 3s
77000K … … … … … 79% 93.1T 3s
77050K … … … … … 79%  156M 3s
77100K … … … … … 79%  152M 3s
77150K … … … … … 80%  177M 3s
77200K … … … … … 80%  179M 3s
77250K … … … … … 80% 93.1T 3s
77300K … … … … … 80%  170M 3s
77350K … … … … … 80%  180M 3s
77400K … … … … … 80% 66.3M 3s
77450K … … … … … 80% 96.3M 2s
77500K … … … … … 80% 93.1T 2s
77550K … … … … … 80% 4.33M 2s
77600K … … … … … 80% 49.2M 2s
77650K … … … … … 80%  105M 2s
77700K … … … … … 80% 93.1T 2s
77750K … … … … … 80% 95.7M 2s
77800K … … … … … 80%  139M 2s
77850K … … … … … 80%  154M 2s
77900K … … … … … 80%  170M 2s
```

```
77950K … … … … … 80% 93.1T 2s
78000K … … … … … 80% 64.0M 2s
78050K … … … … … 80%  159M 2s
78100K … … … … … 80%  137K 2s
78150K … … … … … 81% 71.7M 2s
78200K … … … … … 81%  126M 2s
78250K … … … … … 81% 93.1T 2s
78300K … … … … … 81%  174M 2s
78350K … … … … … 81%  185M 2s
78400K … … … … … 81%  221M 2s
78450K … … … … … 81% 93.1T 2s
78500K … … … … … 81% 28.7M 2s
78550K … … … … … 81%  186M 2s
78600K … … … … … 81%  227M 2s
78650K … … … … … 81%  207M 2s
78700K … … … … … 81% 93.1T 2s
78750K … … … … … 81%  225M 2s
78800K … … … … … 81%  250M 2s
78850K … … … … … 81% 4.11M 2s
78900K … … … … … 81% 36.5M 2s
78950K … … … … … 81%  127M 2s
79000K … … … … … 81% 54.3M 2s
79050K … … … … … 81% 93.1T 2s
79100K … … … … … 82%  126M 2s
79150K … … … … … 82%  142M 2s
79200K … … … … … 82%  152M 2s
79250K … … … … … 82%  148M 2s
79300K … … … … … 82% 93.1T 2s
79350K … … … … … 82%  138M 2s
79400K … … … … … 82%  148M 2s
79450K … … … … … 82%  145M 2s
79500K … … … … … 82% 93.1T 2s
79550K … … … … … 82% 60.7M 2s
79600K … … … … … 82%  116M 2s
79650K … … … … … 82% 4.24M 2s
79700K … … … … … 82%  102M 2s
79750K … … … … … 82%  112M 2s
79800K … … … … … 82% 93.1T 2s
79850K … … … … … 82% 98.2M 2s
79900K … … … … … 82%  149M 2s
79950K … … … … … 82%  144M 2s
80000K … … … … … 82% 93.1T 2s
80050K … … … … … 83%  139M 2s
80100K … … … … … 83% 50.8M 2s
80150K … … … … … 83%  138K 2s
80200K … … … … … 83% 85.9M 2s
80250K … … … … … 83% 93.1T 2s
80300K … … … … … 83%  127M 2s
```

```
80350K … … … … … 83%  164M 2s
80400K … … … … … 83%  147M 2s
80450K … … … … … 83%  166M 2s
80500K … … … … … 83% 93.1T 2s
80550K … … … … … 83%  164M 2s
80600K … … … … … 83%  148M 2s
80650K … … … … … 83% 55.6M 2s
80700K … … … … … 83% 93.1T 2s
80750K … … … … … 83%  163M 2s
80800K … … … … … 83%  179M 2s
80850K … … … … … 83%  173M 2s
80900K … … … … … 83% 4.19M 2s
80950K … … … … … 83% 87.3M 2s
81000K … … … … … 83%  123M 2s
81050K … … … … … 84%  142M 2s
81100K … … … … … 84% 57.3M 2s
81150K … … … … … 84% 93.1T 2s
81200K … … … … … 84%  136M 2s
81250K … … … … … 84%  169M 2s
81300K … … … … … 84%  177M 2s
81350K … … … … … 84% 93.1T 2s
81400K … … … … … 84%  176M 2s
81450K … … … … … 84%  174M 2s
81500K … … … … … 84%  176M 2s
81550K … … … … … 84%  165M 2s
81600K … … … … … 84% 93.1T 2s
81650K … … … … … 84% 70.4M 2s
81700K … … … … … 84% 4.01M 2s
81750K … … … … … 84%  105M 2s
81800K … … … … … 84%  107M 2s
81850K … … … … … 84% 93.1T 2s
81900K … … … … … 84%  153M 2s
81950K … … … … … 84%  154M 2s
82000K … … … … … 85%  168M 2s
82050K … … … … … 85%  154M 2s
82100K … … … … … 85% 93.1T 2s
82150K … … … … … 85%  161M 2s
82200K … … … … … 85%  137K 2s
82250K … … … … … 85% 66.3M 2s
82300K … … … … … 85% 99.1M 2s
82350K … … … … … 85%  130M 2s
82400K … … … … … 85%  143M 2s
82450K … … … … … 85% 93.1T 2s
82500K … … … … … 85%  163M 2s
82550K … … … … … 85%  154M 2s
82600K … … … … … 85%  172M 2s
82650K … … … … … 85% 64.4M 2s
82700K … … … … … 85% 93.1T 2s
```

```
82750K … … … … … 85%  128M 2s
82800K … … … … … 85%  145M 2s
82850K … … … … … 85%  154M 2s
82900K … … … … … 85% 93.1T 2s
82950K … … … … … 86% 4.17M 2s
83000K … … … … … 86% 71.5M 2s
83050K … … … … … 86% 93.1T 2s
83100K … … … … … 86%  146M 2s
83150K … … … … … 86%  117M 2s
83200K … … … … … 86% 53.0M 2s
83250K … … … … … 86%  157M 2s
83300K … … … … … 86% 93.1T 2s
83350K … … … … … 86%  175M 2s
83400K … … … … … 86%  176M 2s
83450K … … … … … 86% 87.6M 2s
83500K … … … … … 86% 93.1T 2s
83550K … … … … … 86%  170M 2s
83600K … … … … … 86%  168M 2s
83650K … … … … … 86%  171M 2s
83700K … … … … … 86% 73.1M 2s
83750K … … … … … 86% 4.16M 2s
83800K … … … … … 86% 73.5M 2s
83850K … … … … … 86%  113M 2s
83900K … … … … … 87% 93.1T 2s
83950K … … … … … 87%  141M 2s
84000K … … … … … 87%  155M 2s
84050K … … … … … 87%  166M 2s
84100K … … … … … 87%  161M 2s
84150K … … … … … 87% 93.1T 2s
84200K … … … … … 87%  159M 2s
84250K … … … … … 87%  137K 2s
84300K … … … … … 87% 88.0M 2s
84350K … … … … … 87% 93.1T 2s
84400K … … … … … 87%  124M 2s
84450K … … … … … 87%  143M 2s
84500K … … … … … 87%  166M 2s
84550K … … … … … 87% 93.1T 2s
84600K … … … … … 87%  169M 2s
84650K … … … … … 87%  165M 2s
84700K … … … … … 87%  172M 2s
84750K … … … … … 87% 59.1M 2s
84800K … … … … … 87% 93.1T 2s
84850K … … … … … 87%  155M 2s
84900K … … … … … 88%  162M 2s
84950K … … … … … 88%  157M 2s
85000K … … … … … 88% 4.22M 2s
85050K … … … … … 88% 74.9M 2s
85100K … … … … … 88%  128M 2s
```

```
85150K … … … … … 88% 72.0M 2s
85200K … … … … … 88% 93.1T 2s
85250K … … … … … 88%  123M 2s
85300K … … … … … 88% 59.0M 2s
85350K … … … … … 88%  139M 2s
85400K … … … … … 88%  168M 2s
85450K … … … … … 88% 93.1T 2s
85500K … … … … … 88%  169M 2s
85550K … … … … … 88%  173M 1s
85600K … … … … … 88%  168M 1s
85650K … … … … … 88% 93.1T 1s
85700K … … … … … 88%  170M 1s
85750K … … … … … 88%  181M 1s
85800K … … … … … 88% 4.10M 1s
85850K … … … … … 89% 43.4M 1s
85900K … … … … … 89%  113M 1s
85950K … … … … … 89%  152M 1s
86000K … … … … … 89%  163M 1s
86050K … … … … … 89% 93.1T 1s
86100K … … … … … 89%  152M 1s
86150K … … … … … 89%  162M 1s
86200K … … … … … 89%  165M 1s
86250K … … … … … 89%  157M 1s
86300K … … … … … 89%  137K 1s
86350K … … … … … 89% 37.3M 1s
86400K … … … … … 89%  147M 1s
86450K … … … … … 89% 93.1T 1s
86500K … … … … … 89%  188M 1s
86550K … … … … … 89%  218M 1s
86600K … … … … … 89%  226M 1s
86650K … … … … … 89%  220M 1s
86700K … … … … … 89% 93.1T 1s
86750K … … … … … 89%  213M 1s
86800K … … … … … 90%  194M 1s
86850K … … … … … 90% 75.8M 1s
86900K … … … … … 90% 93.1T 1s
86950K … … … … … 90%  136M 1s
87000K … … … … … 90%  211M 1s
87050K … … … … … 90% 4.05M 1s
87100K … … … … … 90% 73.5M 1s
87150K … … … … … 90%  148M 1s
87200K … … … … … 90%  137M 1s
87250K … … … … … 90% 93.1T 1s
87300K … … … … … 90%  110M 1s
87350K … … … … … 90%  142M 1s
87400K … … … … … 90% 61.9M 1s
87450K … … … … … 90% 93.1T 1s
87500K … … … … … 90%  145M 1s
```

```
87550K … … … … … 90%  143M 1s
87600K … … … … … 90%  156M 1s
87650K … … … … … 90%  167M 1s
87700K … … … … … 90% 93.1T 1s
87750K … … … … … 90%  149M 1s
87800K … … … … … 91%  175M 1s
87850K … … … … … 91% 4.02M 1s
87900K … … … … … 91%  108M 1s
87950K … … … … … 91% 55.5M 1s
88000K … … … … … 91%  150M 1s
88050K … … … … … 91%  172M 1s
88100K … … … … … 91% 93.1T 1s
88150K … … … … … 91%  170M 1s
88200K … … … … … 91%  168M 1s
88250K … … … … … 91%  112M 1s
88300K … … … … … 91% 93.1T 1s
88350K … … … … … 91%  137K 1s
88400K … … … … … 91% 79.5M 1s
88450K … … … … … 91% 93.1T 1s
88500K … … … … … 91% 51.9M 1s
88550K … … … … … 91%  162M 1s
88600K … … … … … 91%  157M 1s
88650K … … … … … 91%  171M 1s
88700K … … … … … 91% 93.1T 1s
88750K … … … … … 92%  170M 1s
88800K … … … … … 92%  171M 1s
88850K … … … … … 92%  153M 1s
88900K … … … … … 92% 93.1T 1s
88950K … … … … … 92%  164M 1s
89000K … … … … … 92% 59.1M 1s
89050K … … … … … 92%  158M 1s
89100K … … … … … 92% 4.41M 1s
89150K … … … … … 92% 83.6M 1s
89200K … … … … … 92%  123M 1s
89250K … … … … … 92%  120M 1s
89300K … … … … … 92% 93.1T 1s
89350K … … … … … 92%  102M 1s
89400K … … … … … 92%  163M 1s
89450K … … … … … 92%  174M 1s
89500K … … … … … 92% 93.1T 1s
89550K … … … … … 92% 61.8M 1s
89600K … … … … … 92%  124M 1s
89650K … … … … … 92%  151M 1s
89700K … … … … … 93%  152M 1s
89750K … … … … … 93% 93.1T 1s
89800K … … … … … 93%  162M 1s
89850K … … … … … 93%  176M 1s
89900K … … … … … 93% 3.89M 1s
```

```
89950K … … … … … 93% 93.1T 1s
90000K … … … … … 93%  126M 1s
90050K … … … … … 93% 58.7M 1s
90100K … … … … … 93%  169M 1s
90150K … … … … … 93% 93.1T 1s
90200K … … … … … 93%  167M 1s
90250K … … … … … 93%  161M 1s
90300K … … … … … 93%  180M 1s
90350K … … … … … 93%  177M 1s
90400K … … … … … 93%  137K 1s
90450K … … … … … 93% 72.6M 1s
90500K … … … … … 93%  139M 1s
90550K … … … … … 93% 98.7M 1s
90600K … … … … … 93% 55.9M 1s
90650K … … … … … 93% 93.1T 1s
90700K … … … … … 94%  131M 1s
90750K … … … … … 94% 52.0M 1s
90800K … … … … … 94%  137M 1s
90850K … … … … … 94% 93.1T 1s
90900K … … … … … 94%  138M 1s
90950K … … … … … 94%  169M 1s
91000K … … … … … 94%  181M 1s
91050K … … … … … 94%  160M 1s
91100K … … … … … 94% 93.1T 1s
91150K … … … … … 94% 4.39M 1s
91200K … … … … … 94%  109M 1s
91250K … … … … … 94% 93.1T 1s
91300K … … … … … 94%  121M 1s
91350K … … … … … 94%  150M 1s
91400K … … … … … 94%  127M 1s
91450K … … … … … 94% 93.1T 1s
91500K … … … … … 94%  146M 1s
91550K … … … … … 94% 63.4M 1s
91600K … … … … … 94% 57.3M 1s
91650K … … … … … 95%  176M 1s
91700K … … … … … 95%  118M 1s
91750K … … … … … 95% 29.3M 1s
91800K … … … … … 95% 93.1T 1s
91850K … … … … … 95%  165M 1s
91900K … … … … … 95%  180M 1s
91950K … … … … … 95% 4.78M 1s
92000K … … … … … 95% 97.8M 1s
92050K … … … … … 95%  123M 1s
92100K … … … … … 95% 93.1T 1s
92150K … … … … … 95% 60.1M 1s
92200K … … … … … 95%  126M 1s
92250K … … … … … 95%  139M 1s
92300K … … … … … 95%  174M 1s
```

```
92350K … … … … … 95% 93.1T 1s
92400K … … … … … 95% 67.5M 1s
92450K … … … … … 95%  137K 1s
92500K … … … … … 95%  114M 1s
92550K … … … … … 95% 93.1T 1s
92600K … … … … … 96%  128M 1s
92650K … … … … … 96% 47.0M 1s
92700K … … … … … 96%  154M 1s
92750K … … … … … 96% 93.1T 1s
92800K … … … … … 96% 19.9M 1s
92850K … … … … … 96%  120M 1s
92900K … … … … … 96% 93.1T 1s
92950K … … … … … 96%  135M 1s
93000K … … … … … 96%  133M 0s
93050K … … … … … 96%  156M 0s
93100K … … … … … 96%  146M 0s
93150K … … … … … 96% 93.1T 0s
93200K … … … … … 96% 5.19M 0s
93250K … … … … … 96% 73.3M 0s
93300K … … … … … 96% 93.1T 0s
93350K … … … … … 96%  134M 0s
93400K … … … … … 96%  141M 0s
93450K … … … … … 96%  144M 0s
93500K … … … … … 96%  126M 0s
93550K … … … … … 97% 15.6M 0s
93600K … … … … … 97% 93.1T 0s
93650K … … … … … 97% 85.3M 0s
93700K … … … … … 97%  141M 0s
93750K … … … … … 97% 56.0M 0s
93800K … … … … … 97%  170M 0s
93850K … … … … … 97% 93.1T 0s
93900K … … … … … 97% 55.4M 0s
93950K … … … … … 97%  191M 0s
94000K … … … … … 97% 5.60M 0s
94050K … … … … … 97% 76.9M 0s
94100K … … … … … 97%  107M 0s
94150K … … … … … 97% 93.1T 0s
94200K … … … … … 97%  149M 0s
94250K … … … … … 97% 56.7M 0s
94300K … … … … … 97% 40.2M 0s
94350K … … … … … 97% 49.1M 0s
94400K … … … … … 97% 72.5M 0s
94450K … … … … … 97%  147M 0s
94500K … … … … … 97%  138K 0s
94550K … … … … … 98% 72.5M 0s
94600K … … … … … 98%  117M 0s
94650K … … … … … 98%  153M 0s
94700K … … … … … 98% 93.1T 0s
```

```
 94750K … … … … … 98%  206M 0s
 94800K … … … … … 98% 64.2M 0s
 94850K … … … … … 98%  178M 0s
 94900K … … … … … 98% 93.1T 0s
 94950K … … … … … 98%  204M 0s
 95000K … … … … … 98%  215M 0s
 95050K … … … … … 98%  179M 0s
 95100K … … … … … 98%  206M 0s
 95150K … … … … … 98% 93.1T 0s
 95200K … … … … … 98%  214M 0s
 95250K … … … … … 98% 3.93M 0s
 95300K … … … … … 98% 87.9M 0s
 95350K … … … … … 98% 49.0M 0s
 95400K … … … … … 98%  124M 0s
 95450K … … … … … 98%  142M 0s
 95500K … … … … … 99% 93.1T 0s
 95550K … … … … … 99%  164M 0s
 95600K … … … … … 99%  167M 0s
 95650K … … … … … 99%  176M 0s
 95700K … … … … … 99% 93.1T 0s
 95750K … … … … … 99%  178M 0s
 95800K … … … … … 99%  168M 0s
 95850K … … … … … 99% 71.6M 0s
 95900K … … … … … 99%  128M 0s
 95950K … … … … … 99% 93.1T 0s
 96000K … … … … … 99%  166M 0s
 96050K … … … … … 99% 4.13M 0s
 96100K … … … … … 99% 55.5M 0s
 96150K … … … … … 99%  144M 0s
 96200K … … … … … 99% 93.1T 0s
 96250K … … … … … 99%  147M 0s
 96300K … … … … … 99%  161M 0s
 96350K … … … … … 99%  149M 0s
 96400K … … … … … 99% 93.1T 0s
 96450K … … … … ..         100% 46.7M=14s

2026-11-14 11:43:38 (6.83 MB/s) - ‘pretrained_model/pretrained_simclr_model.pth’
saved [98808459/98808459]
```

[4]:
```
# Setup cell.
%pip install thop
import torch
import os
import sys
import types
import importlib
```

```
import pandas as pd
import numpy as np
import torch.optim as optim
import torch.nn as nn
import random
from thop import profile, clever_format
from torch.utils.data import DataLoader
from torchvision.datasets import CIFAR10
import matplotlib.pyplot as plt
%matplotlib inline

if "imp" not in sys.modules:
    imp = types.ModuleType("imp")
    imp.reload = importlib.reload
    sys.modules["imp"] = imp

%load_ext autoreload
%autoreload 2

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

```
Collecting thop
  Downloading thop-0.1.1.post2209072238-py3-none-any.whl.metadata (2.7 kB)
Requirement already satisfied: torch in /usr/local/lib/python3.13/dist-packages
(from thop) (2.14.1+cu130)
Requirement already satisfied: filelock in /usr/local/lib/python3.13/dist-
packages (from torch->thop) (4.2.7)
Requirement already satisfied: typing-extensions>=4.10.0 in
/usr/local/lib/python3.13/dist-packages (from torch->thop) (4.16.0)
Requirement already satisfied: setuptools>=77.0.3 in
/usr/local/lib/python3.13/dist-packages (from torch->thop) (80.10.2)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.13/dist-
packages (from torch->thop) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in
/usr/local/lib/python3.13/dist-packages (from torch->thop) (3.7)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.13/dist-packages
(from torch->thop) (3.1.6)
Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.13/dist-
packages (from torch->thop) (2025.12.0)
Requirement already satisfied: cuda-toolkit==13.0.3 in
/usr/local/lib/python3.13/dist-packages (from cuda-toolkit[cublas,cudart,cufft,c
ufile,cupti,curand,cusolver,cusparse,nvjitlink,nvrtc,nvtx]==13.0.3;
platform_system == "Linux"->torch->thop) (13.0.3.0)
Requirement already satisfied: cuda-bindings<14,>=13.0.3 in
/usr/local/lib/python3.13/dist-packages (from torch->thop) (13.0.3)
Requirement already satisfied: nvidia-cudnn-cu13==9.24.0.43 in
/usr/local/lib/python3.13/dist-packages (from torch->thop) (9.24.0.43)
Requirement already satisfied: nvidia-cusparselt-cu13==0.8.1 in
```

```
/usr/local/lib/python3.13/dist-packages (from torch->thop) (0.8.1)
Requirement already satisfied: nvidia-nccl-cu13==2.30.7 in
/usr/local/lib/python3.13/dist-packages (from torch->thop) (2.30.7)
Requirement already satisfied: nvidia-nvshmem-cu13==3.4.5 in
/usr/local/lib/python3.13/dist-packages (from torch->thop) (3.4.5)
Requirement already satisfied: triton~=3.8.0 in /usr/local/lib/python3.13/dist-
packages (from torch->thop) (3.8.0)
Requirement already satisfied: nvidia-cublas==13.1.1.3.* in
/usr/local/lib/python3.13/dist-packages (from cuda-toolkit[cublas,cudart,cufft,c
ufile,cupti,curand,cusolver,cusparse,nvjitlink,nvrtc,nvtx]==13.0.3;
platform_system == "Linux"->torch->thop) (13.1.1.3)
Requirement already satisfied: nvidia-cuda-nvrtc==13.0.88.* in
/usr/local/lib/python3.13/dist-packages (from cuda-toolkit[cublas,cudart,cufft,c
ufile,cupti,curand,cusolver,cusparse,nvjitlink,nvrtc,nvtx]==13.0.3;
platform_system == "Linux"->torch->thop) (13.0.88)
Requirement already satisfied: nvidia-cuda-runtime==13.0.96.* in
/usr/local/lib/python3.13/dist-packages (from cuda-toolkit[cublas,cudart,cufft,c
ufile,cupti,curand,cusolver,cusparse,nvjitlink,nvrtc,nvtx]==13.0.3;
platform_system == "Linux"->torch->thop) (13.0.96)
Requirement already satisfied: nvidia-cufft==12.0.0.61.* in
/usr/local/lib/python3.13/dist-packages (from cuda-toolkit[cublas,cudart,cufft,c
ufile,cupti,curand,cusolver,cusparse,nvjitlink,nvrtc,nvtx]==13.0.3;
platform_system == "Linux"->torch->thop) (12.0.0.61)
Requirement already satisfied: nvidia-nvjitlink<14,>=13.0.88 in
/usr/local/lib/python3.13/dist-packages (from cuda-toolkit[cublas,cudart,cufft,c
ufile,cupti,curand,cusolver,cusparse,nvjitlink,nvrtc,nvtx]==13.0.3;
platform_system == "Linux"->torch->thop) (13.4.92)
Requirement already satisfied: nvidia-cufile==1.15.1.6.* in
/usr/local/lib/python3.13/dist-packages (from cuda-toolkit[cublas,cudart,cufft,c
ufile,cupti,curand,cusolver,cusparse,nvjitlink,nvrtc,nvtx]==13.0.3;
platform_system == "Linux"->torch->thop) (1.15.1.6)
Requirement already satisfied: nvidia-cuda-cupti==13.0.85.* in
/usr/local/lib/python3.13/dist-packages (from cuda-toolkit[cublas,cudart,cufft,c
ufile,cupti,curand,cusolver,cusparse,nvjitlink,nvrtc,nvtx]==13.0.3;
platform_system == "Linux"->torch->thop) (13.0.85)
Requirement already satisfied: nvidia-curand==10.4.0.35.* in
/usr/local/lib/python3.13/dist-packages (from cuda-toolkit[cublas,cudart,cufft,c
ufile,cupti,curand,cusolver,cusparse,nvjitlink,nvrtc,nvtx]==13.0.3;
platform_system == "Linux"->torch->thop) (10.4.0.35)
Requirement already satisfied: nvidia-cusolver==12.0.4.66.* in
/usr/local/lib/python3.13/dist-packages (from cuda-toolkit[cublas,cudart,cufft,c
ufile,cupti,curand,cusolver,cusparse,nvjitlink,nvrtc,nvtx]==13.0.3;
platform_system == "Linux"->torch->thop) (12.0.4.66)
Requirement already satisfied: nvidia-cusparse==12.6.3.3.* in
/usr/local/lib/python3.13/dist-packages (from cuda-toolkit[cublas,cudart,cufft,c
ufile,cupti,curand,cusolver,cusparse,nvjitlink,nvrtc,nvtx]==13.0.3;
platform_system == "Linux"->torch->thop) (12.6.3.3)
Requirement already satisfied: nvidia-nvtx==13.0.85.* in
```

```
/usr/local/lib/python3.13/dist-packages (from cuda-toolkit[cublas,cudart,cufft,c
ufile,cupti,curand,cusolver,cusparse,nvjitlink,nvrtc,nvtx]==13.0.3;
platform_system == "Linux"->torch->thop) (13.0.85)
Requirement already satisfied: cuda-pathfinder~=1.1 in
/usr/local/lib/python3.13/dist-packages (from cuda-
bindings<14,>=13.0.3->torch->thop) (1.8.5)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in
/usr/local/lib/python3.13/dist-packages (from sympy>=1.13.3->torch->thop)
(1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in
/usr/local/lib/python3.13/dist-packages (from jinja2->torch->thop) (3.0.4)
Downloading thop-0.1.1.post2209072238-py3-none-any.whl (15 kB)
Installing collected packages: thop
Successfully installed thop-0.1.1.post2209072238
```

2 Data Augmentation

Our first step is to perform data augmentation. Implement the `compute_train_transform()` function in `cs231n/simclr/data_utils.py` to apply the following random transformations:

1. Randomly resize and crop to 32x32.
2. Horizontally flip the image with probability 0.5
3. With a probability of 0.8, apply color jitter (see `compute_train_transform()` for definition)
4. With a probability of 0.2, convert the image to grayscale

Now complete `compute_train_transform()` and `CIFAR10Pair.__getitem__()` in `cs231n/simclr/data_utils.py` to apply the data augmentation transform and generate $\tilde{x}_i$ and $\tilde{x}_j$.

Test to make sure that your data augmentation code is correct:

[5]:
```
from cs231n.simclr.data_utils import *
from cs231n.simclr.contrastive_loss import *

answers = torch.load('simclr_sanity_check.key')
```

[6]:
```
from PIL import Image
import torchvision
from torchvision.datasets import CIFAR10

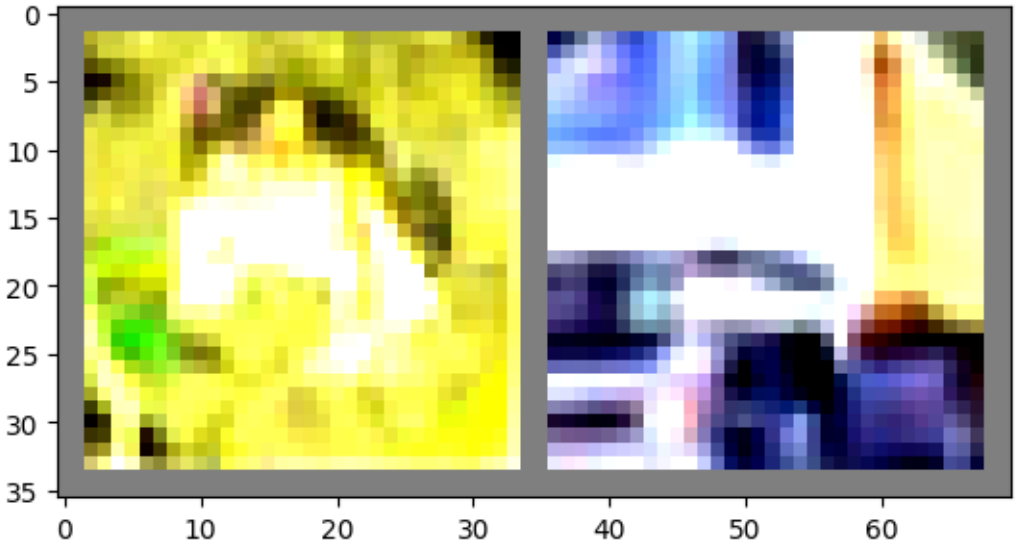
def test_data_augmentation(correct_output=None):
    train_transform = compute_train_transform(seed=2147483647)
    trainset = torchvision.datasets.CIFAR10(root='./data', train=True,␣
 ↪download=True, transform=train_transform)
    trainloader = torch.utils.data.DataLoader(trainset, batch_size=2,␣
 ↪shuffle=False, num_workers=2)
    dataiter = iter(trainloader)
    images, labels = next(dataiter)
```

```
    img = torchvision.utils.make_grid(images)
    img = img / 2 + 0.5     # unnormalize
    npimg = img.numpy()
    plt.imshow(np.transpose(npimg, (1, 2, 0)))
    plt.show()
    output = images

    print("Maximum error in data augmentation: %g"%rel_error( output.numpy(),␣
 ↪correct_output.numpy()))

# Should be less than 1e-07.
test_data_augmentation(answers['data_augmentation'])
```

```
WARNING:matplotlib.image:Clipping input data to the valid range for imshow with
RGB data ([0..1] for floats or [0..255] for integers). Got range
[-0.4334516..1.8768656].
```




```
Maximum error in data augmentation: 0
```

3 Base Encoder and Projection Head

The next steps are to apply the base encoder and projection head to the augmented samples $\tilde{x}_i$ and $\tilde{x}_j$.

The base encoder f extracts representation vectors for the augmented samples. The SimCLR paper found that using deeper and wider models improved performance and thus chose ResNet to use as the base encoder. The output of the base encoder are the representation vectors $h_i = f(\tilde{x}_i)$ and $h_j = f(\tilde{x}_j)$.

The projection head g is a small neural network that maps the representation vectors h_i and h_j to the space where the contrastive loss is applied. The paper found that using a nonlinear projection head improved the representation quality of the layer before it. Specifically, they used a MLP with one hidden layer as the projection head g. The contrastive loss is then computed based on the outputs $z_i = g(h_i)$ and $z_j = g(h_j)$.

We provide implementations of these two parts in `cs231n/simclr/model.py`. Please skim through the file and make sure you understand the implementation.

4 SimCLR: Contrastive Loss

A mini-batch of N training images yields a total of $2N$ data-augmented examples. For each positive pair (i, j) of augmented examples, the contrastive loss function aims to maximize the agreement of vectors z_i and z_j. Specifically, the loss is the normalized temperature-scaled cross entropy loss and aims to maximize the agreement of z_i and z_j relative to all other augmented examples in the batch:

$$l\ (i,j) = -\log \frac{\exp(\ \text{sim}(z_i, z_j)\ /\ \tau)}{\sum_{k=1}^{2N} \mathbb{1}_{k \neq i} \exp(\ \text{sim}(z_i, z_k)\ /\ \tau)}$$

where $\mathbb{1} \in \{0, 1\}$ is an indicator function that outputs 1 if $k \neq i$ and 0 otherwise. τ is a temperature parameter that determines how fast the exponentials increase.

$\text{sim}(z_i, z_j) = \frac{z_i \cdot z_j}{||z_i|| ||z_j||}$ is the (normalized) dot product between vectors z_i and z_j. The higher the similarity between z_i and z_j, the larger the dot product is, and the larger the numerator becomes. The denominator normalizes the value by summing across z_i and all other augmented examples k in the batch. The range of the normalized value is $(0, 1)$, where a high score close to 1 corresponds to a high similarity between the positive pair (i, j) and low similarity between i and other augmented examples k in the batch. The negative log then maps the range $(0, 1)$ to the loss values $(\text{inf}, 0)$.

The total loss is computed across all positive pairs (i, j) in the batch. Let $z = [z_1, z_2, ..., z_{2N}]$ include all the augmented examples in the batch, where $z_1 ... z_N$ are outputs of the left branch, and $z_{N+1} ... z_{2N}$ are outputs of the right branch. Thus, the positive pairs are (z_k, z_{k+N}) for $\forall k \in [1, N]$.

Then, the total loss L is:

$$L = \frac{1}{2N} \sum_{k=1}^{N} [\ l(k,\ k+N) + l(k+N,\ k)\ ]$$

NOTE: this equation is slightly different from the one in the paper. We've rearranged the ordering of the positive pairs in the batch, so the indices are different. The rearrangement makes it easier to implement the code in vectorized form.

We'll walk through the steps of implementing the loss function in vectorized form. Implement the functions `sim`, `simclr_loss_naive` in `cs231n/simclr/contrastive_loss.py`. Test your code by running the sanity checks below.

[7]:
```
from cs231n.simclr.contrastive_loss import *
answers = torch.load('simclr_sanity_check.key')
```

[8]:
```
def test_sim(left_vec, right_vec, correct_output):
    output = sim(left_vec, right_vec).cpu().numpy()
    print("Maximum error in sim: %g"%rel_error(correct_output.numpy(), output))

# Should be less than 1e-07.
test_sim(answers['left'][0], answers['right'][0], answers['sim'][0])
test_sim(answers['left'][1], answers['right'][1], answers['sim'][1])
```

```
Maximum error in sim: 3.81097e-08
Maximum error in sim: 0
```

[9]:
```
def test_loss_naive(left, right, tau, correct_output):
    naive_loss = simclr_loss_naive(left, right, tau).item()
    print("Maximum error in simclr_loss_naive: %g"%rel_error(correct_output,␣
 ↪naive_loss))

# Should be less than 1e-07.
test_loss_naive(answers['left'], answers['right'], 5.0, answers['loss']['5.0'])
test_loss_naive(answers['left'], answers['right'], 1.0, answers['loss']['1.0'])
```

```
Maximum error in simclr_loss_naive: 0
Maximum error in simclr_loss_naive: 1.13123e-07
```

Now implement the vectorized version by implementing `sim_positive_pairs`, `compute_sim_matrix`, `simclr_loss_vectorized` in `cs231n/simclr/contrastive_loss.py`. Test your code by running the sanity checks below.

[10]:
```
def test_sim_positive_pairs(left, right, correct_output):
    sim_pair = sim_positive_pairs(left, right).cpu().numpy()
    print("Maximum error in sim_positive_pairs: %g"%rel_error(correct_output.
 ↪numpy(), sim_pair))

# Should be less than 1e-07.
test_sim_positive_pairs(answers['left'], answers['right'], answers['sim'])
```

```
Maximum error in sim_positive_pairs: 3.81097e-08
```

[11]:
```
def test_sim_matrix(left, right, correct_output):
    out = torch.cat([left, right], dim=0)
    sim_matrix = compute_sim_matrix(out).cpu()
    assert torch.isclose(sim_matrix, correct_output).all(), "correct: {}. got:␣
 ↪{}".format(correct_output, sim_matrix)
    print("Test passed!")

test_sim_matrix(answers['left'], answers['right'], answers['sim_matrix'])
```

```
Test passed!
```

[12]:
```
def test_loss_vectorized(left, right, tau, correct_output):
    vec_loss = simclr_loss_vectorized(left, right, tau, device=left.device).
 ↪item()
    print("Maximum error in loss_vectorized: %g"%rel_error(correct_output,␣
 ↪vec_loss))

# Should be less than 1e-07.
test_loss_vectorized(answers['left'], answers['right'], 5.0, answers['loss']['5.
 ↪0'])
test_loss_vectorized(answers['left'], answers['right'], 1.0, answers['loss']['1.
 ↪0'])
```

```
Maximum error in loss_vectorized: 0
Maximum error in loss_vectorized: 5.65617e-08
```

5 Implement the train function

Complete the `train()` function in `cs231n/simclr/utils.py` to obtain the model's output and use `simclr_loss_vectorized` to compute the loss. (Please take a look at the `Model` class in `cs231n/simclr/model.py` to understand the model pipeline and the returned values)

[13]:
```
from cs231n.simclr.data_utils import *
from cs231n.simclr.model import *
from cs231n.simclr.utils import *
```

5.0.1 Train the SimCLR model

Run the following cells to load in the pretrained weights and continue to train a little bit more. This part will take ~10 minutes and will output to `pretrained_model/trained_simclr_model.pth`.

NOTE: Don't worry about logs such as '*[WARN] Cannot find rule for ...*'. These are related to another module used in the notebook. You can verify the integrity of your code changes through our provided prompts and comments.

[15]:
```
# Do not modify this cell.
feature_dim = 128
temperature = 0.5
k = 200
batch_size = 64
epochs = 1
temperature = 0.5
percentage = 0.5
pretrained_path = './pretrained_model/pretrained_simclr_model.pth'

# Prepare the data.
train_transform = compute_train_transform()
train_data = CIFAR10Pair(root='data', train=True, transform=train_transform,␣
 ↪download=True)
```

```
train_data = torch.utils.data.Subset(train_data, list(np.
 ↪arange(int(len(train_data)*percentage))))
train_loader = DataLoader(train_data, batch_size=batch_size, shuffle=True,␣
 ↪num_workers=16, pin_memory=True, drop_last=True)
test_transform = compute_test_transform()
memory_data = CIFAR10Pair(root='data', train=True, transform=test_transform,␣
 ↪download=True)
memory_loader = DataLoader(memory_data, batch_size=batch_size, shuffle=False,␣
 ↪num_workers=16, pin_memory=True)
test_data = CIFAR10Pair(root='data', train=False, transform=test_transform,␣
 ↪download=True)
test_loader = DataLoader(test_data, batch_size=batch_size, shuffle=False,␣
 ↪num_workers=16, pin_memory=True)

# Set up the model and optimizer config.
model = Model(feature_dim)
model.load_state_dict(torch.load(pretrained_path, map_location='cpu'),␣
 ↪strict=False)
model = model.to(device)
flops, params = profile(model, inputs=(torch.randn(1, 3, 32, 32).to(device),))
flops, params = clever_format([flops, params])
print('# Model Params: {} FLOPs: {}'.format(params, flops))
optimizer = optim.Adam(model.parameters(), lr=1e-3, weight_decay=1e-6)
c = len(memory_data.classes)

# Training loop.
results = {'train_loss': [], 'test_acc@1': [], 'test_acc@5': []} #<< -- output

if not os.path.exists('results'):
    os.mkdir('results')
best_acc = 0.0
for epoch in range(1, epochs + 1):
    train_loss = train(model, train_loader, optimizer, epoch, epochs,␣
 ↪batch_size=batch_size, temperature=temperature, device=device)
    results['train_loss'].append(train_loss)
    test_acc_1, test_acc_5 = test(model, memory_loader, test_loader, epoch,␣
 ↪epochs, c, k=k, temperature=temperature, device=device)
    results['test_acc@1'].append(test_acc_1)
    results['test_acc@5'].append(test_acc_5)

    # Save statistics.
    if test_acc_1 > best_acc:
        best_acc = test_acc_1
        torch.save(model.state_dict(), './pretrained_model/trained_simclr_model.
 ↪pth')
```

```
[INFO] Register count_convNd() for <class 'torch.nn.modules.conv.Conv2d'>.
```

```
[INFO] Register count_normalization() for <class
'torch.nn.modules.batchnorm.BatchNorm2d'>.
[INFO] Register zero_ops() for <class 'torch.nn.modules.activation.ReLU'>.
[INFO] Register zero_ops() for <class 'torch.nn.modules.container.Sequential'>.
[INFO] Register count_adap_avgpool() for <class
'torch.nn.modules.pooling.AdaptiveAvgPool2d'>.
[INFO] Register count_linear() for <class 'torch.nn.modules.linear.Linear'>.
[INFO] Register count_normalization() for <class
'torch.nn.modules.batchnorm.BatchNorm1d'>.
# Model Params: 24.62M FLOPs: 1.31G
```

```
Train Epoch: [1/1] Loss: 3.2581: 100%|      | 390/390 [02:54<00:00,
2.24it/s]
Feature extracting: 100%|      | 782/782 [00:50<00:00, 15.59it/s]
Test Epoch: [1/1] Acc@1:83.52% Acc@5:99.38%: 100%|      | 157/157
[00:12<00:00, 12.96it/s]
```

6 Finetune a Linear Layer for Classification!

Now it's time to put the representation vectors to the test!

We remove the projection head from the SimCLR model and slap on a linear layer to finetune for a simple classification task. All layers before the linear layer are frozen, and only the weights in the final linear layer are trained. We compare the performance of the SimCLR + finetuning model against a baseline model, where no self-supervised learning is done beforehand, and all weights in the model are trained. You will get to see for yourself the power of self-supervised learning and how the learned representation vectors improve downstream task performance.

6.1 Baseline: Without Self-Supervised Learning

First, let's take a look at the baseline model. We'll remove the projection head from the SimCLR model and slap on a linear layer to finetune for a simple classification task. No self-supervised learning is done beforehand, and all weights in the model are trained. Run the following cells.

NOTE: Don't worry if you see low but reasonable performance.

[16]:
```python
class Classifier(nn.Module):
    def __init__(self, num_class):
        super(Classifier, self).__init__()

        # Encoder.
        self.f = Model().f

        # Classifier.
        self.fc = nn.Linear(2048, num_class, bias=True)

    def forward(self, x):
        x = self.f(x)
        feature = torch.flatten(x, start_dim=1)
```

```
        out = self.fc(feature)
        return out
```

[17]:
```
# Do not modify this cell.
feature_dim = 128
temperature = 0.5
k = 200
batch_size = 128
epochs = 10
percentage = 0.1

train_transform = compute_train_transform()
train_data = CIFAR10(root='data', train=True, transform=train_transform,␣
 ↪download=True)
trainset = torch.utils.data.Subset(train_data, list(np.
 ↪arange(int(len(train_data)*percentage))))
train_loader = DataLoader(trainset, batch_size=batch_size, shuffle=True,␣
 ↪num_workers=16, pin_memory=True)
test_transform = compute_test_transform()
test_data = CIFAR10(root='data', train=False, transform=test_transform,␣
 ↪download=True)
test_loader = DataLoader(test_data, batch_size=batch_size, shuffle=False,␣
 ↪num_workers=16, pin_memory=True)

model = Classifier(num_class=len(train_data.classes)).to(device)
for param in model.f.parameters():
    param.requires_grad = False

flops, params = profile(model, inputs=(torch.randn(1, 3, 32, 32).to(device),))
flops, params = clever_format([flops, params])
print('# Model Params: {} FLOPs: {}'.format(params, flops))
optimizer = optim.Adam(model.fc.parameters(), lr=1e-3, weight_decay=1e-6)
no_pretrain_results = {'train_loss': [], 'train_acc@1': [], 'train_acc@5': [],
           'test_loss': [], 'test_acc@1': [], 'test_acc@5': []}

best_acc = 0.0
for epoch in range(1, epochs + 1):
    train_loss, train_acc_1, train_acc_5 = train_val(model, train_loader,␣
 ↪optimizer, epoch, epochs, device='cuda')
    no_pretrain_results['train_loss'].append(train_loss)
    no_pretrain_results['train_acc@1'].append(train_acc_1)
    no_pretrain_results['train_acc@5'].append(train_acc_5)
    test_loss, test_acc_1, test_acc_5 = train_val(model, test_loader, None,␣
 ↪epoch, epochs)
    no_pretrain_results['test_loss'].append(test_loss)
    no_pretrain_results['test_acc@1'].append(test_acc_1)
```

```
    no_pretrain_results['test_acc@5'].append(test_acc_5)
    if test_acc_1 > best_acc:
        best_acc = test_acc_1

# Print the best test accuracy.
print('Best top-1 accuracy without self-supervised learning: ', best_acc)
```

```
[INFO] Register count_convNd() for <class 'torch.nn.modules.conv.Conv2d'>.
[INFO] Register count_normalization() for <class
'torch.nn.modules.batchnorm.BatchNorm2d'>.
[INFO] Register zero_ops() for <class 'torch.nn.modules.activation.ReLU'>.
[INFO] Register zero_ops() for <class 'torch.nn.modules.container.Sequential'>.
[INFO] Register count_adap_avgpool() for <class
'torch.nn.modules.pooling.AdaptiveAvgPool2d'>.
[INFO] Register count_linear() for <class 'torch.nn.modules.linear.Linear'>.
# Model Params: 23.52M FLOPs: 1.31G
```

```
Train Epoch: [1/10] Loss: 2.5539 ACC@1: 10.72% ACC@5: 51.30%: 100%|      |
40/40 [00:09<00:00,  4.12it/s]
Test Epoch: [1/10] Loss: 2.3212 ACC@1: 11.48% ACC@5: 51.60%: 100%|      |
79/79 [00:13<00:00,  5.91it/s]
Train Epoch: [2/10] Loss: 2.4299 ACC@1: 10.88% ACC@5: 51.42%: 100%|      |
40/40 [00:11<00:00,  3.48it/s]
Test Epoch: [2/10] Loss: 2.7025 ACC@1: 10.18% ACC@5: 55.10%: 100%|      |
79/79 [00:13<00:00,  5.93it/s]
Train Epoch: [3/10] Loss: 2.3950 ACC@1: 11.70% ACC@5: 53.12%: 100%|      |
40/40 [00:08<00:00,  4.46it/s]
Test Epoch: [3/10] Loss: 2.5049 ACC@1: 10.24% ACC@5: 53.42%: 100%|      |
79/79 [00:12<00:00,  6.12it/s]
Train Epoch: [4/10] Loss: 2.4029 ACC@1: 12.44% ACC@5: 54.02%: 100%|      |
40/40 [00:10<00:00,  3.85it/s]
Test Epoch: [4/10] Loss: 2.5870 ACC@1: 10.34% ACC@5: 52.39%: 100%|      |
79/79 [00:12<00:00,  6.50it/s]
Train Epoch: [5/10] Loss: 2.4127 ACC@1: 12.24% ACC@5: 54.48%: 100%|      |
40/40 [00:09<00:00,  4.21it/s]
Test Epoch: [5/10] Loss: 2.7166 ACC@1: 14.82% ACC@5: 54.43%: 100%|      |
79/79 [00:13<00:00,  5.81it/s]
Train Epoch: [6/10] Loss: 2.3939 ACC@1: 12.44% ACC@5: 54.02%: 100%|      |
40/40 [00:10<00:00,  3.74it/s]
Test Epoch: [6/10] Loss: 2.3872 ACC@1: 13.67% ACC@5: 54.45%: 100%|      |
79/79 [00:12<00:00,  6.27it/s]
Train Epoch: [7/10] Loss: 2.3648 ACC@1: 13.10% ACC@5: 54.66%: 100%|      |
40/40 [00:09<00:00,  4.41it/s]
Test Epoch: [7/10] Loss: 2.4616 ACC@1: 11.80% ACC@5: 55.54%: 100%|      |
79/79 [00:13<00:00,  5.90it/s]
Train Epoch: [8/10] Loss: 2.3864 ACC@1: 11.86% ACC@5: 55.12%: 100%|      |
40/40 [00:10<00:00,  4.00it/s]
Test Epoch: [8/10] Loss: 2.4651 ACC@1: 14.32% ACC@5: 59.70%: 100%|      |
```

```
79/79 [00:12<00:00,  6.26it/s]
Train Epoch: [9/10] Loss: 2.3793 ACC@1: 13.22% ACC@5: 56.60%: 100%|      |
40/40 [00:09<00:00,  4.38it/s]
Test Epoch: [9/10] Loss: 2.6685 ACC@1: 10.05% ACC@5: 57.28%: 100%|      |
79/79 [00:13<00:00,  5.95it/s]
Train Epoch: [10/10] Loss: 2.4030 ACC@1: 12.96% ACC@5: 57.64%: 100%|      |
40/40 [00:10<00:00,  3.75it/s]
Test Epoch: [10/10] Loss: 2.4337 ACC@1: 15.30% ACC@5: 58.28%: 100%|      |
79/79 [00:12<00:00,  6.34it/s]

Best top-1 accuracy without self-supervised learning:  15.299999999999999
```

6.2 With Self-Supervised Learning

Let's see how much improvement we get with self-supervised learning. Here, we pretrain the SimCLR model using the simclr loss you wrote, remove the projection head from the SimCLR model, and use a linear layer to finetune for a simple classification task.

[18]:
```
# Do not modify this cell.
feature_dim = 128
temperature = 0.5
k = 200
batch_size = 128
epochs = 10
percentage = 0.1
pretrained_path = './pretrained_model/trained_simclr_model.pth'

train_transform = compute_train_transform()
train_data = CIFAR10(root='data', train=True, transform=train_transform,␣
 ↪download=True)
trainset = torch.utils.data.Subset(train_data, list(np.
 ↪arange(int(len(train_data)*percentage))))
train_loader = DataLoader(trainset, batch_size=batch_size, shuffle=True,␣
 ↪num_workers=16, pin_memory=True)
test_transform = compute_test_transform()
test_data = CIFAR10(root='data', train=False, transform=test_transform,␣
 ↪download=True)
test_loader = DataLoader(test_data, batch_size=batch_size, shuffle=False,␣
 ↪num_workers=16, pin_memory=True)

model = Classifier(num_class=len(train_data.classes))
model.load_state_dict(torch.load(pretrained_path, map_location='cpu'),␣
 ↪strict=False)
model = model.to(device)
for param in model.f.parameters():
    param.requires_grad = False
```

```
flops, params = profile(model, inputs=(torch.randn(1, 3, 32, 32).to(device),))
flops, params = clever_format([flops, params])
print('# Model Params: {} FLOPs: {}'.format(params, flops))
optimizer = optim.Adam(model.fc.parameters(), lr=1e-3, weight_decay=1e-6)
pretrain_results = {'train_loss': [], 'train_acc@1': [], 'train_acc@5': [],
           'test_loss': [], 'test_acc@1': [], 'test_acc@5': []}

best_acc = 0.0
for epoch in range(1, epochs + 1):
    train_loss, train_acc_1, train_acc_5 = train_val(model, train_loader,␣
 ↪optimizer, epoch, epochs)
    pretrain_results['train_loss'].append(train_loss)
    pretrain_results['train_acc@1'].append(train_acc_1)
    pretrain_results['train_acc@5'].append(train_acc_5)
    test_loss, test_acc_1, test_acc_5 = train_val(model, test_loader, None,␣
 ↪epoch, epochs)
    pretrain_results['test_loss'].append(test_loss)
    pretrain_results['test_acc@1'].append(test_acc_1)
    pretrain_results['test_acc@5'].append(test_acc_5)
    if test_acc_1 > best_acc:
        best_acc = test_acc_1

# Print the best test accuracy. You should see a best top-1 accuracy of >=70%.
print('Best top-1 accuracy with self-supervised learning: ', best_acc)
```

```
[INFO] Register count_convNd() for <class 'torch.nn.modules.conv.Conv2d'>.
[INFO] Register count_normalization() for <class
'torch.nn.modules.batchnorm.BatchNorm2d'>.
[INFO] Register zero_ops() for <class 'torch.nn.modules.activation.ReLU'>.
[INFO] Register zero_ops() for <class 'torch.nn.modules.container.Sequential'>.
[INFO] Register count_adap_avgpool() for <class
'torch.nn.modules.pooling.AdaptiveAvgPool2d'>.
[INFO] Register count_linear() for <class 'torch.nn.modules.linear.Linear'>.
# Model Params: 23.52M FLOPs: 1.31G

Train Epoch: [1/10] Loss: 1.8173 ACC@1: 65.26% ACC@5: 93.48%: 100%|      |
40/40 [00:08<00:00,  4.54it/s]
Test Epoch: [1/10] Loss: 1.3245 ACC@1: 77.96% ACC@5: 98.16%: 100%|      |
79/79 [00:12<00:00,  6.24it/s]
Train Epoch: [2/10] Loss: 1.1810 ACC@1: 76.14% ACC@5: 97.50%: 100%|      |
40/40 [00:10<00:00,  3.75it/s]
Test Epoch: [2/10] Loss: 0.9339 ACC@1: 79.23% ACC@5: 98.23%: 100%|      |
79/79 [00:12<00:00,  6.19it/s]
Train Epoch: [3/10] Loss: 0.9291 ACC@1: 76.42% ACC@5: 97.94%: 100%|      |
40/40 [00:08<00:00,  4.53it/s]
Test Epoch: [3/10] Loss: 0.7765 ACC@1: 79.85% ACC@5: 98.62%: 100%|      |
79/79 [00:12<00:00,  6.44it/s]
```

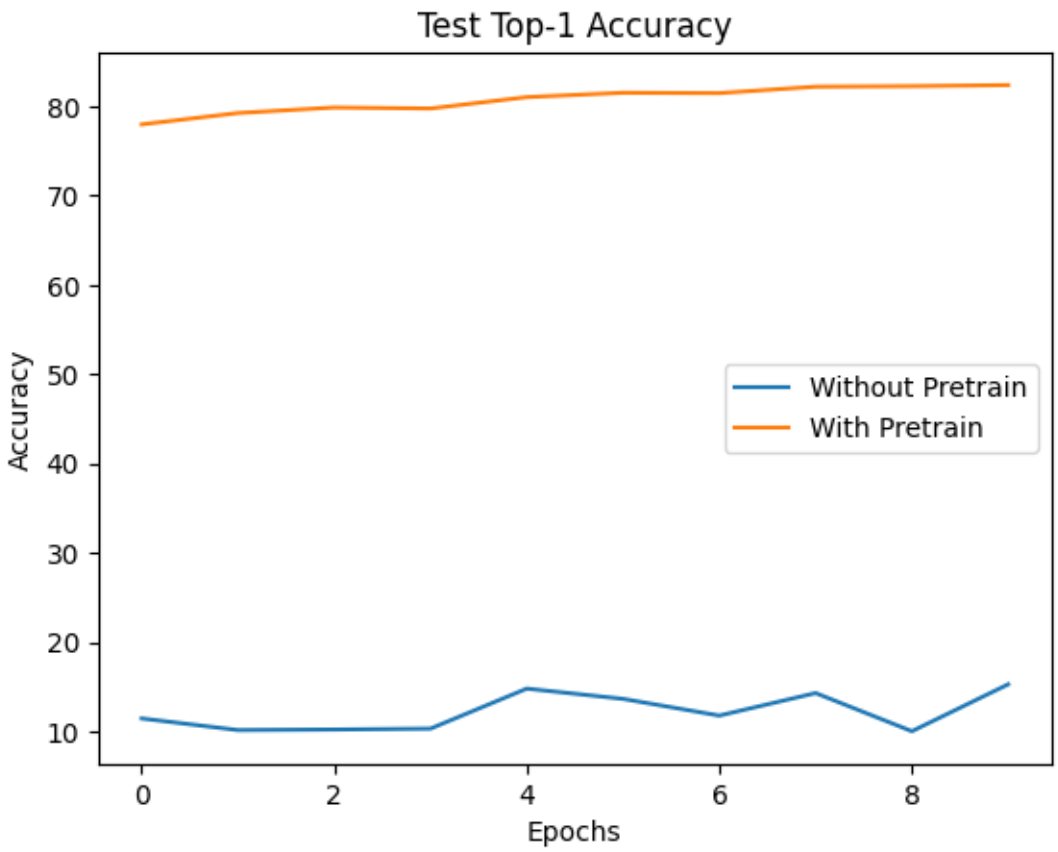
```
Train Epoch: [4/10] Loss: 0.8364 ACC@1: 77.06% ACC@5: 97.80%: 100%|      |
40/40 [00:10<00:00,  3.89it/s]
Test Epoch: [4/10] Loss: 0.7063 ACC@1: 79.74% ACC@5: 98.56%: 100%|      |
79/79 [00:11<00:00,  6.71it/s]
Train Epoch: [5/10] Loss: 0.7646 ACC@1: 77.90% ACC@5: 97.70%: 100%|      |
40/40 [00:09<00:00,  4.17it/s]
Test Epoch: [5/10] Loss: 0.6453 ACC@1: 81.01% ACC@5: 98.76%: 100%|      |
79/79 [00:13<00:00,  5.69it/s]
Train Epoch: [6/10] Loss: 0.7328 ACC@1: 77.40% ACC@5: 97.82%: 100%|      |
40/40 [00:11<00:00,  3.51it/s]
Test Epoch: [6/10] Loss: 0.6091 ACC@1: 81.50% ACC@5: 98.79%: 100%|      |
79/79 [00:12<00:00,  6.29it/s]
Train Epoch: [7/10] Loss: 0.6912 ACC@1: 78.52% ACC@5: 98.40%: 100%|      |
40/40 [00:08<00:00,  4.45it/s]
Test Epoch: [7/10] Loss: 0.5886 ACC@1: 81.46% ACC@5: 98.77%: 100%|      |
79/79 [00:12<00:00,  6.22it/s]
Train Epoch: [8/10] Loss: 0.6757 ACC@1: 78.36% ACC@5: 98.34%: 100%|      |
40/40 [00:11<00:00,  3.46it/s]
Test Epoch: [8/10] Loss: 0.5666 ACC@1: 82.18% ACC@5: 98.92%: 100%|      |
79/79 [00:12<00:00,  6.55it/s]
Train Epoch: [9/10] Loss: 0.6679 ACC@1: 78.74% ACC@5: 98.38%: 100%|      |
40/40 [00:09<00:00,  4.35it/s]
Test Epoch: [9/10] Loss: 0.5568 ACC@1: 82.24% ACC@5: 98.86%: 100%|      |
79/79 [00:13<00:00,  6.02it/s]
Train Epoch: [10/10] Loss: 0.6419 ACC@1: 79.44% ACC@5: 98.14%: 100%|     |
40/40 [00:10<00:00,  3.81it/s]
Test Epoch: [10/10] Loss: 0.5414 ACC@1: 82.36% ACC@5: 98.88%: 100%|      |
79/79 [00:12<00:00,  6.54it/s]

Best top-1 accuracy with self-supervised learning:  82.36
```

6.2.1 Plot your Comparison

Plot the test accuracies between the baseline model (no pretraining) and same model pretrained with self-supervised learning.

[19]:
```
plt.plot(no_pretrain_results['test_acc@1'], label="Without Pretrain")
plt.plot(pretrain_results['test_acc@1'], label="With Pretrain")
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.title('Test Top-1 Accuracy')
plt.legend()
plt.show()
```


Test Top-1 Accuracy
Accuracy
Epochs
Without Pretrain
With Pretrain
80
70
60
50
40
30
20
10
0
2
4
6
8

[ ]:

DDPM

November 14, 2026

Generative AI Use: For the purposes of the assignments, the use of generative AI is subject to the same policies regarding collaboration. Just as with other collaborators, each student must write down the solutions independently of the output of the interaction and the submission should include a note denoting the nature of the collaboration. The use of generative AI tools to substantially complete sections of the assignments is not in line with the spirit of the assignments, and would be a violation of the Honor Code.

[1]:
```
# This mounts your Google Drive to the Colab VM.
from google.colab import drive
drive.mount('/content/drive')

# TODO: Enter the foldername in your Drive where you have saved the unzipped
# assignment folder, e.g. 'cs231n/assignments/assignment3/'
FOLDERNAME = "cs231n/assignments/assignment3/"
assert FOLDERNAME is not None, "[!] Enter the foldername."

# Now that we've mounted your Drive, this ensures that
# the Python interpreter of the Colab VM can load
# python files from within it.
import sys
sys.path.append('/content/drive/My Drive/{}'.format(FOLDERNAME))

# # This downloads the Emoji dataset to your Drive
# # if it doesn't already exist.
# %cd /content/drive/My\ Drive/$FOLDERNAME/cs231n/datasets/
# !bash get_datasets.sh
# %cd /content/drive/My\ Drive/$FOLDERNAME
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call
drive.mount("/content/drive", force_remount=True).
```

1 Denoising Diffusion Probabilistic Models

So far, we have explored discriminative models, which are trained to produce a labeled output. These range from straightforward image classification to sentence generation where the problem was still framed as classification, with labels in vocabulary space and a recurrence mechanism capturing multi-word labels. Now, we will expand our repertoire by building a generative model capable of creating novel images that resemble a given set of training images.

There are many types of generative models, including Generative Adversarial Networks (GANs), autoregressive models, normalizing flow models, and Variational Autoencoders (VAEs), all of which can synthesize striking images. However, in 2020, Ho et al. introduced Denoising Diffusion Probabilistic Models (DDPMs) by combining diffusion probabilistic models with denoising score matching. This resulted in a generative model that is both simple to train and powerful enough to generate complex, high-quality images. The following provides a high-level overview of DDPMs. For more details, please refer to the course slides and the original DDPM paper [1].

2 Forward Process

Let $q(x_0)$ be the distribution of clean dataset images. We define a forward noising process as a Markov chain of small noising steps:

$$q(x_t|x_{t-1}) \sim N(x_t; \sqrt{1-\beta_t}x_{t-1}, \beta_t I)$$

where the stepwise variance $(\beta_1, ..., \beta_T)$ determines the noise schedule. Due to the properties of Gaussian distributions, we can express $q(x_t|x_0)$ in closed form as:

$$q(x_t|x_0) \sim N(x_t; \sqrt{\bar{\alpha}_t}x_0, (1-\bar{\alpha}_t)I)$$

where $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_t$. If the noise schedule $(\beta_1, ..., \beta_T)$ is set properly, the final distribution $q(x_T)$ becomes indistinguishable from pure Gaussian noise $N(0, I)$.

Recall that sampling from a Gaussian distribution $x \sim N(\mu, \sigma^2)$ is equivalent to computing $\sigma * \epsilon + \mu$ where $\epsilon \sim N(0, 1)$. Hence, sampling from $q(x_t|x_{t-1})$ or $q(x_t|x_0)$ is straightforward given x_{t-1} or x_0 respectively. Because of this, the forward process is simple and does not require learning.

3 Reverse Process

The reverse process reconstructs a clean image x_0 from pure noise x_T through multiple steps. Let $p(x_{t-1}|x_t)$ denote the reverse step of $q(x_t|x_{t-1})$. The first key insight is that learning to reverse each individual denoising step is easier than reversing the entire forward process in one go. In other words, learning $p(x_{t-1}|x_t)$ for each t is easier than directly learning $p(x_0|x_T)$.

However, learning $p(x_{t-1}|x_t)$ is still challenging. Although $q(x_t|x_{t-1})$ is Gaussian, $p(x_{t-1}|x_t)$ could take any complex form and is almost certainly not Gaussian. Modeling and sampling from an arbitrary distribution is significantly harder than working with a simple parametric distribution like a Gaussian.

The second key insight is that if the stepwise noise β_t in the forward process is small enough, then the reverse step $p(x_{t-1}|x_t)$ is also close to a Gaussian distribution. Thus, we only need to estimate its mean and variance. In practice, setting the variance of $p(x_{t-1}|x_t)$ to match β_t (the same as in the forward step) works well. Consequently, learning the reverse process reduces to learning the mean $\mu(x_t, t; \theta)$ where θ represents the parameters of a neural network.

4 Denoising Objective

Generative models are optimized by minimizing the expected negative log-likelihood $\mathbb{E}[-\log p_\theta(x_0)]$ of the dataset samples. The likelihood of each sample can be expressed as: $p_\theta(x_0) = p(x_T)\prod_{t=1}^{T} p(x_{t-1}|x_t)$. Since this objective is intractable in many cases, various classes of generative models optimize the variational lower bound on the negative log-likelihood.

Ho et al. demonstrated that this objective is equivalent to minimizing the following denoising loss

$$\mathbb{E}_{t,x_0,\epsilon}\left[\|\epsilon - \epsilon_\theta(\sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon, t)\|^2\right]$$

where t is uniform between 1 and T, x_0 is clean sample, ϵ is sampled from standard gaussian $N(0, I)$, and ϵ_θ is a neural network model trained to predict the noise ϵ from the input noisy sample $x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon$. In other words, ϵ_θ learns to denoise the input noisy image. Note that this is equivalent to predicting the clean sample, since the noise can be recovered from the noisy image and the clean sample following the equation $x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon$.

[1] Denoising Diffusion Probabilistic Models. Jonathan Ho, Ajay Jain, Pieter Abbeel. Link

5 In This Notebook...

We will implement and train a DDPM model to generate small 32 x 32 emoji images conditioned on text prompts. First, we will implement the forward noising process based on Eq. (4) of the paper [1]. Then we will build a UNet model that takes x_t and t as inputs (optionally with other conditioning like text-prompt) and outputs a tensor of the same shape as x_t. Finally, we will implement the denoising objective and train our DDPM model.

We use the text encoder from a pretrained CLIP[2] model to encode input text into a 512-dimensional vector. To speed up training, we've already pre-encoded the text data from the training set.

[2] Learning transferable visual models from natural language supervision. Radford et. al. Link

[2]:
```
!pip install git+https://github.com/openai/CLIP.git
```

```
Collecting git+https://github.com/openai/CLIP.git
  Cloning https://github.com/openai/CLIP.git to /tmp/pip-req-build-mhues7dm
  Running command git clone --filter=blob:none --quiet
https://github.com/openai/CLIP.git /tmp/pip-req-build-mhues7dm
  Resolved https://github.com/openai/CLIP.git to commit
d05afc436d78f1c48dc0dbf8e5980a9d471f35f6
  Installing build dependencies … done
  Getting requirements to build wheel … done
  Preparing metadata (pyproject.toml) … done
Requirement already satisfied: ftfy in /usr/local/lib/python3.13/dist-packages
(from clip==1.0) (6.3.1)
Requirement already satisfied: packaging in /usr/local/lib/python3.13/dist-
packages (from clip==1.0) (26.3)
Requirement already satisfied: regex in /usr/local/lib/python3.13/dist-packages
```

```
(from clip==1.0) (2025.11.3)
Requirement already satisfied: tqdm in /usr/local/lib/python3.13/dist-packages
(from clip==1.0) (4.67.3)
Requirement already satisfied: torch in /usr/local/lib/python3.13/dist-packages
(from clip==1.0) (2.14.1+cpu)
Requirement already satisfied: torchvision in /usr/local/lib/python3.13/dist-
packages (from clip==1.0) (0.29.1+cpu)
Requirement already satisfied: wcwidth in /usr/local/lib/python3.13/dist-
packages (from ftfy->clip==1.0) (0.9.4)
Requirement already satisfied: filelock in /usr/local/lib/python3.13/dist-
packages (from torch->clip==1.0) (4.2.7)
Requirement already satisfied: typing-extensions>=4.10.0 in
/usr/local/lib/python3.13/dist-packages (from torch->clip==1.0) (4.16.0)
Requirement already satisfied: setuptools>=77.0.3 in
/usr/local/lib/python3.13/dist-packages (from torch->clip==1.0) (80.10.2)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.13/dist-
packages (from torch->clip==1.0) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in
/usr/local/lib/python3.13/dist-packages (from torch->clip==1.0) (3.7)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.13/dist-packages
(from torch->clip==1.0) (3.1.6)
Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.13/dist-
packages (from torch->clip==1.0) (2025.12.0)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in
/usr/local/lib/python3.13/dist-packages (from sympy>=1.13.3->torch->clip==1.0)
(1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in
/usr/local/lib/python3.13/dist-packages (from jinja2->torch->clip==1.0) (3.0.4)
Requirement already satisfied: numpy in /usr/local/lib/python3.13/dist-packages
(from torchvision->clip==1.0) (2.4.6)
Requirement already satisfied: pillow!=8.3.*,>=5.3.0 in
/usr/local/lib/python3.13/dist-packages (from torchvision->clip==1.0) (11.3.0)
```

[3]:
```python
import sys
import types
import importlib

if "imp" not in sys.modules:
    imp = types.ModuleType("imp")
    imp.reload = importlib.reload
    sys.modules["imp"] = imp

%load_ext autoreload
%autoreload 2

import numpy as np
import torch
```

```
import random
import matplotlib.pyplot as plt
import torchvision.utils as tv_utils
from cs231n.emoji_dataset import EmojiDataset
from cs231n.gaussian_diffusion import GaussianDiffusion

def rel_error(x, y):
    """Returns relative error."""
    return np.max(np.abs(x - y) / (np.maximum(1e-10, np.abs(x) + np.abs(y))))
```

[4]:
```
# First, let's load and visualize the dataset.
# Each sample of the dataset is a tuple: (image, {"text_emb": <tensor>, "text": <string>})
# We will use a pretrained text-encoder to encode text into an embedding vector.
# We have pre-encoded the dataset texts into embeddings for faster training.
image_size = 32
dataset = EmojiDataset(image_size)
```

```
emoji_data.npz already downloaded.
text_embeddings.pt already downloaded.
```

[5]:
```
def visualize_samples(dataset, num_samples=25, grid_size=(5, 5)):
    # Randomly sample indices
    indices = random.sample(range(len(dataset)), num_samples)
    samples = [dataset[i] for i in indices]

    # Inspect one sample
    img_shape = list(samples[0][0].shape)
    emb_shape = list(samples[0][1]["text_emb"].shape)
    print(f"One sample: (image: {img_shape}, {{ \"text_emb\": {emb_shape}, \"text\": string }})")

    # Extract images and texts
    images = torch.stack([sample[0] for sample in samples])  # Stack images
    texts = [sample[1]["text"] for sample in samples]  # Extract text descriptions

    # Create a grid of images
    grid_img = tv_utils.make_grid(images, nrow=grid_size[1], padding=2)

    # Convert to numpy for plotting
    grid_img = grid_img.permute(1, 2, 0).numpy()

    # Plot the images
    fig, ax = plt.subplots(figsize=(10, 10))
    ax.imshow(grid_img)
    ax.axis("off")
```

```
    # Add text annotations
    grid_w, grid_h = grid_size
    img_w, img_h = grid_img.shape[1] // grid_w, grid_img.shape[0] // grid_h

    for i, text in enumerate(texts):
        row, col = divmod(i, grid_w)
        x, y = col * img_w, row * img_h
        ax.text(x+5, y+5, text[:30], fontsize=8, color='white',␣
↪bbox=dict(facecolor='black', alpha=0.5))

    plt.show()

visualize_samples(dataset)
```

```
One sample: (image: [3, 32, 32], { "text_emb": [512], "text": string })
```



5.1 q_sample

Now we will define the forward noising process. Read through the GaussianDiffusion class in `cs231n/gaussian_diffusion.py`. Consult the original DDPM paper[1] for the equations. Implement `q_sample` method and test it below. You should see zero relative error.

[6]:
```
# Test GaussianDiffusion.q_sample method
sz = 2
b = 3

diffusion = GaussianDiffusion(
      model=None,
      image_size=sz,
      timesteps=1000,
      beta_schedule="sigmoid",
)

t = torch.tensor([0, 300, 999]).long()
x_start = torch.linspace(-0.9, 0.6, b*3*sz*sz).view(b, 3, sz, sz)
noise = torch.linspace(-0.7, 0.8, b*3*sz*sz).view(b, 3, sz, sz)
x_t = diffusion.q_sample(x_start, t, noise)

expected_x_t = np.array([
    [
        [[-0.9119949, -0.86840147], [-0.8248081, -0.7812148]],
        [[-0.7376214, -0.694028], [-0.65043473, -0.6068413]],
        [[-0.563248, -0.51965463], [-0.47606122, -0.43246788]],
    ],
    [
        [[-0.42800453, -0.37039882], [-0.31279305, -0.2551873]],
        [[-0.19758154, -0.1399758], [-0.08237009, -0.024764337]],
        [[0.032841414, 0.090447165], [0.14805292, 0.20565866]],
    ],
    [
        [[0.32864183, 0.37152246], [0.41440308, 0.45728368]],
        [[0.50016433, 0.5430449], [0.5859255, 0.6288062]],
        [[0.67168677, 0.7145674], [0.757448, 0.8003287]],
    ],
]).astype(np.float32)

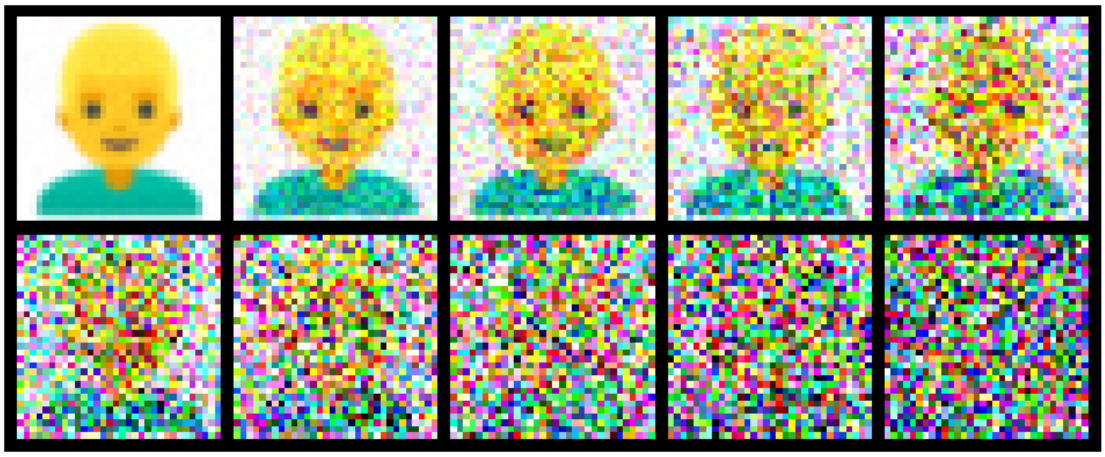
# Should see zero relative error
print("x_t error: ", rel_error(x_t.numpy(), expected_x_t))
```

```
x_t error:  0.0
```

[7]:
```
# Let's visualize the noisy images at various timesteps.
diffusion = GaussianDiffusion(
      model=None,
      image_size=image_size,
      timesteps=1000,
)

B = 10
img = dataset[770][0]  # 3 x H x W
x_start = img[None].repeat(B, 1, 1, 1)  # B x 3 x H x W
noise = torch.randn_like(x_start)  # B x 3 x H x W
t = torch.linspace(0, 1000-1, B).long()

x_start = diffusion.normalize(x_start)
x_t = diffusion.q_sample(x_start, t, noise)
x_t = diffusion.unnormalize(x_t).clamp(0, 1)
grid_img = tv_utils.make_grid(x_t, nrow=5, padding=2)
grid_img = grid_img.permute(1, 2, 0).cpu().numpy()
fig, ax = plt.subplots(figsize=(10, 10))
ax.imshow(grid_img)
ax.axis("off")
plt.show()
```



A diffusion model can be trained to predict either the clean image or the noise, as one can be derived from the other (explained in 'Denoising Objective' section above). Implement `predict_start_from_noise` and `predict_noise_from_start` methods and test them below. You should see relative error less than 1e-5.

[8]:
```
# Test `predict_noise_from_start` and `predict_start_from_noise`
sz = 2
b = 3
```

```
diffusion = GaussianDiffusion(
      model=None,
      image_size=sz,
      timesteps=1000,
      beta_schedule="sigmoid",
)

t = torch.tensor([1, 300, 998]).long()
x_start = torch.linspace(-0.91, 0.67, b*3*sz*sz).view(b, 3, sz, sz)
noise = torch.linspace(-0.73, 0.81, b*3*sz*sz).view(b, 3, sz, sz)
x_t = diffusion.q_sample(x_start, t, noise)

pred_noise = diffusion.predict_noise_from_start(x_t, t, x_start)
pred_x_start = diffusion.predict_start_from_noise(x_t, t, noise)

# Should relative errors around 1e-5 or less
print("noise error: ", rel_error(pred_noise.numpy(), noise.numpy()))
print("x_start error: ", rel_error(pred_x_start.numpy(), x_start.numpy()))
```

```
noise error:  1.0600407e-06
x_start error:  1.8902663e-06
```

5.2 UNet Model

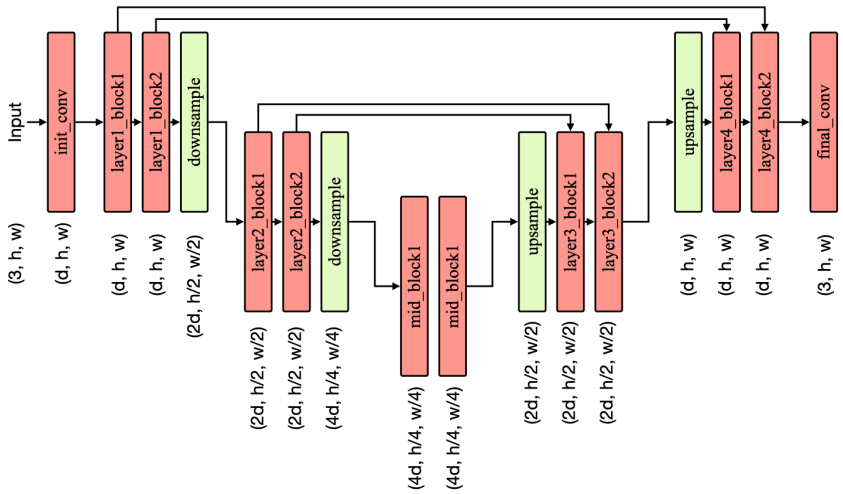
Now that we have defined the forward process, let's define the UNet model for denoising the input image. UNet is a neural network architecture designed for image-to-image tasks like segmentation, style transfer, etc. It consists of an encoder (or downsampling module) that transforms the input image into hierarchical features with decreasing spatial resolution and increasing feature dimensions. The decoder (or upsampling module) then upsamples the features by progressively restoring the spatial resolution, mirroring the encoder's structure. At each decoder layer, features from the corresponding encoder layer are concatenated, providing a direct pathway for finer details. This approach reduces the burden on the bottleneck layers, allowing them to focus on capturing high-level representations rather than memorizing fine details.

We will use UNet in this case because both our input and output are aligned images with same dimensions: C x H x W. Each ResNet block in the UNet will also take an additional input vector called context for conditioning. We will generate the context vector by encoding the diffusion timestep and text-prompt.

Run the cell below to get a rough outline of the UNet architecture. Each red box represents a ResNet block containing 2 or 3 convolutional layers that maintain the spatial resolution of the feature maps. The context vector input to every ResNet block is ommitted for clarity. The shape written below each box indicates the output tensor shape after that block. Additional arrows illustrate the skip connections, which enable the U-Net to preserve fine-grained details in the output. For example, the output of `layer1_block1` with shape (d, h, w) will be concatenated with the output of `layer4_block1`, also with shape (d, h, w), before being passed to `layer4_block2`. Therefore, `layer4_block2` will receive an input of shape (2*d, h, w).

[9]:
```
from IPython.display import Image
Image(f'/content/drive/My Drive/{FOLDERNAME}/unet.png')
```

[9]:



Implement the `Unet.__init__` method in `cs231n/unet.py` to define the upsampling and downsampling blocks of the UNet model, and then test it below. If your implementation is correct, you should not see any error. Calling `Unet(dim=d, condition_dim=condition_dim, dim_mults=(2,4))` should successfully create a UNet model corresponding to the architecture shown in the figure above.

[10]:
```
from cs231n.unet import Unet, ResnetBlock, Downsample, Upsample

dim = 2
condition_dim = 4
dim_mults = (1, 2, 4)
unet = Unet(dim=dim, condition_dim=condition_dim, dim_mults=dim_mults)


# Check number of layers
assert len(unet.downs) == len(dim_mults), "Number of Unet downsampling blocks␣
 ↪is wrong."
assert len(unet.ups) == len(dim_mults), "Number of Unet upsampling blocks is␣
 ↪wrong."


# Check layers
try:
  expected_downs_dims = [2, 2, 8, 2, 2, 8, 2, 2, 8, 2, 2, 8, 4, 4, 8, 4, 4, 8]
```

```
  downs_dims = [
      d for m in unet.downs for d in [
          m[0].dim, m[0].dim_out, m[0].context_dim, m[1].dim, m[1].dim_out,␣
 ↪m[1].context_dim,
      ]
  ]
  assert downs_dims == expected_downs_dims, "Dimensions don't match"
except Exception as e:
  raise RuntimeError("Downsampling blocks wrongly configured") from e


try:
  expected_ups_dims = [8, 4, 8, 8, 4, 8, 4, 2, 8, 4, 2, 8, 4, 2, 8, 4, 2, 8]
  ups_dims = [
      d for m in unet.ups for d in [
          m[1].dim, m[1].dim_out, m[1].context_dim, m[2].dim, m[2].dim_out,␣
 ↪m[2].context_dim,
      ]
  ]
  assert ups_dims == expected_ups_dims, "Dimensions don't match"
except Exception as e:
  raise RuntimeError("Upsampling blocks wrongly configured") from e

# Check number of parameters
num_params = sum(p.numel() for p in unet.parameters())
expected_num_params = 6499
assert num_params == expected_num_params, "Unet model creation is wrong!"
```

Fill in `Unet.forward` method in `cs231n/unet.py` and test it below. For now, don't worry about `Unet.cfg_forward` method. You should see relative error less than 1e-6.

[11]:
```
np.random.seed(231)
torch.manual_seed(231)

dim = 4
condition_dim = 4
dim_mults = (2, 4)
unet = Unet(dim=dim, condition_dim=condition_dim, dim_mults=dim_mults)
unet.eval()

b = 2
h = w = 4
inp_x = torch.randn(b, 3, h, w)
inp_text_emb = torch.randn(b, condition_dim)
inp_t = torch.tensor([8, 25]).long()
out = unet.forward(x=inp_x, time=inp_t,
                   model_kwargs={"text_emb": inp_text_emb}).detach().numpy()
```

```
expected_out = np.array(
      [[[[ 0.3530089,   0.49302575,  0.55152094,  0.20567769],
         [ 1.1295223,   0.05811183,  0.2143015,   0.23898259],
         [ 0.44839963,  1.0291728,   0.18519637,  0.21956968],
         [ 0.7035912,   0.716347,    0.9166326,  -0.11552593]],
        [[ 0.04585427, -0.26847702,  0.08526254,  0.2396591 ],
         [ 0.2858743,  -0.29726404,  0.02462834,  0.2350314 ],
         [ 0.3121434,   0.3962962,   0.65541625, -0.300183  ],
         [ 0.3409412,   0.35369393,  0.37445623,  0.10297439]],
        [[ 0.49440542,  0.72878885,  0.6669705,   0.56822944],
         [ 1.0084232,   0.19204213,  0.61240125,  0.575651  ],
         [ 0.6452671,   0.91168964,  0.6458106,   0.25952405],
         [ 0.7147298,   0.7707498,   0.88941365,  0.35422024]]],
       [[[ 0.3186316,  -0.17632714,  0.440247,    0.05124736],
         [ 0.6866561,   0.3019496,   0.18366373,  0.42941707],
         [-0.09776017,  0.55849403,  0.30689716,  0.12954405],
         [ 0.3667726,   0.740231,    0.43682802,  0.08275545]],
        [[-0.29714173, -0.28515863, -0.02289355,  0.05637485],
         [-0.10230345,  0.182563,    0.8107456,   0.18209176],
         [ 0.57062995,  0.07061654,  0.2864037,   0.17969263],
         [ 0.31491554,  0.5652035,   0.3810383,   0.05889529]],
        [[ 0.14559639,  0.36408228,  0.5657894,   0.24066216],
         [ 0.4431491,   0.38140664,  1.1457629,   0.40314198],
         [ 0.6673274,   0.5922724,   0.44549578,  0.5544277 ],
         [ 0.5405652,   0.88729244,  0.66340685,  0.23414826]]]])

print("forward error: ", rel_error(out, expected_out))
```

```
forward error:  4.342315411549735e-08
```

6 p_losses

Now that we have model implementation done, let's write the DDPM's denoising training step. As mentioned before, optimizing the denoising loss is equivalent to minimizing the expected negative log likelihood of the dataset. Complete the `GaussianDiffusion.p_losses` method in `cs231n/gaussian_diffusion.py` and test it below. You should see relative error less than 1e-6 .

[12]:
```
np.random.seed(231)
torch.manual_seed(231)

dim = 4
condition_dim = 4
dim_mults = (2, 4)
unet = Unet(dim=dim, condition_dim=condition_dim, dim_mults=dim_mults)
unet.eval()
```

```
h = w = 4
b = 3
diffusion = GaussianDiffusion(
      model=unet,
      image_size=h,
      timesteps=1000,
      beta_schedule="sigmoid",
      objective="pred_x_start",
)

inp_x = torch.randn(b, 3, h, w)
inp_model_kwargs = {"text_emb": torch.randn(b, condition_dim)}
out = diffusion.p_losses(inp_x, inp_model_kwargs)
expected_out = 30.0160

print("forward error: ", rel_error(out.item(), expected_out))
```

```
forward error:  7.432147198281673e-07
```

6.1 p_sample

There is one final ingredient remaining now. DDPM generates samples by iteratively performing the reverse process. Each iteration of this reverse process involves sampling from $p(x_{t-1}|x_t)$. Open `cs231n/gaussian_diffusion.py` and implement `p_sample` method by following Equation (6) from the paper. This equation describes sampling from the posterior of the forward process, conditioned on x_t and x_0, where x_0 can be derived from the denoising model's output. We have already implemented `sample` method that iteratively calls `p_sample` to generate images from input texts.

Test your implementation of `p_sample` below, you should see relative errors less than 1e-6.

[13]:
```
np.random.seed(231)
torch.manual_seed(231)

dim = 4
condition_dim = 4
dim_mults = (2,)
unet = Unet(dim=dim, condition_dim=condition_dim, dim_mults=dim_mults)
unet.eval()

h = w = 4
b = 1
inp_x_t = torch.randn(b, 3, h, w)
inp_model_kwargs = {"text_emb": torch.randn(b, condition_dim)}
t = 231

# test 1
diffusion = GaussianDiffusion(
      model=unet,
```

```
      image_size=h,
      timesteps=1000,
      beta_schedule="sigmoid",
      objective="pred_x_start",
)
out = diffusion.p_sample(inp_x_t, t, inp_model_kwargs).detach().numpy()

expected_out = np.array(
    [[[[ 1.1249855,   0.098786,   -0.674154,    1.3266845 ],
         [-0.21832949,  0.4874724,   1.0712924,  -0.7355726 ],
         [-0.19198017,  1.0347139,   0.03363481,  0.47500697],
         [-1.0178545,  -0.52259684, -0.11972852, -0.02105119]],
        [[ 0.06438226, -1.5796007,   0.43629852, -0.67392266],
         [-1.1564192,  -1.7009112,   0.878653,   -1.2563533 ],
         [-0.45194352,  1.0274173,   0.62859684, -0.49643248],
         [-0.18070872,  0.51325583, -0.74465716, -0.5262963 ]],
        [[ 0.76330817,  1.5878401,   1.9983996,   0.6618041 ],
         [ 0.16782297,  0.5638586,  -0.8150751,   1.9507835 ],
         [ 0.7401889,   1.2696184,   1.8468435,  -1.4796975 ],
         [ 0.6636643,  -0.83933365,  0.78857577,  0.13521622]]]])
print("forward error: ", rel_error(out, expected_out))

# test 2
diffusion = GaussianDiffusion(
      model=unet,
      image_size=h,
      timesteps=1000,
      beta_schedule="cosine",
      objective="pred_noise",
)
out = diffusion.p_sample(inp_x_t, t, inp_model_kwargs).detach().numpy()

expected_out = np.array(
    [[[[ 1.1139543,   0.11015256, -0.7337392,   1.3292073 ],
         [-0.286727,    0.48420003,  1.0737748,  -0.764173  ],
         [-0.22848499,  1.0443672,   0.07089197,  0.50183666],
         [-1.0453997,  -0.5474187,  -0.06630269, -0.11730157]],
        [[ 0.10919069, -1.5161378,   0.38542387, -0.6693473 ],
         [-1.0730051,  -1.6837747,   0.87086374, -1.2265388 ],
         [-0.48162907,  1.0328836,   0.6336425,  -0.47346604],
         [-0.11769794,  0.5335539,  -0.71324545, -0.47889465]],
        [[ 0.8311781,   1.6457081,   1.9775101,   0.563749  ],
         [ 0.25127694,  0.57862943, -0.8148284,   1.8744429 ],
         [ 0.75546896,  1.1215659,   1.9582558,  -1.535511  ],
         [ 0.62724066, -0.8814953,   0.77542645,  0.12667023]]]])
print("forward error: ", rel_error(out, expected_out))
```

```
forward error:  6.96344541990084e-08
```

```
forward error:  3.3032469825584936e-08
```

6.2 Training

We have all ingredients needed for DDPM training and we can train the model on our Emoji dataset. You don't have to code anything here but we encourage you to look at the training code at `cs231n/ddpm_trainer.py`.

For the rest of the notebook, we will use a pretrained model from `cs231n/exp/pretrained` folder which is already trained for many iterations on this dataset. However, you are free to train your own model on colab GPU (make sure to change the `results_folder`). Note that it may take more than 12 hours on T4 GPU before you start seeing a reasonable generation.

[14]:
```python
from cs231n.ddpm_trainer import Trainer

dim = 48
image_size = 32
results_folder = f"/content/drive/My Drive/{FOLDERNAME}/cs231n/exp/pretrained"
condition_dim = 512

device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")

model = Unet(
    dim=dim,
    dim_mults=(1, 2, 4, 8),
    condition_dim=condition_dim,
)
print("Number of parameters:", sum(p.numel() for p in model.parameters()))

diffusion = GaussianDiffusion(
    model,
    image_size=image_size,
    timesteps=100,  # number of diffusion steps
    objective="pred_noise",  # "pred_x_start" or "pred_noise"
)

dataset = EmojiDataset(image_size)

trainer = Trainer(
    diffusion,
    dataset,
    device,
    train_batch_size=256,
    weight_decay=0.0,
    train_lr=1e-3,
    train_num_steps=50000,
    results_folder=results_folder,
)
```

```
# You are not required to train it yourself.
# trainer.train()
```

```
Number of parameters: 12444963
emoji_data.npz already downloaded.
text_embeddings.pt already downloaded.
```

[15]:
```
# Instead, we will load a pretrained model.
trainer.download_pretrained()
trainer.load(70000)
```

```
Downloading…/content/drive/My
Drive/cs231n/assignments/assignment3//cs231n/exp/pretrained/model-70000.pt
Download complete.
loading model from /content/drive/My
Drive/cs231n/assignments/assignment3//cs231n/exp/pretrained/model-70000.pt.
```

[16]:
```
# Helper function to get CLIP text embedding during inference time.
from cs231n.emoji_dataset import ClipEmbed
clip_embedder = ClipEmbed(device)
def get_text_emb(text):
    return trainer.ds.embed_new_text(text, clip_embedder)

# Helper function to visualize generations.
def show_images(img):
    # img: B x T x 3 x H x W
    plt.figure(figsize=(10, 10))
    img2 = img.clamp(0, 1).permute(0, 3, 1, 4, 2).flatten(0, 1).flatten(1, 2).
 ↪cpu().numpy()
    plt.imshow(img2)
    plt.axis('off')

    plt.show()
```

```
100%|                    | 338M/338M [00:08<00:00, 43.0MiB/s]
/usr/local/lib/python3.13/dist-packages/torch/jit/_serialization.py:176:
FutureWarning: `torch.jit.load` is deprecated. Please switch to `torch.export`.
  warnings.warn(
```

6.3 Sampling

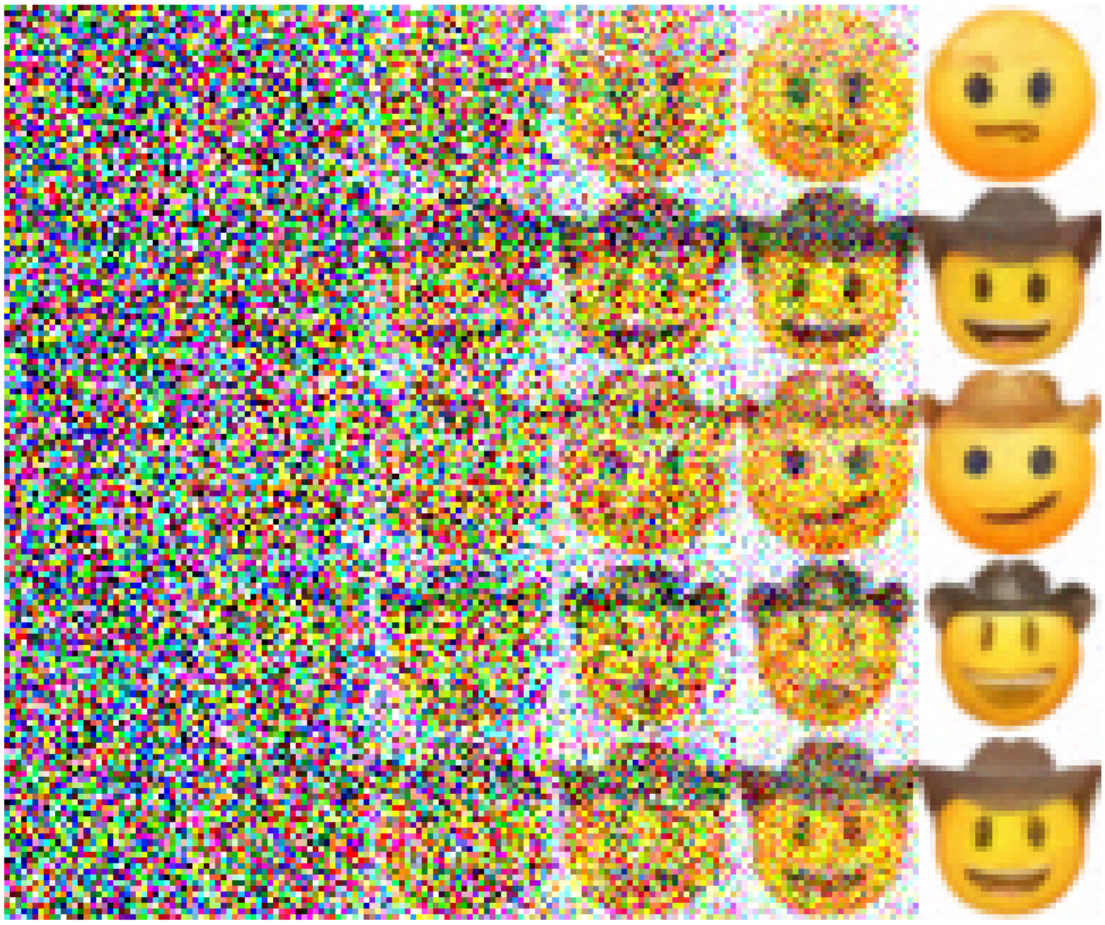
Run the cell below to visualize emoji generations conditioned on a text prompt. Feel free to modify the prompt to explore different generations. Since our emoji dataset is quite small, it is insufficient to train a fully generalizable text-to-image model. Because of that, generations for unseen prompts may be poor or may not be faithful to the input text (this may also happen for seen examples but less common).

For faster sampling, you may use a GPU runtime. If you switch the runtime type, be sure to rerun the entire notebook.

[17]:
```
# text = "crying face"  # seen example, good generations
text = "face with cowboy hat"  # seen example, good generations
# text = "crying face with cowboy hat"  # unseen example, bad generations
text_emb = get_text_emb(text)
text_emb = text_emb[None].expand(5, -1).to(device)


img = trainer.diffusion_model.sample(
    batch_size=5,
    model_kwargs={"text_emb": text_emb},
    return_all_timesteps=True
)
show_images(img[:, ::20])
```

```
sampling loop time step:   0%|          | 0/100 [00:00<?, ?it/s]
```



6.4 Classifier Free Guidance

Generative models are typically evaluated on fidelity (i.e., the quality or realism of the generated samples) and diversity (the variability or coverage of the sample space). For conditional generative models, fidelity additionally refers to how faithfully the generated samples adhere to the input conditions. These two metrics are often in tension with each other, leading to a trade-off between them. Ho et al. introduced a simple technique called classifier-free guidance [3], which allows explicit control over this trade-off.

In classifier-free guidance, during the training of a conditional diffusion model $\epsilon_\theta(x_t, t, c)$, the condition c is randomly dropped (i.e. replaced with $c = \phi$) with some probability (typically 0.1 to 0.2). During each denoising step of sampling, the prediction is updated as:

$$\epsilon_\theta(x_t, t, c) \leftarrow (w + 1)\epsilon_\theta(x_t, t, c) - w\epsilon_\theta(x_t, t, \phi)$$

where w is a positive scalar (the guidance scale). In other words, we perform two predictions during each denoising step, one conditional and one unconditional, and combine them linearly to favor the conditional generation. w is a hyperparameter that is tuned to optimize a model-specific evaluation metric. A higher w makes the generations more faithful to the condition but tends to reduce their diversity.

[3] Classifier-Free Diffusion Guidance. Jonathan Ho, Tim Salimans. Link

Implement the classifier-free guidance in `Unet.cfg_forward` method in `cs231n/unet.py` and test it below. You should see relative error less than 1e-6.

[18]:
```
np.random.seed(231)
torch.manual_seed(231)

dim = 4
condition_dim = 4
dim_mults = (2, 4)
unet = Unet(dim=dim, condition_dim=condition_dim, dim_mults=dim_mults)
unet.eval()

b = 2
h = w = 4
inp_x = torch.randn(b, 3, h, w)
inp_text_emb = torch.randn(b, condition_dim)
inp_t = torch.tensor([8, 25]).long()
out = unet.forward(x=inp_x, time=inp_t,
                   model_kwargs={"text_emb": inp_text_emb, "cfg_scale": 2.31}
                   ).detach().numpy()
expected_out = np.array(
      [[[[ 0.3063889,   0.49175858,  0.53406465,  0.17793494],
         [ 1.0605793,   0.07291106,  0.26333052,  0.2807781 ],
         [ 0.47007525,  1.0033083,   0.15596265,  0.21308547],
         [ 0.6635294,   0.69716704,  0.7332697,  -0.14285703]],
        [[-0.14788617, -0.17531544,  0.06930217,  0.24703366],
         [ 0.2998472,  -0.25145096,  0.06968095,  0.28924388],
         [ 0.311122,    0.35599577,  0.6346719,  -0.30859607],
```

```
         [ 0.3706389,   0.33177835,  0.29829532,  0.11159261]],
        [[ 0.34566414,  0.79019654,  0.6489645,   0.5683719 ],
         [ 0.98117805,  0.176887,    0.6220584,   0.6494926 ],
         [ 0.66813314,  0.90341353,  0.65361,     0.21999073],
         [ 0.73401093,  0.75344837,  0.71971107,  0.32417077]]],
       [[[ 0.46322435, -1.3915588,   1.0951779,   0.37392217],
         [ 0.6877824,   0.14802843, -0.7159017,   0.22476494],
         [-0.28032526,  0.50201464,  0.0972808,   0.23250426],
         [ 0.38375473,  0.86657,     0.4424622,   0.2161349 ]],
        [[-0.34920466, -0.48507357,  0.06529156,  0.29454708],
         [-0.62153226,  0.0083245,   1.7356712,  -0.0690068 ],
         [ 0.7745638,  -0.11548355,  0.237822,    0.22680345],
         [ 0.06337905,  0.7001358,   0.36441565,  0.17163567]],
        [[ 0.16040441, -0.2551211,   0.8887521,   0.11380547],
         [-0.12803352,  0.2783994,   1.4620805,   0.19044566],
         [ 0.9112923,   0.75666,     0.3765695,   0.66629255],
         [ 0.3461262,   1.0881201,   0.6392708,   0.31749305]]]])


print("forward error: ", rel_error(out, expected_out))
```

```
Classifier-free guidance scale: 2.31
forward error:  2.341653968553437e-07
```

Run the cell below to visualize emoji generations using classifier-free guidance. Feel free to modify the "cfg_scale" parameter value as well. As mentioned earlier, since our model does not generalize well, you may or may not observe faithful generations even with a high guidance scale.

[19]:
```
# text = "crying face"  # seen example, good generations
text = "face with cowboy hat"  # seen example, good generations
# text = "crying face with cowboy hat"  # unseen example, bad generations
text_emb = get_text_emb(text)
text_emb = text_emb[None].expand(5, -1).to(device)


img = trainer.diffusion_model.sample(
    batch_size=5,
    model_kwargs={"text_emb": text_emb, "cfg_scale": 1},
    return_all_timesteps=True
)
show_images(img[:, ::20])
```

```
sampling loop time step:   0%|          | 0/100 [00:00<?, ?it/s]
```

```
Classifier-free guidance scale: 1
```



[19]:

CLIP_DINO

November 14, 2026

Generative AI Use: For the purposes of the assignments, the use of generative AI is subject to the same policies regarding collaboration. Just as with other collaborators, each student must write down the solutions independently of the output of the interaction and the submission should include a note denoting the nature of the collaboration. The use of generative AI tools to substantially complete sections of the assignments is not in line with the spirit of the assignments, and would be a violation of the Honor Code.

[1]:
```
# This mounts your Google Drive to the Colab VM.
from google.colab import drive
drive.mount('/content/drive', force_remount=True)

# TODO: Enter the foldername in your Drive where you have saved the unzipped
# assignment folder, e.g. 'cs231n/assignments/assignment3/'
FOLDERNAME = "cs231n/assignments/assignment3/"
assert FOLDERNAME is not None, "[!] Enter the foldername."

# Now that we've mounted your Drive, this ensures that
# the Python interpreter of the Colab VM can load
# python files from within it.
import sys
sys.path.append('/content/drive/My Drive/{}'.format(FOLDERNAME))
```

```
Mounted at /content/drive
```

[2]:
```
# This downloads the COCO dataset to your Drive if it doesn't already exist
# (you should already have this dataset from a previous notebook!)
# Uncomment the following if you don't have it.
# %cd /content/drive/My\ Drive/$FOLDERNAME/cs231n/datasets/
# !bash get_coco_captioning.sh
# %cd /content/drive/My\ Drive/$FOLDERNAME
```

[3]:
```
# Some useful python libraries
! pip install ftfy regex tqdm
! pip install git+https://github.com/openai/CLIP.git
! pip install decord
```

```
Requirement already satisfied: ftfy in /usr/local/lib/python3.13/dist-packages
(6.3.1)
Requirement already satisfied: regex in /usr/local/lib/python3.13/dist-packages
```

```
(2025.11.3)
Requirement already satisfied: tqdm in /usr/local/lib/python3.13/dist-packages
(4.67.3)
Requirement already satisfied: wcwidth in /usr/local/lib/python3.13/dist-
packages (from ftfy) (0.9.4)
Collecting git+https://github.com/openai/CLIP.git
  Cloning https://github.com/openai/CLIP.git to /tmp/pip-req-build-hyjt4bfl
  Running command git clone --filter=blob:none --quiet
https://github.com/openai/CLIP.git /tmp/pip-req-build-hyjt4bfl
  Resolved https://github.com/openai/CLIP.git to commit
d05afc436d78f1c48dc0dbf8e5980a9d471f35f6
  Installing build dependencies … done
  Getting requirements to build wheel … done
  Preparing metadata (pyproject.toml) … done
Requirement already satisfied: ftfy in /usr/local/lib/python3.13/dist-packages
(from clip==1.0) (6.3.1)
Requirement already satisfied: packaging in /usr/local/lib/python3.13/dist-
packages (from clip==1.0) (26.3)
Requirement already satisfied: regex in /usr/local/lib/python3.13/dist-packages
(from clip==1.0) (2025.11.3)
Requirement already satisfied: tqdm in /usr/local/lib/python3.13/dist-packages
(from clip==1.0) (4.67.3)
Requirement already satisfied: torch in /usr/local/lib/python3.13/dist-packages
(from clip==1.0) (2.14.1+cu130)
Requirement already satisfied: torchvision in /usr/local/lib/python3.13/dist-
packages (from clip==1.0) (0.29.1+cu130)
Requirement already satisfied: wcwidth in /usr/local/lib/python3.13/dist-
packages (from ftfy->clip==1.0) (0.9.4)
Requirement already satisfied: filelock in /usr/local/lib/python3.13/dist-
packages (from torch->clip==1.0) (4.2.7)
Requirement already satisfied: typing-extensions>=4.10.0 in
/usr/local/lib/python3.13/dist-packages (from torch->clip==1.0) (4.16.0)
Requirement already satisfied: setuptools>=77.0.3 in
/usr/local/lib/python3.13/dist-packages (from torch->clip==1.0) (80.10.2)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.13/dist-
packages (from torch->clip==1.0) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in
/usr/local/lib/python3.13/dist-packages (from torch->clip==1.0) (3.7)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.13/dist-packages
(from torch->clip==1.0) (3.1.6)
Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.13/dist-
packages (from torch->clip==1.0) (2025.12.0)
Requirement already satisfied: cuda-toolkit==13.0.3 in
/usr/local/lib/python3.13/dist-packages (from cuda-toolkit[cublas,cudart,cufft,c
ufile,cupti,curand,cusolver,cusparse,nvjitlink,nvrtc,nvtx]==13.0.3;
platform_system == "Linux"->torch->clip==1.0) (13.0.3.0)
Requirement already satisfied: cuda-bindings<14,>=13.0.3 in
/usr/local/lib/python3.13/dist-packages (from torch->clip==1.0) (13.0.3)
```

```
Requirement already satisfied: nvidia-cudnn-cu13==9.24.0.43 in
/usr/local/lib/python3.13/dist-packages (from torch->clip==1.0) (9.24.0.43)
Requirement already satisfied: nvidia-cusparselt-cu13==0.8.1 in
/usr/local/lib/python3.13/dist-packages (from torch->clip==1.0) (0.8.1)
Requirement already satisfied: nvidia-nccl-cu13==2.30.7 in
/usr/local/lib/python3.13/dist-packages (from torch->clip==1.0) (2.30.7)
Requirement already satisfied: nvidia-nvshmem-cu13==3.4.5 in
/usr/local/lib/python3.13/dist-packages (from torch->clip==1.0) (3.4.5)
Requirement already satisfied: triton~=3.8.0 in /usr/local/lib/python3.13/dist-
packages (from torch->clip==1.0) (3.8.0)
Requirement already satisfied: nvidia-cublas==13.1.1.3.* in
/usr/local/lib/python3.13/dist-packages (from cuda-toolkit[cublas,cudart,cufft,c
ufile,cupti,curand,cusolver,cusparse,nvjitlink,nvrtc,nvtx]==13.0.3;
platform_system == "Linux"->torch->clip==1.0) (13.1.1.3)
Requirement already satisfied: nvidia-cuda-nvrtc==13.0.88.* in
/usr/local/lib/python3.13/dist-packages (from cuda-toolkit[cublas,cudart,cufft,c
ufile,cupti,curand,cusolver,cusparse,nvjitlink,nvrtc,nvtx]==13.0.3;
platform_system == "Linux"->torch->clip==1.0) (13.0.88)
Requirement already satisfied: nvidia-cuda-runtime==13.0.96.* in
/usr/local/lib/python3.13/dist-packages (from cuda-toolkit[cublas,cudart,cufft,c
ufile,cupti,curand,cusolver,cusparse,nvjitlink,nvrtc,nvtx]==13.0.3;
platform_system == "Linux"->torch->clip==1.0) (13.0.96)
Requirement already satisfied: nvidia-cufft==12.0.0.61.* in
/usr/local/lib/python3.13/dist-packages (from cuda-toolkit[cublas,cudart,cufft,c
ufile,cupti,curand,cusolver,cusparse,nvjitlink,nvrtc,nvtx]==13.0.3;
platform_system == "Linux"->torch->clip==1.0) (12.0.0.61)
Requirement already satisfied: nvidia-nvjitlink<14,>=13.0.88 in
/usr/local/lib/python3.13/dist-packages (from cuda-toolkit[cublas,cudart,cufft,c
ufile,cupti,curand,cusolver,cusparse,nvjitlink,nvrtc,nvtx]==13.0.3;
platform_system == "Linux"->torch->clip==1.0) (13.4.92)
Requirement already satisfied: nvidia-cufile==1.15.1.6.* in
/usr/local/lib/python3.13/dist-packages (from cuda-toolkit[cublas,cudart,cufft,c
ufile,cupti,curand,cusolver,cusparse,nvjitlink,nvrtc,nvtx]==13.0.3;
platform_system == "Linux"->torch->clip==1.0) (1.15.1.6)
Requirement already satisfied: nvidia-cuda-cupti==13.0.85.* in
/usr/local/lib/python3.13/dist-packages (from cuda-toolkit[cublas,cudart,cufft,c
ufile,cupti,curand,cusolver,cusparse,nvjitlink,nvrtc,nvtx]==13.0.3;
platform_system == "Linux"->torch->clip==1.0) (13.0.85)
Requirement already satisfied: nvidia-curand==10.4.0.35.* in
/usr/local/lib/python3.13/dist-packages (from cuda-toolkit[cublas,cudart,cufft,c
ufile,cupti,curand,cusolver,cusparse,nvjitlink,nvrtc,nvtx]==13.0.3;
platform_system == "Linux"->torch->clip==1.0) (10.4.0.35)
Requirement already satisfied: nvidia-cusolver==12.0.4.66.* in
/usr/local/lib/python3.13/dist-packages (from cuda-toolkit[cublas,cudart,cufft,c
ufile,cupti,curand,cusolver,cusparse,nvjitlink,nvrtc,nvtx]==13.0.3;
platform_system == "Linux"->torch->clip==1.0) (12.0.4.66)
Requirement already satisfied: nvidia-cusparse==12.6.3.3.* in
/usr/local/lib/python3.13/dist-packages (from cuda-toolkit[cublas,cudart,cufft,c
```

```
ufile,cupti,curand,cusolver,cusparse,nvjitlink,nvrtc,nvtx]==13.0.3;
platform_system == "Linux"->torch->clip==1.0) (12.6.3.3)
Requirement already satisfied: nvidia-nvtx==13.0.85.* in
/usr/local/lib/python3.13/dist-packages (from cuda-toolkit[cublas,cudart,cufft,c
ufile,cupti,curand,cusolver,cusparse,nvjitlink,nvrtc,nvtx]==13.0.3;
platform_system == "Linux"->torch->clip==1.0) (13.0.85)
Requirement already satisfied: cuda-pathfinder~=1.1 in
/usr/local/lib/python3.13/dist-packages (from cuda-
bindings<14,>=13.0.3->torch->clip==1.0) (1.8.5)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in
/usr/local/lib/python3.13/dist-packages (from sympy>=1.13.3->torch->clip==1.0)
(1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in
/usr/local/lib/python3.13/dist-packages (from jinja2->torch->clip==1.0) (3.0.4)
Requirement already satisfied: numpy in /usr/local/lib/python3.13/dist-packages
(from torchvision->clip==1.0) (2.4.6)
Requirement already satisfied: pillow!=8.3.*,>=5.3.0 in
/usr/local/lib/python3.13/dist-packages (from torchvision->clip==1.0) (11.3.0)
Requirement already satisfied: decord in /usr/local/lib/python3.13/dist-packages
(0.6.0)
Requirement already satisfied: numpy>=1.14.0 in /usr/local/lib/python3.13/dist-
packages (from decord) (2.4.6)
```

1 State-of-the-Art Pretrained Image Models

In the previous exercise, you learned about SimCLR and how contrastive self-supervised learning can be used to learn meaningful image representations. In this notebook, we will explore two more recent models that also aim to learn high-quality visual representations and have demonstrated strong and robust performance on a variety of downstream tasks.

First, we will examine the CLIP model. Like SimCLR, CLIP uses a contrastive learning objective, but instead of contrasting two augmented views of the same image, it contrasts two different modalities: text and image. To train CLIP, OpenAI collected a large dataset of ~400M image-text pairs from the internet, including sources like Wikipedia and image alt text. The resulting model learns rich, high-level image features and has achieved impressive zero-shot performance on many vision benchmarks.

Next, we will explore DINO, a self-supervised learning method for vision tasks that applies contrastive learning in a self-distillation framework with multi-crop augmentation strategy. The authors showed that the features learned by DINO ViTs are fine-grained and semantically rich with explicit information about the semantic segmentation of the image.

2 CLIP

As explained above, CLIP's training objective incorporates both text and images, building upon the principles of contrastive learning. Consider this quote from the SimCLR notebook: >The goal of the contrastive loss is to maximize agreement between the final vectors $z_i = g(h_i)$ and $z_j = g(h_j)$.

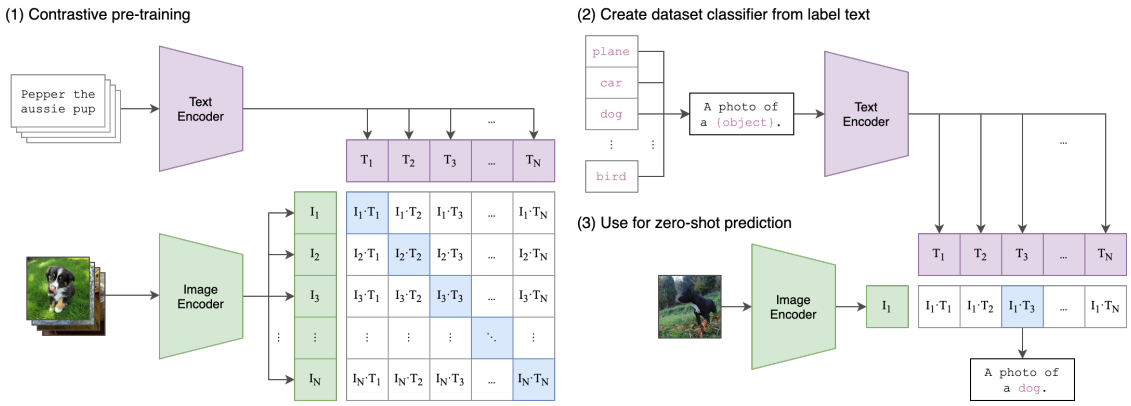
Similarly, CLIP is trained to maximize agreement between two vectors. However, because these vectors come from different modalities, CLIP uses two separate encoders: a transformer-based Text Encoder and a Vision Transformer (ViT)-based Image Encoder. Note that some smaller, more efficient versions of CLIP use a ResNet as the Image Encoder instead of a ViT.

Run the cell below to visualize the training and inference pipeline of CLIP.

During the pretraining phase, each batch consists of multiple images along with their corresponding captions. Each image is independently processed by an Image Encoder—typically a visual model like a Vision Transformer (ViT) or a Convolutional Neural Network (ConvNet)—which produces an image embedding I_n. Likewise, each caption is independently processed by a Text Encoder to generate a corresponding text embedding T_n. Next, we compute the pairwise similarities between all image-text combinations, meaning each image is compared with every caption, and vice versa. The training objective is to maximize the similarity scores along the diagonal of the resulting similarity matrix – that is, the scores for the matching image-caption pairs (I_n, T_n). Through backpropagation, the model learns to assign higher similarity scores to true matches than to mismatched pairs.

Through this setup, CLIP effectively learns to represent images and texts in a shared latent space. In this space, semantic concepts are encoded in a modality-independent way, enabling meaningful cross-modal comparisons between visual and textual inputs.

[4]:
```
from IPython.display import Image as ColabImage
ColabImage(f'/content/drive/My Drive/{FOLDERNAME}/CLIP.png')
```

[4]:


Inline Question 1 -

Why does CLIP's learning depend on the batch size? If the batch size is fixed, what strategy can we use to learn rich image features?

Your Answer :

CLIP uses other samples in the batch as negative examples, so a larger batch provides more image-text comparisons and improves learning. If the batch size is fixed, we can use **hard negative mining** to select challenging mismatched image-text pairs, helping the model learn richer and more meaningful image features.

3 Loading COCO dataset

We'll use the same captioning dataset you used to train your RNN captioning model, but instead of generating the captions lets see if we can match each image to the correct caption.

[5]:
```
import sys
import types
import importlib

if "imp" not in sys.modules:
    imp = types.ModuleType("imp")
    imp.reload = importlib.reload
    sys.modules["imp"] = imp

%load_ext autoreload
%autoreload 2

import time, os, json
import numpy as np
import matplotlib.pyplot as plt
import torch
import clip
import torch
from tqdm.auto import tqdm

from PIL import Image
from cs231n.clip_dino import *

def rel_error(x, y):
    """Returns relative error."""
    return np.max(np.abs(x - y) / (np.maximum(1e-10, np.abs(x) + np.abs(y))))
```

[6]:
```
from cs231n.coco_utils import load_coco_data, sample_coco_minibatch,␣
↪decode_captions
from cs231n.image_utils import image_from_url
```

[7]:
```
# Load COCO data from disk into a dictionary.
# this is the same dataset you used for the RNN captioning notebook :)
data = load_coco_data(pca_features=True)

# Print out all the keys and values from the data dictionary.
for k, v in data.items():
    if type(v) == np.ndarray:
        print(k, type(v), v.shape, v.dtype)
    else:
        print(k, type(v), len(v))
```

```
base dir  /content/drive/My
Drive/cs231n/assignments/assignment3/cs231n/datasets/coco_captioning
train_captions <class 'numpy.ndarray'> (400135, 17) int32
train_image_idxs <class 'numpy.ndarray'> (400135,) int32
val_captions <class 'numpy.ndarray'> (195954, 17) int32
val_image_idxs <class 'numpy.ndarray'> (195954,) int32
train_features <class 'numpy.ndarray'> (82783, 512) float32
val_features <class 'numpy.ndarray'> (40504, 512) float32
idx_to_word <class 'list'> 1004
word_to_idx <class 'dict'> 1004
train_urls <class 'numpy.ndarray'> (82783,) <U63
val_urls <class 'numpy.ndarray'> (40504,) <U63
```

[8]:
```
# we're just using the loaded captions from COCO, so we need to decode them and␣
 ↪get rid of the special tokens.
decoded_captions= []
for caption in data['val_captions']:
  caption = decode_captions(caption, data['idx_to_word'])\
    .replace('<START>', '')\
    .replace('<END>', '')\
    .replace('<UNK>', '')\
    .strip()
  decoded_captions.append(caption)
```

[9]:
```
# lets get 10 examples
mask = np.array([135428, 122586, 122814, 133173, 176639, 163828,  98169,   6931,
        19488, 175760])
first_captions = [decoded_captions[elem] for elem in mask]

img_idxs = data['val_image_idxs'][mask]       # the images the captions refer to
first_images   = [image_from_url(data['val_urls'][j]) for j in img_idxs]
```

[10]:
```
for i, (caption, image) in enumerate(zip(first_captions, first_images)):
    plt.imshow(image)
    plt.axis('off')
    caption_str = caption
    plt.title(caption_str)
    plt.show()
```

```
Output hidden; open in https://colab.research.google.com to view.
```

4 Running the CLIP Model

First we'll use the pretrained CLIP model to extract features from the texts and images separetely.

[11]:
```
device = "cuda" if torch.cuda.is_available() else "cpu"
clip_model, clip_preprocess = clip.load("ViT-B/32", device=device)
```

```
/usr/local/lib/python3.13/dist-packages/torch/jit/_serialization.py:176:
FutureWarning: `torch.jit.load` is deprecated. Please switch to `torch.export`.
  warnings.warn(
```

[12]:
```python
# You can check the model layers by printing the model.
# CLIP's model code is available at https://github.com/openai/CLIP/tree/main/
 ↪clip
# print(clip_model)
```

[13]:
```python
# First, we encode the captions into vectors in the shared embedding space.
# Since we're using a Transformer as the text encoder, we need to tokenize the␣
 ↪text first.
text_tokens = clip.tokenize(first_captions).to(device)
with torch.no_grad():
    text_features = clip_model.encode_text(text_tokens)

# Sanity check, print the shape
print(text_features.shape)

# Note: For your implementations, use the above clip_model.encode_text()␣
 ↪function. Avoid using clip_model.forward().
```

```
torch.Size([10, 512])
```

[14]:
```python
# Then, we encode the images into the same embedding space.
processed_images = [
    clip_preprocess(Image.fromarray(img)).unsqueeze(0)
    for img in first_images
]
images_tensor = torch.cat(processed_images, dim=0).to(device)

with torch.no_grad():
    image_features = clip_model.encode_image(images_tensor)

# sanity check, print the shape
print(image_features.shape)

# Note: For your implementations, use the above clip_model.encode_image()␣
 ↪function. Avoid using clip_model.forward().
```

```
torch.Size([10, 512])
```

Open `cs231n/clip_dino.py` and implement `get_similarity_no_loop` to compute similarity scores between text features and image features. Test your implementation below, you should see relative errors less than 1e-5.

[15]:
```python
from cs231n.clip_dino import get_similarity_no_loop
torch.manual_seed(231)
np.random.seed(231)
```

```
M, N, D = 5, 6, 10

test_text_features = torch.randn(N, D)
test_image_features = torch.randn(M, D)
out = get_similarity_no_loop(test_text_features, test_image_features)

expected_out = np.array([
    [ 0.1867811 , -0.23494351,  0.44155994, -0.18950461,  0.00100103],
    [ 0.17905031, -0.25469488, -0.64330417,  0.25097957, -0.09327742],
    [-0.4407011 , -0.4365381 ,  0.32857686, -0.3765278 ,  0.01049389],
    [ 0.24815483,  0.42157224, -0.08459304,  0.14132318, -0.26935193],
    [ 0.02309848, -0.01441101,  0.5469337 ,  0.6018773 ,  0.21581158],
    [ 0.41579214, -0.014449  , -0.7242257 ,  0.39348006,  0.0822239 ],
]).astype(np.float32)

print("relative error: ", rel_error(out.numpy(), expected_out))
```

```
relative error:  4.8262887e-06
```

[16]:
```
# Let's visualize the similarities between our batch of images and their␣
 ↪captions.

similarities = get_similarity_no_loop(text_features, image_features).cpu().
 ↪detach().numpy()

plt.figure(figsize=(20, 14))
plt.imshow(similarities, vmin=0.1, vmax=0.3)
plt.yticks(range(len(text_features)), first_captions, fontsize=18)
plt.xticks([])
for i, image in enumerate(first_images):
    plt.imshow(image, extent=(i - 0.5, i + 0.5, -1.6, -0.6), origin="lower")
for x in range(similarities.shape[1]):
    for y in range(similarities.shape[0]):
        plt.text(x, y, f"{similarities[y, x]:.2f}", ha="center", va="center",␣
 ↪size=12)

for side in ["left", "top", "right", "bottom"]:
    plt.gca().spines[side].set_visible(False)

plt.xlim([-0.5, len(image_features) - 0.5])
plt.ylim([len(text_features) + 0.5, -2])

plt.title("Cosine similarity between text and image features", size=20)
plt.show()
```



5 Zero Shot Classifier

You will be able to see a high similarity between matching image-caption pairs above. We can leverage this property to design an image classifier that doesn't require any labeled data (i.e., a zero-shot classifier). Each class can be represented using an appropriate natural language description, and any input image will be classified into the class whose description has the highest similarity with the image in CLIP's embedding space.

Implement `clip_zero_shot_classifier` in `cs231n/clip_dino.py` and test it below. You should be able to see the following predictions:

['a person', 'an animal', 'an animal', 'food', 'a person', 'a landscape', 'other', 'other', 'other', 'a person']

[17]:
```
from cs231n.clip_dino import clip_zero_shot_classifier

classes = ["a person", "an animal", "food", "a landscape", "other"]

pred_classes = clip_zero_shot_classifier(
    clip_model, clip_preprocess, first_images, classes, device)

print(pred_classes)
```

```
['a person', 'an animal', 'an animal', 'food', 'a person', 'a landscape',
'other', 'an animal', 'other', 'a person']
```

Run the cell below to visualize the predictions. As you can see, CLIP offers a straightforward way to perform reasonable zero-shot classification across any class taxonomy.

CLIP was the first model to outperform standard supervised training on ImageNet classification without using any ImageNet images or labels (The original CLIP paper has many such interesting experiments and analysis).

[18]:
```
# Visualize the zero shot predictions
for i, (pred_class, image) in enumerate(zip(pred_classes, first_images)):
    plt.imshow(image)
    plt.axis('off')
    plt.title(pred_class)
    plt.show()
```

```
Output hidden; open in https://colab.research.google.com to view.
```

6 Image Retrieval using CLIP

Just as we used CLIP to retrieve the matching class name for each image, we can also use it to retrieve matching images from text inputs (semantic image retrieval). Implement the `CLIPImageRetriever` in `cs231n/clip_dino.py` and test it by running the two cells below. The expected top 2 outputs for each query are provided in the comments.

[19]:
```
from cs231n.clip_dino import CLIPImageRetriever
clip_retriever = CLIPImageRetriever(clip_model, clip_preprocess, first_images,␣
 ↪device)
```

[20]:
```
query = "sports"  # tennis, skateboard
# query = "black and white"  # bathroom, zerbas
img_indices = clip_retriever.retrieve(query)

for img_index in img_indices:
    plt.imshow(first_images[img_index])
    plt.axis('off')
    plt.show()
```





Inline Question 2 -

CLIP learns to align image and text representations in a shared latent space using a contrastive loss. How would you extend this idea to more than two modalities?

YourAnswer :

We can extend CLIP by using separate encoders for each modality (image, text, audio, video) and mapping their representations into a shared latent space. Then, use a contrastive loss to bring matching samples from different modalities closer while pushing unrelated samples apart.

7 DINO

As mentioned earlier, models trained with vanilla contrastive learning methods such as SimCLR and CLIP require very large batch sizes. This makes them computationally expensive and limits their accessibility. Subsequent works, like BYOL, propose an alternative approach that avoids the need for numerous negative samples by using a student-teacher framework. This method performs surprisingly well and was later adopted by DINO .

Similar to SimCLR, DINO is trained to maximize the agreement between two vectors derived from different views of the same image. However, unlike SimCLR, DINO uses two separate encoders which are trained differently. The student network is updated via backpropagation to match the outputs of the teacher network. The teacher network is not updated via backpropagation; instead, its weights are updated using an exponential moving average (EMA) of the student's weights. This means that the teacher model evolves more slowly and provides a stable target for the student to learn from.

Run the cell below to visualize the DINO training pipeline.

[21]:
```
from IPython.display import Image as ColabImage
ColabImage(f'/content/drive/My Drive/{FOLDERNAME}/dino.gif')
```

```
Output hidden; open in https://colab.research.google.com to view.
```

[22]:
```
# first let's get rid of the CLIP model that's currently using memory
del clip_model
# Uncomment the following if you are using GPU runtime
# torch.cuda.empty_cache()
# torch.cuda.ipc_collect()
```

[23]:
```
# Load smallest dino model. ViT-S/8. Here ViT-S has ~22M parameters and
# works on 8x8 patches.
dino_model = torch.hub.load('facebookresearch/dino:main', 'dino_vits8')
dino_model.eval().to(device)
```

```
Using cache found in /root/.cache/torch/hub/facebookresearch_dino_main
```

[23]:
```
VisionTransformer(
  (patch_embed): PatchEmbed(
    (proj): Conv2d(3, 384, kernel_size=(8, 8), stride=(8, 8))
  )
  (pos_drop): Dropout(p=0.0, inplace=False)
```

```
    (blocks): ModuleList(
      (0-11): 12 x Block(
        (norm1): LayerNorm((384,), eps=1e-06, elementwise_affine=True, bias=True)
        (attn): Attention(
          (qkv): Linear(in_features=384, out_features=1152, bias=True)
          (attn_drop): Dropout(p=0.0, inplace=False)
          (proj): Linear(in_features=384, out_features=384, bias=True)
          (proj_drop): Dropout(p=0.0, inplace=False)
        )
        (drop_path): Identity()
        (norm2): LayerNorm((384,), eps=1e-06, elementwise_affine=True, bias=True)
        (mlp): Mlp(
          (fc1): Linear(in_features=384, out_features=1536, bias=True)
          (act): GELU(approximate='none')
          (fc2): Linear(in_features=1536, out_features=384, bias=True)
          (drop): Dropout(p=0.0, inplace=False)
        )
      )
    )
    (norm): LayerNorm((384,), eps=1e-06, elementwise_affine=True, bias=True)
    (head): Identity()
  )
```

[24]:
```
# the image we will be playing around with
sample_image = Image.fromarray(first_images[0]).convert("RGB")
sample_image
```

[24]:



8 DINO Attention Maps

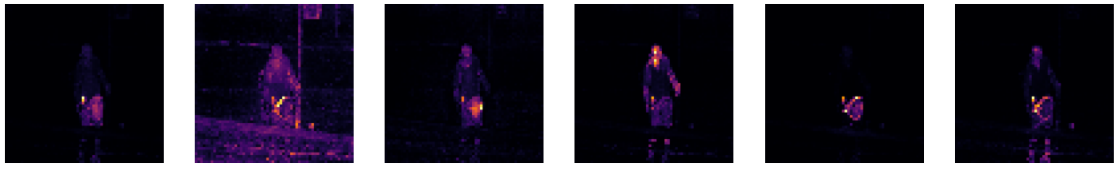
Since the loaded DINO checkpoint is based on the ViT architecture, we can visualize what each attention head is focusing on. The code below generates heatmaps showing which patches of the original image the [CLS] token attends to across the various heads in the final layer. Although this model was trained using a self-supervised objective without any explicit instruction to recognize "structure" in images, still…

Do you notice any patterns?

[25]:
```
# Preprocess
from torchvision import transforms as T
transform = T.Compose([
    T.Resize((480, 480)),
    T.ToTensor(),
    T.Normalize((0.485, 0.456, 0.406), (0.229, 0.224, 0.225)),
])
img_tensor = transform(sample_image)
w, h = img_tensor.shape[1:]
img_tensor = img_tensor[None].to(device)

# Extract attention
with torch.no_grad():
    attn = dino_model.get_last_selfattention(img_tensor)[0, :, 0, 1:]
nh, tokens = attn.shape
w_feat, h_feat = w // 8, h // 8
attn = attn.reshape(nh, w_feat, h_feat)
attn = torch.nn.functional.interpolate(attn.unsqueeze(0), scale_factor=8,␣
 ↪mode="nearest")[0].cpu().numpy()

# Plot attention heads
fig, axes = plt.subplots(1, nh, figsize=(3 * nh, 3))
for i in range(nh):
    ax = axes[i] if nh > 1 else axes
    ax.imshow(attn[i], cmap='inferno')
    ax.axis('off')
plt.show()
```



[26]:
```
# Extract patch token features and discard [CLS] token.
with torch.no_grad():
    all_tokens = dino_model.get_intermediate_layers(img_tensor, n=1)[0]  # (1,␣
 ↪1+N, D)
    patch_tokens = all_tokens[:, 1:, :]  # (N, D)

print(img_tensor.shape)
print(all_tokens.shape)
print(patch_tokens.shape)
```

```
torch.Size([1, 3, 480, 480])
torch.Size([1, 3601, 384])
torch.Size([1, 3600, 384])
```

Inline Question 3

How do we get the tensor shapes printed above? Explain your answer.

YourAnswer :

1. `(1, 3, 480, 480)` – This is our input image.
   - `1` means one image in the batch.
   - `3` represents the RGB channels.
   - `480 × 480` is the resized image dimensions.
2. `(1, 3601, 384)` – This represents the image tokens.
   - `3600` patch tokens (60×60 patches).
   - `1` extra `[CLS]` token.
   - `384` is the embedding size of each token.
3. `(1, 3600, 384)` – Same as above, but with the `[CLS]` token removed. It contains only the image patch tokens.

9 DINO Features

To understand what the model is encoding in each patch, we can visualize the contents of each patch token. Since these embeddings are high-dimensional and difficult to interpret directly, we'll use PCA to identify the directions of highest variance in the feature space.

In the next cell, we visualize the three principal directions of variance in the feature space. This reveals the dominant structure that the patch embeddings are capturing.

[27]:
```
from sklearn.decomposition import PCA

np.random.seed(231)

# PCA
pca = PCA(n_components=3)
patch_pca = pca.fit_transform(patch_tokens.cpu().numpy()[0])
```

```
# Normalize PCA components to [0, 1] for RGB display
patch_rgb = (patch_pca - patch_pca.min(0)) / (patch_pca.max(0) - patch_pca.
  ↪min(0))

# Reshape to image grid (60x60, 3)
patch_rgb_img = patch_rgb.reshape(60, 60, 3)

# Show as image
plt.figure(figsize=(6, 6))
plt.imshow(patch_rgb_img)
plt.axis('off')
plt.title("Patch Embeddings (PCA → RGB)")
plt.show()
```



Inline Question 4 -

What kind of structure do you see in the visualization above? What does it imply when a region consistently appears in a specific color? What does it mean when two regions have distinctly

different color? Remember that PCA reveals the directions of highest variance in the feature space across all patches. A patch's color reflects its distinct feature content.

YourAnswer :

The visualization shows that patches belonging to similar regions often have similar colors, meaning they have similar features. If a region consistently has one color, DINO has learned similar representations for that area. Different colors indicate that the regions have different visual features or belong to different objects or structures.

10 A Simple Segmentation Model over DINO Features

In the previous section, we saw that DINO features can provide surprisingly good segmentation cues. Now, let's put that idea to the test by training a simple segmentation model on the DAVIS dataset. The DAVIS dataset (Densely Annotated VIdeo Segmentation) was created for video object segmentation tasks. It provides frame-by-frame, pixel-level annotations of objects within videos. For this experiment, we'll train our model using the annotations from just a single frame of a video and see how well it performs on the remaining frames of the same.

Our model will be intentionally minimal: we'll extract DINO features per patch and train a lightweight per-patch classifier using only the patches from that one annotated frame. Typically, you would train on the full dataset and evaluate on a separate validation set containing different videos. But here, we will test the one-shot capabilities of DINO features.

[30]:
```
import sys
!{sys.executable} -m pip install -U importlib_resources
```

```
Collecting importlib_resources
  Downloading importlib_resources-7.1.0-py3-none-any.whl.metadata (4.0 kB)
Downloading importlib_resources-7.1.0-py3-none-any.whl (37 kB)
Installing collected packages: importlib_resources
Successfully installed importlib_resources-7.1.0
```

[31]:
```
import importlib_resources
print("Installation successful!")
```

```
Installation successful!
```

[32]:
```
from cs231n.clip_dino import DavisDataset

# A helper class to work with DAVIS dataset.
# It may take ~5 minutes on the first run of this cell to download the dataset.
davis_ds = DavisDataset()

# Get a specific test video. Do NOT change this for submission.
frames, masks = davis_ds.get_sample(7)
num_classes = masks.max() + 1

print(frames.shape, masks.shape, num_classes)
```

```
WARNING:absl:Variant folder /root/tensorflow_datasets/davis/480p/2.1.0 has no
dataset_info.json

Downloading and preparing dataset Unknown size (download: Unknown size,
generated: Unknown size, total: Unknown size) to
/root/tensorflow_datasets/davis/480p/2.1.0…

Dl Completed…: 0 url [00:00, ? url/s]

Dl Size…: 0 MiB [00:00, ? MiB/s]

Extraction completed…: 0 file [00:00, ? file/s]

Generating splits…:   0%|          | 0/2 [00:00<?, ? splits/s]

Generating train examples…: 0 examples [00:00, ? examples/s]

Shuffling /root/tensorflow_datasets/davis/480p/incomplete.3RZ82S_2.1.0/
  ↪davis-train.tfrecord-[0-9][0-9][0-9][0-…

Generating validation examples…: 0 examples [00:00, ? examples/s]

Shuffling /root/tensorflow_datasets/davis/480p/incomplete.3RZ82S_2.1.0/
  ↪davis-validation.tfrecord-[0-9][0-9][0-…

WARNING:absl:`FeatureConnector.dtype` is deprecated. Please change your code to
use NumPy with the field `FeatureConnector.np_dtype` or use TensorFlow with the
field `FeatureConnector.tf_dtype`.
WARNING:absl:`FeatureConnector.dtype` is deprecated. Please change your code to
use NumPy with the field `FeatureConnector.np_dtype` or use TensorFlow with the
field `FeatureConnector.tf_dtype`.

Dataset davis downloaded and prepared to
/root/tensorflow_datasets/davis/480p/2.1.0. Subsequent calls will reuse this
data.
video soapbox  99 frames
(99, 480, 854, 3) (99, 480, 854, 1) 4
```

[33]:
```
# Get DINO patch features and corresponding class labels for a middle frame
train_fi = 40
X_train = davis_ds.process_frames(frames[train_fi:train_fi+1], dino_model,␣
  ↪device)[0]
Y_train = davis_ds.process_masks(masks[train_fi:train_fi+1], device)[0]
print(X_train.shape, Y_train.shape)
```

```
torch.Size([3600, 384]) torch.Size([3600])
```

Complete the implementation of the `DINOSegmentation` class in `cs231n/clip_dino.py`, and test it by running the two cells below. You should achieve a mean IoU greater than 0.45 on the first test frame and greater than 0.50 on the last test frame. To prevent overfitting on the training patch features, consider designing a very lightweight model (e.g., a linear layer or a 2-layer MLP) and applying appropriate weight decay.

You may use GPU runtime to speed up training and evaluation. Make sure to rerun the entire notebook if you change runtime type.

[34]:
```
from cs231n.clip_dino import DINOSegmentation, compute_iou
torch.manual_seed(231)
np.random.seed(231)
dino_segmentation = DINOSegmentation(device, num_classes)
dino_segmentation.train(X_train, Y_train, num_iters=500)


# Test on first, middle, and last frame
ious = []
test_fis = [0, train_fi, 98]
gt_viz = []
pred_viz = []
for fi in test_fis:
  X_test = davis_ds.process_frames(frames[fi:fi+1], dino_model, device)[0]
  Y_test = davis_ds.process_masks(masks[fi:fi+1], device)[0]
  Y_pred = dino_segmentation.inference(X_test)
  iou = compute_iou(Y_pred, Y_test, num_classes)
  ious.append(iou)

  gt_viz.append(davis_ds.mask_frame_overlay(Y_test, frames[fi]))
  pred_viz.append(davis_ds.mask_frame_overlay(Y_pred, frames[fi]))

gt_viz = np.concatenate(gt_viz, 1)
pred_viz = np.concatenate(pred_viz, 1)
```

```
Training:   0%|          | 0/500 [00:00<?, ?it/s]
```

[35]:
```
print(f"Mean IoU on first test frames: {ious[0]:.3f}")  # should be >0.45
print(f"Mean IoU on last test frames: {ious[2]:.3f}")  # should be >0.50
```

```
Mean IoU on first test frames: 0.483
Mean IoU on last test frames: 0.563
```

Now let's visualize the results. Run the two cells below to display the ground truth and predicted segmentation masks for the first, middle, and last frames. Note that the middle frame is part of the training set, while the other frames are unseen.

[36]:
```
Image.fromarray(gt_viz)
```

[36]:



[37]:
```
Image.fromarray(pred_viz)
```

[37]:



Now run the following three cells to evaluate and visualize the entire video. You should achieve a mean IoU greater than 0.55. The saved visualization video may take some time to process in Google Drive, but you can download it to your computer and view it locally.

[38]:
```
# Run on all frames
ious = []
gt_viz = []
pred_viz = []
for fi in range(len(frames)):
  if fi % 20 == 0:
    print(f"{fi} / {len(frames)}")
  X_test = davis_ds.process_frames(frames[fi:fi+1], dino_model, device)[0]
  Y_test = davis_ds.process_masks(masks[fi:fi+1], device)[0]
  Y_pred = dino_segmentation.inference(X_test)
  iou = compute_iou(Y_pred, Y_test, num_classes)
  ious.append(iou)

  gt_viz.append(davis_ds.mask_frame_overlay(Y_test, frames[fi]))
  pred_viz.append(davis_ds.mask_frame_overlay(Y_pred, frames[fi]))

gt_viz = np.stack(gt_viz)  # T x H x W x 3
pred_viz = np.stack(pred_viz)  # T x H x W x 3
final_viz = np.concatenate([gt_viz, pred_viz], -2)  # T x H x 2W x 3
```

```
0 / 99
20 / 99
40 / 99
60 / 99
80 / 99
```

[39]:
```
print(f"Mean IoU on all frames: {sum(ious) / len(ious):.3f}")  # should be >0.55
```

```
Mean IoU on all frames: 0.672
```

[40]:
```
def write_video_from_array(array, output_path, fps = 12):
    T, H, W, _ = array.shape
```

```
    fourcc = cv2.VideoWriter_fourcc(*'mp4v')
    out = cv2.VideoWriter(output_path, fourcc, fps, (W, H))
    for i in range(T):
        frame = array[i]
        out.write(frame)
    out.release()
    print(f"Video saved to {output_path}")


# It might take a while to process in google drive but you can just download it␣
 ↪and watch on your computer
write_video_from_array(final_viz, f"/content/drive/My Drive/{FOLDERNAME}/
 ↪dino_res.mp4")
```

```
Video saved to /content/drive/My
Drive/cs231n/assignments/assignment3//dino_res.mp4
```

Inline Question 5 -

If you train a segmentation model on CLIP ViT's patch features, do you expect it to perform better or worse than DINO? Why should that be the case?

YourAnswer :

CLIP would likely perform worse than DINO because CLIP is trained mainly to align images with text, while DINO learns detailed visual features through self-supervised learning. DINO captures local structures, boundaries, and object regions better, making its patch features more suitable for segmentation.

```
[ ]:
```